



23 DE JULIO DE 2026

APUNTES DE FUNDAMENTOS DE PROGRAMACIÓN I

GRADO EN INGENIERÍA INFORMÁTICA Y EN GRADO EN CIENCIA E INGENIERÍA DE DATOS

ZENÓN JOSÉ HERNÁNDEZ FIGUEROA. JOSÉ DANIEL GONZÁLEZ DOMÍNGUEZ
Y JUAN CARLOS RODRÍGUEZ DEL PINO

ESCUELA DE INGENIERÍA INFORMÁTICA
ULPGC



Contenido

Tema 1	Conceptos básicos de informática	3
Procesamiento de la información		3
Informática		3
¿Qué es un ordenador?		3
Tipos de ordenadores		4
Hardware		4
Despiece general		4
Placa base		5
Microprocesador		6
Memoria RAM		6
Unidades de almacenamiento		7
Conectividad y comunicaciones		9
Software		10
Software de sistema		10
Software de aplicación		11
Más sobre datos		12
Bases de datos		12
Anexo: elementos, funciones y terminología básica del ordenador		15
Caracteres que representan símbolos usados en lenguajes de programación		15
Teclas especiales de un teclado		16
Algunos elementos, funciones y terminología básica del ordenador		20
Escribiendo texto en el ordenador		21
Atajos de teclado		22
Ventana		22
Diálogos y campos de diálogo		23
Terminal y consola		23
Gestión de archivos y carpetas		23
IDE (Integrated Development Environment)		24
Tema 2	Conceptos básicos de programación	26
Programas y algoritmos		26
Procesamiento de la información		26
Algoritmos		26
Programa y lenguaje		27
Python		28
¡Hola, mundo!		29

¡Hola, mundo!	29
Help().....	30
Secuencias de instrucciones.....	31
Comentarios	32
Entrada de datos, variables	32
Entrada de datos	32
Los nombres de las variables.....	34
Los valores de las variables	34
Asignación	35
Asignación múltiple	36
Entrada de valores no textuales	37
Tipos de datos, operadores.....	38
Tipos de datos	38
Gestión de tipos de datos.....	42
Operadores.....	44
Anexo: Resumen de reglas de estilo PEP 8 de escritura de código en Python.....	47
Depuración de programas.....	50
Depuración	50
Depuración web (Python tutor)	51
Depuración con un depurador gráfico (Thonny)	54
Depuración con el depurador integrado de Python (pdb)	59
Tema 3 Introducción a la programación estructurada	64
Estructuras de control	64
Condiciones	64
Ejecución alternativa	65
Ejecución repetitiva	68
Funciones y Módulos	71
Funciones	71
Funciones con parámetros	72
Funciones que devuelven un resultado	74
Funciones polimórficas.....	75
Anotación de los tipos de los parámetros de una función.....	76
Módulos y paquetes.....	77
Dónde escribir una función	77
Módulos	78
El módulo __main__	79

Paquetes	80
Tema 4 Secuencias.....	84
Tipos de datos secuencia. Contenedores secuenciales.....	84
Secuencias y contenedores secuenciales	84
Valores de tipo str, list y tuple	84
Operaciones básicas de los contenedores secuenciales.	85
Esquemas de iteración secuencial.....	88
Esquema general de recorrido	88
Recorrido con tratamiento selectivo (esquema de selección)	91
Búsqueda lineal (esquema de búsqueda)	92
Búsqueda del máximo/mínimo de una secuencia	94
Esquema acumulador.....	95
Acumulador selectivo	96
Esquema contador	96
Mutable e inmutable.....	96
Operaciones que modifican listas	96
Consecuencias de la mutabilidad	98
Más esquemas de tratamiento	101
Esquemas de procesamiento sincronizado de varias secuencias	103
Procesamiento sincronizado de varias secuencias.....	103
Procesamiento sincronizado usando zip	103
Procesamiento hasta la longitud de la más larga usando zip_longest.....	104
Procesamiento hasta la longitud de la más larga usando índices	105
Estructuras multidimensionales.....	106
Secuencias de secuencias.....	106
Tablas y matrices	106
Tratando secuencias de secuencias.....	107
Tratando matrices.....	108
ANEXO: Operaciones de secuencias, contenedores y listas	110
Operaciones de listas	111
Tema 5 Recursividad	112
Solución de problemas mediante diseño descendente	112
Solución de problemas mediante recursividad	113
¿Qué es la recursividad?.....	113
Caracterización de un problema recursivo	114
Identificación de los factores que determinan el tamaño de un problema	114

Paso del problema a una versión más pequeña.....	115
Implementación de la solución	116
Análisis de las soluciones recursivas	117
Estructuras de datos recursivas.....	120
Vista recursiva de una estructura de datos	120
Estructuras de datos construidas recursivamente	121
Inmersión recursiva	122
Tema 6 Contenedores asociativos	124
Diccionarios	124
Operaciones en diccionarios	125
Eliminar una pareja clave:valor de un diccionario	125
El método get().....	126
Diccionarios como valores de diccionarios	127
Conjuntos	127
Modificación de conjuntos	128
Pertenencia e inclusión	130
Álgebra de conjuntos.....	130
Iterando en contenedores asociativos	131
Tratamiento de contenedores asociativos	133
Tema 7 Strings de caracteres.....	138
Operaciones de strings.....	138
Strings.....	138
Secuencias de escape	138
Mayúsculas y minúsculas	139
Categorías de caracteres	140
Acceso a los componentes de una string	140
Concatenación de strings	141
Localización de una substring.....	141
El método join()	143
El método split()	143
El método replace()	144
Recorte y relleno: los métodos strip(), lstrip(), rstrip(), ljust(), rjust(), center() y zfill().....	144
Formateo de strings	145
Texto con formato	145
Interpolación de strings: f-strings.....	145
Modificadores de formato.....	146

Modificadores de formato - ancho de campo.....	147
Modificadores de formato - Precisión.....	148
Modificadores de formato - Notación numérica.....	149
El método format().....	150
Tema 8 Expresiones regulares	152
Búsqueda de patrones	152
Lenguaje de expresiones regulares	152
Secuencias de escape	153
Metacaracteres.....	154
Uso de expresiones regulares en Python	158
Preparación	158
Operaciones	159
Anexo – Chuleta de expresiones regulares	163
Tema 9 Archivos de texto	164
Archivos de texto.....	164
Apertura y cierre de un archivo.....	165
Tratamiento de los datos de un archivo	166
Esquema estándar de tratamiento de un fichero de texto en modo lectura.....	166
Tratamiento explícito como secuencia de líneas en Python	167
Leer un fichero en una única string.....	168
Esquema de escritura.....	168
Cláusula <i>with</i>	169
Anexo – resumen de las operaciones de manejo de ficheros de texto.....	170
Anexo - el módulo <i>os</i>	170
Glosario de términos de Fundamentos de Programación I.....	172
Glosario alfabético.....	172
Términos complementarios de alta recurrencia	183
Operaciones y herramientas de consulta rápida.....	187

Fundamentos

Tema 1 Conceptos básicos de informática

"Los ordenadores son como el Anillo Único. Tiene que meter usted en ellos una buena parte de su vigor nativo hasta dominar sus triquiñuelas y, hecho esto, se supone que las máquinas aumentarán sus capacidades hasta el infinito y más allá" - **Juan Manuel Grijalvo**

Procesamiento de la información

Informática

El diccionario de la RAE define la **informática** como el "conjunto de conocimientos científicos y técnicas que hacen posible el tratamiento automático de la información mediante **ordenadores**".

Wikipedia define la **información** como un "conjunto organizado de **datos** procesados, que constituyen un mensaje que cambia el estado de conocimiento del sujeto o sistema que recibe dicho mensaje", mientras que WordReference la define como "conjunto de datos sobre una materia determinada".

Wikipedia define un **dato** como "una representación simbólica (numérica, alfabética, algorítmica, espacial, etc.) de un atributo o variable cuantitativa o cualitativa".

Por ejemplo, la información sobre una persona puede estar compuesta de datos como: nombre, apellidos, fecha de nacimiento, sexo, dirección, estado civil, ocupación, etc. Algunos datos, como la edad, pueden calcularse a partir de otros, como la fecha de nacimiento y la fecha actual.

El tratamiento de la información plantea **problemas** cuyo propósito es hallar un dato desconocido a partir de otros datos conocidos. Los ordenadores se utilizan para solucionar problemas de tratamiento de información. Ejemplos de problemas sencillos:

1. Dadas la base y la altura de un triángulo, calcular su superficie.
2. A partir de los datos de partidos jugados, ganados y empatados por los equipos de una liga deportiva, listar la clasificación.

¿Qué es un ordenador?

Un **ordenador**, o computadora, es un dispositivo electrónico diseñado para procesar datos y realizar cálculos a gran velocidad. Funciona ejecutando programas que le permiten realizar tareas específicas. Los componentes principales de un ordenador son: el **hardware**, las partes físicas tangibles, y el **software**, los programas que indican al hardware qué hacer.

La diferencia principal entre un ordenador y un sistema informático radica en el alcance y la composición de ambos términos. Un ordenador abarca exclusivamente el hardware y software básicos necesarios para realizar funciones de cómputo.

Un **sistema informático** es un conjunto más amplio que incluye no solo el ordenador, sino también todos los elementos necesarios para que funcione dentro de un contexto y con un propósito específico. Esto incluye ordenadores, tecnologías de comunicaciones (hardware y software), datos, usuarios y procedimientos.

Tipos de ordenadores

Se pueden diferenciar diversos tipos de ordenadores: servidores, PCs (computadoras personales), laptops, etc. La diferencia entre ellos radica en su diseño, propósito, capacidad y forma de uso.

Un **servidor** es un ordenador diseñado para proporcionar servicios y gestionar recursos para otros dispositivos (clientes) en una red: gestiona las redes, almacena grandes volúmenes de datos, ejecuta aplicaciones corporativas o alberga servicios en la nube. Se caracteriza por un alto rendimiento, gran capacidad de almacenamiento y memoria RAM, alta disponibilidad (normalmente 24/7) y conectividad, estando optimizado para trabajar en redes, con múltiples puertos y opciones de redundancia.

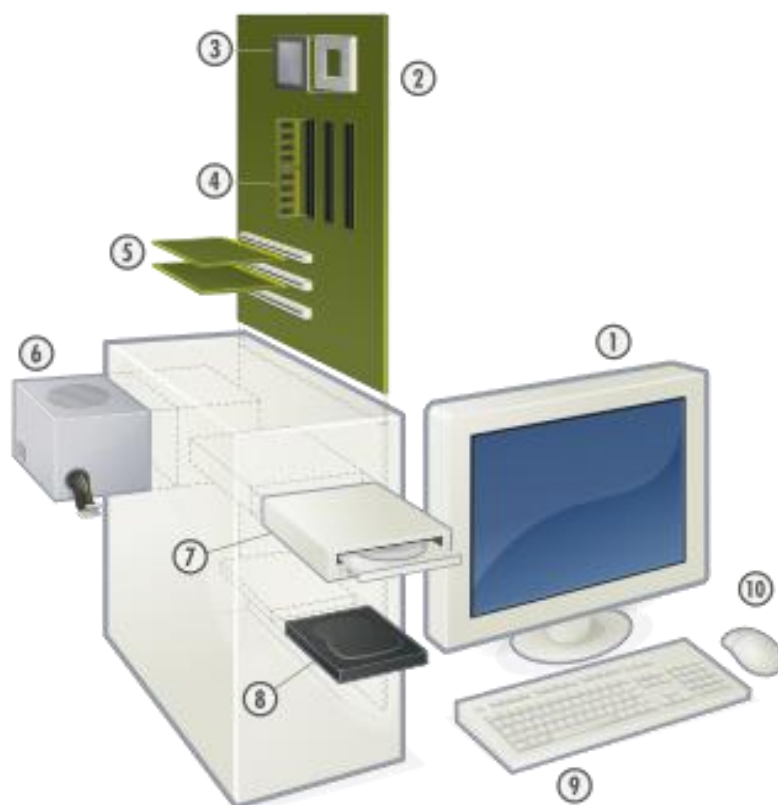
Un **PC** es un ordenador de uso personal pensado para tareas como navegar por internet, crear documentos, jugar videojuegos o trabajar en proyectos personales o profesionales. Su rendimiento y capacidad son los adecuados para ese tipo de tareas.

Un **laptop** es básicamente un PC compacto y portátil diseñado para la movilidad, que integra pantalla, teclado, batería y componentes internos en un solo dispositivo.

Las **tabletas** y **smartphones** cumplen todos los requisitos como máquinas de proceso de información para ser equiparables a ordenadores ultraportátiles.

Hardware

Despiece general



La figura, muestra el despiece general de un ordenador, en este caso un PC, en el que podemos distinguir:

1. Monitor
2. Placa base
3. *Procesador*
4. *Memoria principal (RAM: Random-Access Memory)*
5. *Placas de expansión*
6. Fuente de alimentación
7. Unidad de almacenamiento óptico
8. Disco duro, unidad de estado sólido
9. Teclado
10. Ratón

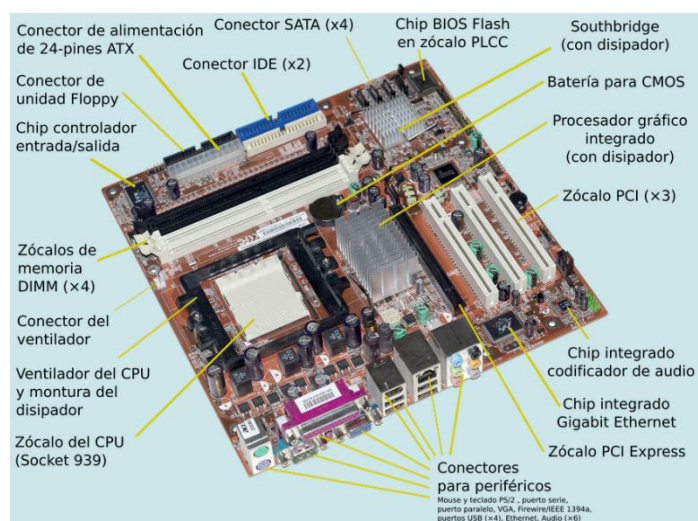
El monitor, el teclado y el ratón son los dispositivos típicos de entrada o salida que permiten la interacción del usuario con el ordenador. Son dispositivos periféricos que se enchufan a conectores específicos de la placa base. Otros periféricos de entrada/salida pueden ser: un scanner, una tableta digitalizadora, unos altavoces, etc.

La fuente de alimentación (*Power Supply Unit* - PSU) se encarga de suministrar energía al resto de los componentes del ordenador. Es esencial elegir una fuente de alimentación de potencia adecuada para alimentar todos los componentes de la computadora. Lo ideal es que una fuente de alimentación funcione entre el 50 y el 80% de su potencia máxima; un funcionamiento continuo fuera de ese rango puede provocar bloqueos del sistema o su deterioro prematuro.

Placa base

La placa base, también conocida como tarjeta madre, placa madre o placa principal (*motherboard* o *mainboard* en inglés), es una tarjeta de circuito impreso a la que se conectan todos los componentes que constituyen la computadora. Los componentes más destacados de una placa base son:

- Uno o varios **conectores de alimentación**, que reciben alimentación eléctrica de la fuente de alimentación de la computadora y la distribuyen a la CPU, el conjunto de chips, la memoria principal y las tarjetas de expansión.
- Uno o más **zócalos (socket) de CPU**, en los que se instala el microprocesador y lo conectan con el resto de los componentes a través del **bus frontal** de la placa base.
- El **chipset**, que es un conjunto de circuitos electrónicos que gestionan las transferencias de datos entre los diferentes componentes del ordenador (procesador, memoria, tarjeta gráfica, unidad de almacenamiento secundario, etc.). El *chipset*, generalmente, se divide en dos secciones:
 - El puente norte (**northbridge**), que gestiona la interconexión entre el microprocesador, la memoria RAM y la unidad de procesamiento gráfico.
 - El puente sur (**southbridge**), que gestiona la interconexión entre los periféricos y los dispositivos de almacenamiento, como los discos duros o las unidades de disco óptico.
- **Ranuras** para insertar módulos de **memoria RAM** (*Random-Access Memory*) del tipo adecuado, dependiendo de la velocidad, capacidad y fabricante requeridos según la compatibilidad de la placa base y la CPU.
- Chips de memoria no volátiles (generalmente **Flash ROM** en placas base modernas) que contienen el *firmware* o BIOS del sistema (un software que localiza y reconoce todos los dispositivos necesarios para cargar el sistema operativo en la memoria RAM).
- **Las ranuras de expansión**, que pueden acoger placas o tarjetas de expansión que



se utilizan para agregar características o aumentar el rendimiento de la computadora (por ejemplo, una tarjeta gráfica, con una Unidad de Procesamiento Gráfico - **GPU**).

- Un **generador de reloj** que produce la señal del reloj del sistema para sincronizar los distintos componentes.
- Conectores para discos duros, conectores para periféricos (puertos USB, conectores de vídeo, audio, etc.) y controladores integrados de periféricos (controladores de discos, de Ethernet, de USB, de gráficos, etc.).

Microprocesador

El **microprocesador** (o CPU, *Central Processing Unit*) es el circuito integrado central más complejo de un sistema informático; a modo de ilustración, se le suele llamar por analogía el «cerebro» de un ordenador.



Ejecuta los programas (desde el sistema operativo hasta las aplicaciones de usuario), compuestos de instrucciones simples programadas en código máquina binario (lenguaje de "bajo nivel") que va leyendo típicamente de la RAM: operaciones aritméticas y lógicas (tales como sumar, restar, multiplicar, dividir, operaciones lógicas binarias), comparaciones (>, <, ...) para saltar o no a otra parte del programa, así como accesos a memoria RAM o dispositivos para lectura o escritura de datos elementales.

Puede contener una o más unidades centrales de procesamiento (CPU), constituidas esencialmente por **registros**, una **unidad de control**, una **Unidad Aritmético-Lógica** (ALU) y una unidad de cálculo en coma flotante (conocida antiguamente como "coprocesador matemático").

En las computadoras, el microprocesador se monta en la llamada placa base, sobre un zócalo conocido como zócalo de CPU, que permite las conexiones eléctricas entre los circuitos de la placa y el procesador. Sobre el procesador, ajustado a la placa base se fija un disipador térmico de un material con elevada conductividad térmica, que por lo general es de aluminio, y en algunos casos de cobre. Este disipador es indispensable en los microprocesadores, que consumen bastante energía, la cual, en gran parte, es emitida en forma de calor: en algunos casos, pueden consumir tanta energía como una lámpara incandescente (de 40 a 130 vatios).

Memoria RAM

La memoria de acceso aleatorio (*Random-Access Memory*, RAM) se utiliza como memoria de trabajo de computadoras y otros dispositivos para el sistema operativo, los programas y la mayor parte del software. Viene en módulos que se insertan en ranuras de la placa base.



En la RAM se cargan todas las instrucciones que ejecuta la unidad central de procesamiento (procesador) y otras unidades del computador, además de contener los datos que manipulan los distintos programas. Es volátil, o sea, su contenido se borra al desconectarse de la alimentación eléctrica.

Se denominan «de acceso aleatorio» porque se puede leer o escribir en una posición de memoria con un tiempo de espera igual para cualquier posición, sin necesidad de seguir un orden secuencial para acceder a la información de la manera más rápida posible.

Los ordenadores actuales usan módulos DIMM (*Dual In-line Memory Module*, en ordenadores de sobremesa), o SO-DIMM (*Small Outline DIMM*, en ordenadores portátiles). La cantidad máxima de memoria que puede instalarse en un ordenador viene determinada por el número de ranuras multiplicado por la capacidad de los módulos que se le pueden poner.

El tipo de memoria se identifica mediante unas siglas, que indican la tecnología usada, separadas por un guion de un número que indica la frecuencia efectiva del reloj, es decir, el ritmo de transferencia de información. Por ejemplo, DDR3-1600 corresponde a *Double Data Rate type-3 Synchronous Dynamic Random-access Memory*, con un ritmo efectivo de 1600 millones de transferencias por segundo.

El modelo del módulo se identifica con unas siglas separadas por un guion de un número que indica el ancho de banda aproximado (es decir, el máximo caudal de transferencia en millones de bytes por segundo). Por ejemplo, PC3-12800 indica un ancho de banda de 12800 MB/s. Este valor se obtiene multiplicando por 8 la frecuencia efectiva del reloj indicada en el tipo de memoria.

Unidades de almacenamiento

Un disco duro o disco rígido (en inglés, **Hard Disk Drive**, HDD) es un dispositivo de almacenamiento de datos no volátil (no pierde la información al desconectarle de la alimentación eléctrica, a diferencia de la RAM), que emplea un sistema de grabación magnética para almacenar datos digitales.

Se compone de uno o más platos circulares unidos a un mismo eje que gira a gran velocidad dentro de una caja metálica sellada. Sobre cada plato, y en cada una de sus caras, se sitúa un cabezal de lectura/escritura que flota sobre una delgadísima lámina de aire generada por la rotación de los discos. Los platos se dividen en circunferencias concéntricas llamadas pistas. El conjunto de pistas que están a la misma distancia respecto del eje en los distintos platos y caras forman un cilindro. Las pistas, a su vez, se dividen en sectores, que según el estándar actual tienen un tamaño de 512 bytes. Las pistas externas, al ser más largas, admiten más sectores que las internas.



Las características más importantes de los discos duros son:

- **Capacidad:** Cantidad de información que puede almacenar. Usualmente se mide en gigabytes (GB, miles de millones de bytes) o terabytes (TB, billones de bytes).
- **Tiempo medio de acceso:** Tiempo medio que tarda la cabeza en situarse en la pista y el sector deseado; se obtiene sumando:
 - **Tiempo medio de búsqueda:** Tiempo medio que tarda la cabeza en situarse en la pista deseada; aproximadamente un tercio del tiempo empleado por la cabeza en desplazarse desde la pista más periférica hasta la más interna del disco.
 - **Latencia media:** Tiempo medio que tarda la cabeza en situarse en el sector deseado; equivale a la mitad del tiempo empleado en una rotación completa del disco. Este tiempo es la inversa de la frecuencia de rotación, que se suele expresar en número de revoluciones por minuto (RPM).
- **Tasa de transferencia:** Ritmo al que puede transferir los datos una vez que la cabeza está situada en la pista y sector correctos (puede variar según la ubicación de la pista en el disco).
- **Interfaz de conexión:** Existen diferentes formas de conectar el disco al ordenador, por ejemplo, SCSI (*Small Computer System Interface*), que es muy popular en servidores, y SATA (*Serial ATA*), común en ordenadores de sobremesa.
- Otros aspectos a tener en cuenta: También se debe considerar el consumo energético y la protección contra impactos por caídas (especialmente en el caso de portátiles), ya que podrían dañar su delicada mecánica.

Las unidades de estado sólido, SSD (*Solid-State Drive*), son un tipo de dispositivo de almacenamiento de datos para el mismo propósito que los HDD, pero que utiliza memoria no volátil en circuitos integrados, como la memoria *flash*, para almacenar los datos, en lugar de los platos o discos magnéticos giratorios de las unidades de discos duros convencionales. Por compatibilidad, muchas veces se diseñan con similares dimensiones a las de los discos duros convencionales.

En comparación con los discos duros tradicionales, las unidades de estado sólido son menos sensibles a los golpes, al no tener partes móviles; son silenciosas y ofrecen un muchísimo menor tiempo de acceso y de latencia, lo que se traduce en una mejora considerable del rendimiento, por ejemplo, en los tiempos de carga del sistema operativo. En contrapartida, su vida útil es muy inferior, ya que tienen un número limitado de ciclos de escritura, pudiendo producirse la pérdida absoluta de los datos de forma inesperada e irreparable.

Los "discos" SSD suelen hacer uso de la misma interfaz SATA que los discos duros, por lo que son fácilmente intercambiables sin tener que recurrir a adaptadores o tarjetas de expansión para hacerlos compatibles con el equipo.

Al preparar un disco para su uso, se le suele dividir (particionar) en uno o varios sistemas de ficheros, que pueden ser de distinto tipo (FAT, NTFS, HFS+, ext3, etc.). Esto permite a un sistema operativo compatible usar esa parte (unidad) del disco como un conjunto organizado de carpetas y ficheros, determinando además, por eficiencia, el tamaño contiguo mínimo de sectores (cluster, o bloque) que se va a usar, (por ejemplo, 8). Un fichero puede ocupar uno o varios *clusters*, pero dos ficheros no pueden compartir un mismo *cluster*. Parte del disco puede dedicarse para *swap*, es decir, a mover transitoriamente a disco parte del contenido de la RAM cuando su tamaño es insuficiente para el uso actual.

Conectividad y comunicaciones

DVI: *Digital Visual Interface* o "Interfaz Digital Visual", es una interfaz de video diseñada (particularmente en modo digital DVI-D) para obtener la máxima calidad de visualización posible en pantallas digitales, tales como los monitores LCD de pantalla plana y los proyectores digitales.

USB: *Universal Serial Bus* o "Bus Universal en Serie", es un estándar industrial que define los cables, conectores y protocolos usados para comunicar y suministrar una ligera alimentación eléctrica entre computadoras, periféricos y dispositivos electrónicos.

VGA: *Video Graphics Array* o "Adaptador Gráfico de Vídeo", se utiliza para denominar a una pantalla de computadora analógica estándar, a la resolución 640 × 480 píxeles, al conector VGA tipo D subminiatura de 15 contactos o a la tarjeta gráfica que comercializó IBM por primera vez en 1988.

VGA fue el último estándar de gráficos introducido por IBM al que se atuvieron la mayoría de los fabricantes de computadoras compatibles con IBM, convirtiéndose en el mínimo que todo el hardware gráfico soportaba antes de cargar un dispositivo específico. Por ejemplo, la pantalla de inicio de Microsoft Windows aparece mientras la máquina sigue funcionando en modo VGA, razón por la que dicha pantalla siempre se mostrarán con una resolución y profundidad de color reducidas.

La norma VGA fue oficialmente reemplazada por *Extended Graphics Array* de IBM, aunque en la práctica fue sustituida por numerosas extensiones clónicas, ligeramente distintas a VGA desarrolladas por otros fabricantes y conocidas en conjunto como *Super VGA*.

HDMI: *High-Definition Multimedia Interface*, traducido como "Interfaz Multimedia de Alta Definición". Es una norma para la interconexión de audio y vídeo digital sin comprimir, que se convirtió en el sustituto digital del analógico Euroconector. HDMI permite el uso del vídeo estándar, mejorado o de alta definición, así como audio digital multicanal en un solo cable. Incluye también 8 canales de audio digital sin comprimir.

LAN: (Ethernet): Una red de área local (*Local Area Network*) define una red que interconecta ordenadores en un área limitada, como un domicilio, una escuela o una empresa. Sus características se definen en contraposición a las redes de área amplia (*Wide Area Network* - WAN) que interconectan ordenadores en una zona geográfica extensas usando servicios de telecomunicaciones.

Ethernet:(pronunciado /'i:θərnɛt/ en inglés) es un estándar de redes de área local de tipo CSMA/CD, es decir, con acceso a un medio de transmisión compartido (el "éter"), con detección de la onda portadora y de colisiones (cuando más de un dispositivo intenta usar simultáneamente el medio común, se retrae durante un tiempo aleatorio progresivo). Ethernet define las características de cableados posibles y la señalización de nivel físico, así como los formatos de tramas de datos del nivel de enlace de datos del modelo OSI. Ethernet fue la base para la redacción del estándar internacional IEEE 802.3. Típicamente, un *router* proporciona acceso a Internet a los dispositivos de la red local.

Wi-Fi: *Wireless Fidelity*, pronunciado /'waɪfaɪ/ en inglés, aunque /'wɪfi/ en algunos países hispanohablantes, es una sección inalámbrica de una Ethernet en la que el medio compartido es una banda del espacio radioeléctrico. Los dispositivos con acceso Wi-Fi, tales como un ordenador personal, una consola de videojuegos, un *smartphone* o un reproductor de audio digital, se conectan al resto (cableado) de la Ethernet mediante un punto de acceso (un mismo aparato

puede hacer de punto de acceso y *router*). Estos pueden alcanzar unos 20 metros de cobertura en interiores, y mayor distancia al aire libre. Para que puedan coexistir Wi-Fi independientes en un mismo espacio físico sin interferirse, se distribuyen en distintas subbandas de frecuencia (canales). Los adaptadores enchufables PLC (*powerline communication*) funcionan como puntos de acceso inalámbrico que permiten aprovechar el cableado eléctrico para conectarse con el resto de la Ethernet.

Software

El **software** son los programas que hacen funcionar un ordenador; es la parte intangible del sistema, frente al **hardware**, que es la parte tangible, al estar formado por el conjunto de componentes físicos. Al clasificar el software, se puede distinguir entre el sistema operativo y las aplicaciones, o, de modo más amplio, entre software de sistema y software de aplicación.

- El **software de sistema** abarca aquellos programas que se ocupan del funcionamiento general del ordenador, facilitando la ejecución de las aplicaciones y la interacción con el usuario y los dispositivos externos.
- El **software de aplicación** comprende aquellos programas diseñados para satisfacer una necesidad concreta del usuario.

Software de sistema

El software de sistema incluye los programas que hacen funcionar el ordenador, los que se usan para configurar, analizar y administrar el sistema, y los que se emplean para desarrollar otros componentes software.

Arranque del sistema (bootstrap)

Al encenderse el ordenador, se ejecutan automáticamente las instrucciones contenidas en la memoria no volátil de la BIOS (*Basic Input-Output System*), una rutina que verifica el hardware y que, de acuerdo con su configuración, localiza el sector de arranque (*boot sector*) del dispositivo de arranque predeterminado (HDD, SSD, DVD, etc.) lo carga en memoria y le cede el control. A su vez, este comienza a cargar en memoria el sistema operativo, que pasa a tomar el control del ordenador.

Sistema operativo (operating system, OS)

Es el principal componente del software de sistema. Su propósito es tomar el control del *hardware* y actuar de intermediario entre este y el resto de los programas, liberando a estos de la necesidad de gestionar los detalles del *hardware*. Sus cometidos principales son: la ejecución de programas, la gestión de recursos (memoria principal, almacenamiento secundario, tiempo de CPU, etc.), la comunicación entre los componentes de la máquina y las aplicaciones y la seguridad.

Los sistemas operativos más populares en los ordenadores de sobremesa son: Windows, OSX y Linux; y en los dispositivos móviles Android e iOS.

Controladores de dispositivos (device drivers)

En computación, un controlador de dispositivo es un programa de computadora que opera o controla un tipo particular de dispositivo que está conectado a una computadora. Un controlador proporciona una interfaz de software para dispositivos de *hardware*, permitiendo que los sistemas operativos y otros programas de computadora accedan a las funciones del dispositivo

sin necesidad de conocer detalles precisos sobre el *hardware* que se está utilizando (por ejemplo: controladores (*drivers*) de disco, de tarjeta gráfica, de dispositivo de audio, etc.).

Servidores (servers)

Son programas que se ejecutan, y suelen estar continuamente disponibles para proporcionar un servicio a otros programas (clientes), normalmente de forma remota. Por ejemplo, *Apache Tomcat* o *Internet Information Service* son servidores HTTP que sirven páginas web y documentos a requerimiento de un navegador (*web browser*) que, normalmente, se ejecuta en una máquina remota. Un servidor FTP (*File Transfer Protocol*) proporciona servicios de carga y descarga de ficheros a una aplicación cliente. Un servidor de bases de datos (como MySQL) proporciona acceso, local o remoto, a un Sistema de Bases de Datos, entre otros.

Utilidades (utilities)

Las utilidades son programas diseñados para ayudar a analizar, configurar, optimizar o mantener un ordenador o sistema informático. Generalmente, el sistema operativo incluye un conjunto básico de programas de utilidad, aunque es frecuente que el usuario pueda instalar utilidades adicionales. Algunos ejemplos de programas de utilidad pueden ser: el Administrador de tareas de Windows, un desfragmentador de disco, un antivirus, la aplicación de configuración de iOS, un limpiador de registro, un monitor de actividad, una terminal de línea de comandos, etc.

Software de aplicación

El *software* de aplicación tiene como objetivo satisfacer alguna necesidad del usuario, frente al *software* de sistema, que se centra en el funcionamiento del ordenador. Cabe distinguir entre *software* de aplicación de propósito general y *software* de aplicación de propósito específico.

El *software* de aplicación de propósito general satisface necesidades básicas comunes a la mayoría de los usuarios. Ejemplos de aplicaciones de propósito general son:

- Clientes de correo
- Navegadores (*web browsers*)
- Procesadores de texto
- Hojas de cálculo
- Mensajería electrónica
- ...

El *software* de propósito específico satisface necesidades particulares de usuarios especializados, por ejemplo:

- *Software* específico para Ingeniería Civil
- Aplicaciones médicas
- *Software* para Ciencia de Datos
- Entornos Integrados de Desarrollo de Software
- ...

Más sobre datos

Los **datos** son propiedades de los objetos/entidades, normalmente obtenidos por medición u observación. Para que los datos puedan ser procesados, deben ser expresados. Los datos se pueden expresar mediante texto, voz, imagen (gráficamente), electrónicamente, etc. En términos generales, se puede decir que los datos se expresan mediante signos o señales.

Para trabajar con los datos, estos se agrupan en unidades lógicas superiores denominadas **registros** (oraciones). Un registro es una unidad de datos lógica de datos. Sin embargo, un registro no es el elemento de datos más pequeño, ya que puede ser descompuesto. Al igual que en la vida cotidiana, donde la oración consta de palabras, en los datos el registro se divide en atributos. El atributo es la parte más pequeña direccionable del registro. Así como una palabra se puede descomponer en letras individuales, en algunos casos es posible dividir un atributo, aunque, al igual que las palabras, perdería su significado.

Los atributos pueden ser atómicos o estructurados. Un ejemplo de atributo estructurado puede ser una dirección (calle, número de casa, ciudad, código postal, etc.). Es conveniente evitar tales atributos y dividirlos en atributos atómicos. Los atributos tienen una posición determinada en el registro, y un significado específico. Para que un atributo desempeñe su función correctamente, los valores que puede adoptar deben ser significativos. El conjunto de valores permitidos que un atributo puede adquirir se denomina **dominio**.

El dominio no solo especifica que el atributo "NOMBRE DE UNA PERSONA" es una *string* (esto corresponde a la determinación del tipo de datos, que se explicará más adelante con más detalle). El dominio es más específico: representa todos los valores significativos para un atributo dado. Por ejemplo, en el dominio del atributo "NOMBRE DE UNA PERSONA" se puede incluir el valor "Juan", pero no el valor "x7br_15", aunque sea también una *string*.

El tipo de registro determina qué atributos (incluidos sus respectivos dominios) contiene. El orden de los atributos individuales también juega un papel importante. El tipo de registro (oración) se especifica por sus atributos, por ejemplo, "NOMBRE DE UNA PERSONA", "ACTIVIDAD, OBJETO". Una instancia de este tipo de registro podría ser: "Juan conduce un coche".

La **información** es intangible. La información son datos que tienen significado; es, por tanto, un subconjunto de datos. La información puede responder a preguntas, reduciendo así la ignorancia o incertidumbre. Puede estar contenida tanto en los signos (o señales) como en su disposición. Su apariencia física puede variar (texto, imagen, señales, etc.).

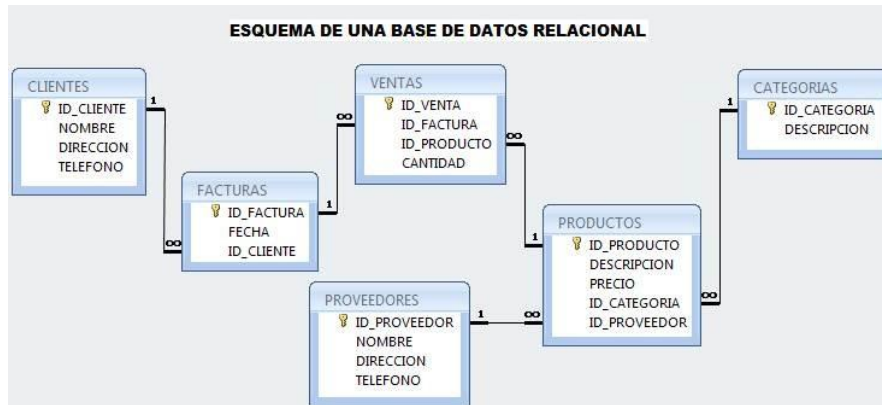
Los datos proporcionan información únicamente a quienes pueden interpretarlos, es decir, a quienes pueden comprender su sintaxis y su semántica. Lo que es información para una persona, puede ser solo datos para otra.

Bases de datos

Cuando se trabaja con grandes volúmenes de datos, los registros del mismo tipo se agrupan en **archivos de datos**. Para trabajar con ellos (por ejemplo, buscar, eliminar, editar), cada registro debe distinguirse claramente de los demás. Por lo tanto, cada registro debe estar identificado por la *clave* de archivo. Una **clave** es un conjunto de atributos que identifican de forma única un registro. El número de atributos que pertenecen a la clave se llama k . El número total de atributos del registro se denota como n . Siempre se cumple que $k \leq n$. El objetivo es mantener dicho valor

lo más pequeño posible. Todas las claves mínimas crean un espacio K^* . Una de las claves se designa como clave principal.

Una **Base de Datos** es un conjunto de archivos homogéneos y estructurados que almacenan datos para su posterior procesamiento. La distribución de los datos en los archivos individuales responde a un significado global. Todos los datos dentro de un mismo archivo presentan la misma estructura.



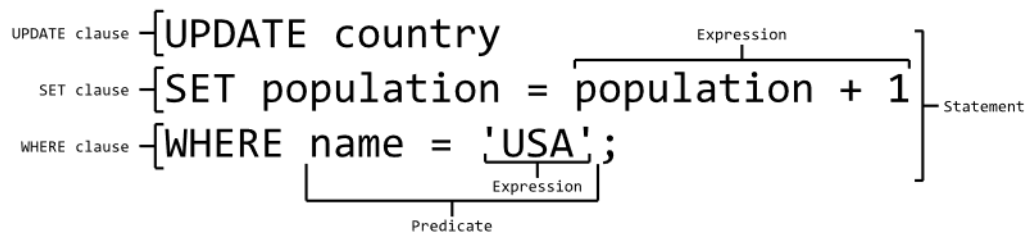
Para acceder a la base de datos, existe una herramienta llamada **Sistema de Gestión de Base de Datos** (SGBD), a menudo abreviada como **DBMS** (por *Data Base Management System*). Un sistema de gestión de base de datos es una herramienta de software integrada que permite definir, crear y administrar el acceso a la base de datos y trabajar con ella. Normalmente, se trata de una colección de programas que actúan como interfaz entre los programas de aplicación y los datos almacenados; es un *software* de aplicación de propósito general. Algunos ejemplos de SGBD son: *MySQL*, *MariaDB*, *PostgreSQL*, *Microsoft SQL Server*, *Oracle Database* y *Microsoft Access*.

El Sistema de Gestión de Bases de Datos cumple muchas funciones: en primer lugar, permite manipular los datos; garantiza que solo los usuarios autorizados puedan acceder ellos; permite el acceso concurrente de múltiples usuarios; proporciona gestión de transacciones; dependiendo del modelo utilizado, crea una base de datos y define su esquema (estructura); en caso de fallo, posibilita la recuperación; y comprueba la integridad de los datos, entre otras muchas tareas.

Los beneficios que aporta el DBMS al acceso directo a los datos son indiscutibles. Entre los principales, se encuentran:

- **Abstracción de datos:** el usuario no accede directamente a los archivos de origen, sino que trabaja con estructuras formalizadas en un nivel lógico superior de abstracción.
- **Independencia de los datos:** si los datos físicos cambian, ello no afecta el funcionamiento de los programas de aplicación. La interfaz de datos seguirá siendo la misma hacia el exterior.
- **Administración de datos centralizada:** todos los datos se encuentran en un solo lugar. Todo es tratado de manera homogénea. Es posible mostrar una descripción estructurada de los datos.
- **Capacidad de formular consultas *ad hoc* fuera de los programas de aplicación:** los usuarios pueden realizar consultas directamente sobre la base de datos a través del DBMS, sin necesidad de otros programas.

SQL (por sus siglas en inglés *Structured Query Language*; en español, lenguaje de consulta estructurada) es un lenguaje específico de dominio, diseñado para administrar y recuperar información de sistemas de gestión de bases de datos relacionales. Una de sus principales características es el uso del álgebra y del cálculo relacional para efectuar consultas, con el fin de obtener información de forma sencilla, así como realizar modificaciones sobre las bases de datos.



De: User:SqlPac, modified by Ferdna - Trabajo propio, basado en: *Sql statement anatomy.png*, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=7557212>

Anexo: elementos, funciones y terminología básica del ordenador

Caracteres que representan símbolos usados en lenguajes de programación

En este documento se recogen caracteres y símbolos que a menudo se encuentran en la programación; se indican sus respectivos nombres en español e inglés. Además, se proporcionan ejemplos de cómo se utilizan estos caracteres en contextos de programación reales y, para facilitar su identificación, se especifica su ubicación en el teclado. **Conocer estos símbolos es fundamental para los programadores**, ya que son elementos de uso cruciales en muchos lenguajes de programación. Se recuerda que cada lenguaje tiene sus propias reglas y convenciones, por lo que el uso de estos símbolos puede variar dependiendo del lenguaje que se esté utilizando.

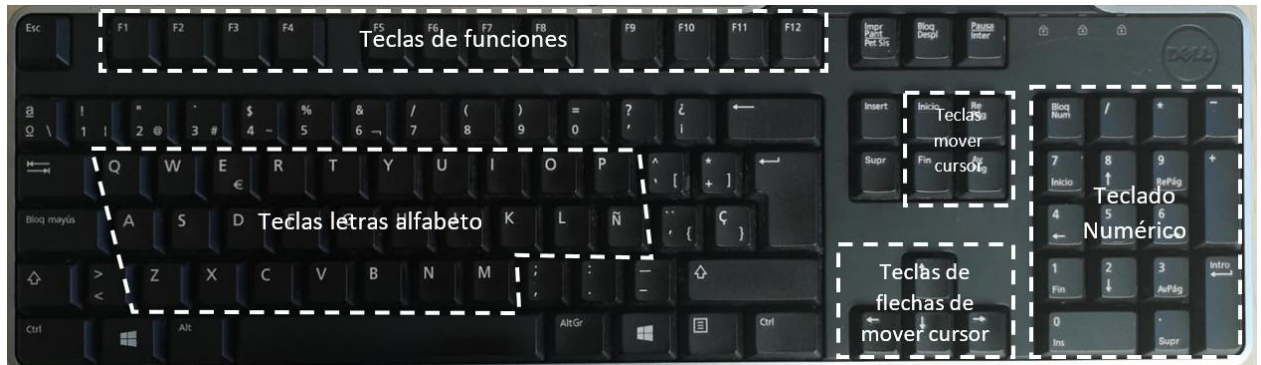
Símbolo	Nombre en Español	Nombre en Inglés	Ejemplo en Informática	Tecla
!	Exclamación	Exclamation mark	Se usa en Python y otros lenguajes para formar el operador de comparación de desigualdad ($a \neq b$)	Shift+1
"	Comilla doble, comillas	Double quote	Se usa en Python y otros lenguajes para encerrar una string, p. ej., "Hello, world!"	Shift+2
#	Signo numérico o almohadilla	Number sign, hashtag, hash	Se usa para comentarios en scripts de Python, p. ej., # Esto es un comentario	AltGr+3
\$	Signo de dólar	Dollar sign	Identifica variables en scripts de Bash, p. ej., \$variable	Shift+4
%	Porcentaje	Percent sign	Se usa en Python como operador módulo, p. ej., 5 % 2 da 1, el resto de dividir 5 entre 2	Shift+5
&	Ampersand, "y" comercial	Ampersand	Se usa en Python y otros lenguajes como operador de producto bit a bit, p. ej., 5 & 3 → 101 & 011 == 001	Shift+6
'	Comilla, comilla simple, apóstrofo	Apostrophe	Se usa en Python y otros lenguajes para encerrar una string, p. ej., 'Hola mundo'	Tecla a la derecha del cero (0)
(	Paréntesis izquierdo	Left parenthesis	Se usa en Python y otros lenguajes para abrir un grupo en una expresión, p. ej., (x * (y + z))	Shift+8
)	Paréntesis derecho	Right parenthesis	Se usa en Python y otros lenguajes para cerrar un grupo en una expresión, p. ej., (x * (y + z))	Shift+9
*	Asterisco	Asterisk	Se usa en Python y otros lenguajes como operador de multiplicación, p. ej., 3 * 3. En SQL y otros lenguajes se usa como comodín para indicar "todos" en una expresión de búsqueda, p. ej., SELECT * FROM table	Shift + tecla con + a la izquierda de ↵
+	Más	Plus sign	Se usa en Python y otros lenguajes como operador de suma, p. ej., 2 + 2	Tecla a la izquierda de ↵ o en teclado numérico
,	Coma	Comma	Se usa en Python y otros lenguajes para separar elementos en una secuencia, p. ej., [1, 2, 3, 4]	Tecla a la derecha de la M
-	Menos, guion	Minus, hyphen	Se usa en Python y otros lenguajes como operador de resta, p. ej., 5 - 2	Tecla debajo de la Ñ
.	Punto	Period, full stop	Se usa para acceder a propiedades y métodos en Python, p. ej., objeto.propiedad	Tecla debajo de la Ñ

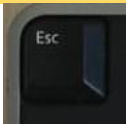
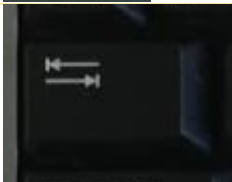
Símbolo	Nombre en Español	Nombre en Inglés	Ejemplo en Informática	Tecla
/	Barra de división	Slash	Se usa para división en programación, p. ej., 10 / 5, y en direcciones de URL (https://ejemplo.org/)	Shift+7 o en teclado numérico
:	Dos puntos	Colon	Se usa para definir diccionarios en Python, p. ej., {'key': 'value'}	Shift + tecla con punto
;	Punto y coma	Semicolon	Finaliza una instrucción en muchos lenguajes de programación, p. ej., x = 5;	Shift + tecla con coma
<	Menor que	Less than	Se usa para comparación en Python, p. ej., if x < y:	Tecla a la izquierda de la Z
=	Igual	Equal sign	Asigna un valor a una variable en Python, p. ej., x = 5	Shift+0
>	Mayor que	Greater than	Se usa para comparación en Python, p. ej., if x > y	Shift + tecla con <
?	Interrogación	Question mark	Se usa como operador ternario en lenguaje C, p. ej., x > y ? x : y	Shift + tecla con comilla
@	Arroba	At sign	Se usa para decoradores en Python, p. ej., @classmethod	AltGr+2
[	Corchete izquierdo	Left bracket	Comienza una lista en Python, p. ej., [1, 2, 3]	AltGr+ tecla con [a la derecha de la P
\	Barra invertida	Backslash	Se usa para escapar caracteres en strings, p. ej., "\n" es una nueva línea	AltGr+ tecla a la izquierda del 1
]	Corchete derecho	Right bracket	Cierra una lista o matriz en Python, p. ej., [1, 2, 3]	AltGr+ tecla con]
^	Acento circunflejo	Circumflex, caret	Se usa en Python como operador XOR a nivel de bits, p. ej., x ^ y → 1010 ^ 0110 == 1100	Shift tecla con [y después espacio
_	Guion bajo	Underscore	Se usa para separar palabras en nombres de variables en Python, p. ej., my_variable	Shift + - (menos o guion)
`	Acento grave	Grave accent	Encierra strings en formato Markdown para formateo de código, p. ej. `code`	Tecla con [y después espacio
{	Abre llave	Left brace, left curly bracket	Comienza un conjunto o diccionario en Python, p. ej., {1, 2, 3}	AltGr + tecla con { a la derecha de Ñ
	Barra vertical	Pipe, vertical bar	Se usa en Python como operador OR a nivel de bits, p. ej., x y → 101 011 == 111	AltGr + tecla con 1
}	Cierra llave	Right brace, right curly bracket	Finaliza un conjunto o diccionario en Python, p. ej., {1, 2, 3}	AltGr + tecla con } a la derecha de {
~	Tilde	Tilde	Se usa en Python como operador de complemento a nivel de bits, p. ej., ~x	AltGr + 4 y después espacio

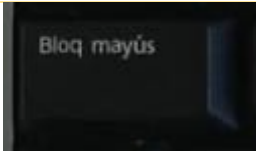
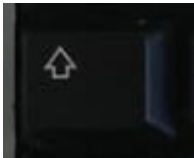
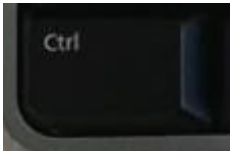
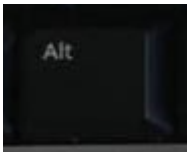
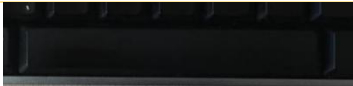
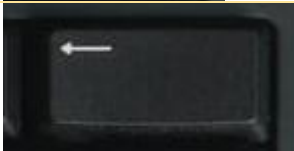
Teclas especiales de un teclado

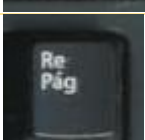
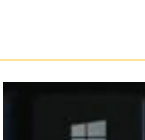
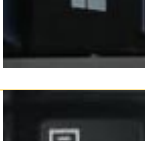
En este documento exploramos la disposición y funciones de un teclado "QWERTY" estándar para Windows en español. Se presentan las teclas que representan símbolos y caracteres

especiales, además de las teclas funcionales que permiten la edición de texto, el movimiento del cursor, la selección de texto y más. Para un programador, conocer bien **el teclado** es crucial, ya que **es la principal herramienta que se utiliza para escribir y modificar código**. En este documento, se pone el foco en aquellas teclas que van más allá de los simples caracteres alfabéticos, ayudando a entenderlas y a sacarles el máximo provecho.



Tecla	Nombre en Español	Nombre en Inglés	Acción
	Escape	Escape	Cierra la ventana o el diálogo activo.
	Función 1-12	Function 1-12	Ejecutan funciones específicas que varían dependiendo del programa en uso; por ejemplo, F1 generalmente abre la ayuda del programa en ejecución.
	Imprimir Pantalla	Print Screen	Copia la imagen de toda la pantalla al portapapeles.
	Bloq Despl	Scroll Lock	Altera el comportamiento de las teclas de dirección (flechas). Cuando está activado, las teclas de flecha no mueven el cursor, sino que desplazan la ventana, como la rueda del ratón. Pocos programas actuales admiten esta acción.
	Pausa o Interrumpir	Pause o break	Pausa o interrumpe la ejecución de programas en una terminal.
	Tabulación	Tab	Avanza al siguiente campo en un diálogo o inserta un carácter de indentación/sangrado de código.

		Bloq Mayús	Caps Lock	Cuando está activa, al pulsar una tecla de letra se genera un carácter en mayúscula en lugar de minúscula.
		Mayúscula	Shift	Modifica el comportamiento de muchas teclas. Para provocar su efecto, debe mantenerse pulsada mientras se pulsa otra tecla. En las teclas de letras del alfabeto hace que se genere un carácter en mayúscula o minúscula, contrario al que se generaría si no se pulsase. En teclas con más de un símbolo, produce el símbolo ubicado en la parte superior.
		Control	Control	Modifica el comportamiento de otras teclas al mantenerse pulsado. En muchos casos, genera una acción en el sistema o en el programa en ejecución, por ejemplo. Ctrl-C, Ctrl-V, etc.
		Alternar	Alt	Modifica el comportamiento de algunas teclas. Para provocar su efecto, hay que mantenerla pulsada mientras se pulsa otra. En muchos casos, genera una acción en el sistema o el programa activo, por ejemplo, Alt-F4, Alt-F11, etc.
		Espacio	Space	Inserta el carácter de espacio en blanco.
		Intro	Enter	Envía una entrada de texto (línea de texto) o ejecuta una acción en un diálogo.
		Retroceso	Backspace	Borra el carácter anterior al cursor de texto.
		Suprimir	Delete	Borra el carácter en la posición del cursor de texto.

		Inicio	Home	Mueve el cursor de texto al inicio de la línea.
		Fin	End	Mueve el cursor de texto al final de la línea.
		Re Pág	Page Up	Desplaza la vista una página hacia arriba.
		Av Pág	Page Down	Desplaza la vista una página hacia abajo.
		Teclas de dirección	Arrow keys	Permiten navegar en el texto modificando la posición del cursor.
		Bloq Num	Num Lock	Cambia el comportamiento del teclado numérico cuando se comparte con teclas de dirección. Cuando está activo, las teclas generan números; cuando está desactivado las teclas generan la acción de desplazamiento correspondiente. 
		Windows	Windows	Abre el menú de inicio o, en combinación con otras teclas, realiza una acción en el sistema, por ejemplo, la vista de tareas al pulsar Windows+Tab.
		Menú de aplicación	Application Menu	Abre el menú contextual del programa en ejecución (similar al clic derecho del ratón).
		Insertar	Insert	Cambia entre el modo de inserción y sobrescritura. Si está activo, al pulsar una tecla, se reemplaza la que está a la derecha del cursor de texto por la pulsada y se avanza el cursor.

	Alt Gráfico	Right Alt	Modifica el comportamiento de algunas teclas en teclados de idioma específico como el español. Para provocar su efecto, hay que mantenerla pulsada mientras se pulsa la otra tecla. En teclas con más de un carácter, genera el que se encuentra en la parte inferior derecha.  
---	-------------	-----------	--

Algunos elementos, funciones y terminología básica del ordenador

El ratón

El ratón de un ordenador es un dispositivo de entrada que se utiliza para interactuar con la interfaz gráfica del sistema operativo y las aplicaciones.



Cursor

El cursor es el indicador gráfico que se muestra en la pantalla y que representa la posición actual del ratón. Al mover físicamente el ratón, el cursor se desplaza por la pantalla. Dependiendo del contexto y del software que se esté utilizando, el cursor puede cambiar de forma para indicar diferentes acciones posibles. Por ejemplo, se convierte en una mano al pasar sobre un enlace en un navegador web, o en un *I-beam* (una línea vertical con dos pequeñas líneas horizontales en la parte superior e inferior) en un procesador de textos para indicar donde se insertará el texto.

Botones del ratón y clics

La mayoría de los ratones disponen de al menos dos botones: el botón izquierdo y el derecho. Algunos ratones también incluyen un tercer botón en el medio.

Botón izquierdo: Se utiliza principalmente para seleccionar elementos y para 'hacer clic' en enlaces o botones dentro de un programa. Una pulsación rápida del botón izquierdo se conoce como "clic". Si se mantiene presionado este botón mientras se arrastra el ratón, se puede seleccionar texto o mover elementos. Un "doble clic" (dos clics rápidos seguidos) en el botón izquierdo generalmente abre archivos o aplicaciones.

Botón derecho: Normalmente se utiliza para abrir menús contextuales que muestran opciones específicas para el elemento sobre el cual se hace clic con dicho botón. Por ejemplo, al hacer clic con el botón derecho sobre un archivo, se muestran opciones como abrir, copiar, mover o borrar ese archivo.

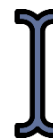
Rueda de desplazamiento: La rueda de desplazamiento es una pequeña rueda situada entre los dos botones principales del ratón. Al girarla, se puede desplazar la vista hacia arriba o hacia abajo en una página, lo cual resulta útil al leer documentos largos o páginas web. Además, en algunos programas, si se mantiene presionada la tecla Ctrl mientras se gira la rueda del ratón, se puede acercar o alejar la vista.

Escribiendo texto en el ordenador

La entrada de texto en un ordenador se realiza principalmente a través del teclado, aunque los ratones y las pantallas táctiles también pueden desempeñar un papel en este proceso. El método más común para escribir texto es mediante el teclado.

Cursor de texto

El cursor de texto, también conocido como punto de inserción o *caret*, es el indicador visual que muestra la posición actual dentro de un campo de texto o documento de texto en un programa o aplicación. Cada vez que se pulsa una tecla, el carácter correspondiente aparece en el punto donde está situado el cursor de texto.



Este cursor suele representarse como una línea vertical intermitente o un bloque sólido que parpadea. Su ubicación indica el lugar donde se insertará el texto o donde se realizarán las ediciones.

Edición de texto

Para editar texto, se pueden usar las teclas de dirección para mover el cursor hasta la ubicación donde se desean realizar cambios. Las teclas de borrar (*Backspace*) y suprimir (*Delete*) permiten eliminar caracteres. *Backspace* borra el carácter a la izquierda del cursor, mientras que *Delete* borra el carácter a la derecha del cursor.

También se pueden utilizar teclas especiales para navegar por el texto más rápidamente, moviendo el cursor de texto. Por ejemplo, la tecla *Inicio/Home* mueve el cursor al principio de una línea, mientras que *Fin/End* lo desplaza al final.

Portapapeles

El portapapeles es una función del sistema operativo que almacena temporalmente la información que ha sido cortada o copiada. Solo se puede ver y acceder al último elemento que se haya copiado o cortado; si se copia o se corta algo nuevo, este nuevo contenido reemplaza al anterior.



Selección de texto

Antes de copiar o cortar texto, este debe ser seleccionado. Para ello, se puede mantener presionada la tecla *Shift* mientras se utilizan las teclas de dirección para extender la selección de caracteres. Con el ratón, se puede mantener presionado el botón izquierdo y arrastrarlo por el texto que se desea seleccionar.

Copiar, cortar, pegar y reemplazar

Una vez que se ha seleccionado el texto, se puede copiar al portapapeles mediante el comando "Copiar" (Ctrl+C en Windows, Cmd+C en macOS), lo cual deja el texto original intacto. Si se desea mover el texto, se puede utilizar el comando "Cortar" (Ctrl+X en Windows, Cmd+X en macOS), que borra el texto original del documento al copiarlo al portapapeles.

Para insertar el contenido del portapapeles en un documento, se puede utilizar el comando "Pegar" (Ctrl+V en Windows, Cmd+V en macOS). Para reemplazar texto, se debe seleccionar previamente, de modo que al pegar se sustituya el contenido seleccionado por el que se encuentra en el portapapeles. La acción de pegar no elimina el contenido del portapapeles, por lo que se puede utilizar varias veces mientras no se haya copiado o cortado algo nuevo.

Arrastrar y soltar / Drag & Drop

Arrastrar y soltar es otra manera de mover o copiar texto. Primero se selecciona el texto, luego se hace clic sobre cualquier parte seleccionada y se mantiene presionado el botón del ratón mientras se arrastra el cursor a un nuevo lugar en el documento. Al soltar el botón del ratón, el texto se moverá o copiará a esa ubicación.

Si el texto se arrastra dentro de la misma aplicación, generalmente se mueve. Si se arrastra entre aplicaciones diferentes, generalmente se copia. Para copiar dentro de la propia aplicación, normalmente se debe mantener pulsada la tecla Ctrl antes de soltar.

Atajos de teclado

Los atajos de teclado son combinaciones de teclas que realizan tareas específicas en una aplicación de software. A continuación, se muestran atajos de teclado de uso frecuente:

- **Ctrl + C:** Copiar el texto o elemento seleccionado.
- **Ctrl + X:** Cortar el texto o elemento seleccionado.
- **Ctrl + V:** Pegar el texto o elemento copiado o cortado.
- **Ctrl + Z:** Deshacer la última acción.
- **Ctrl + Y:** Rehacer la última acción deshecha.
- **Ctrl + A:** Seleccionar todo el texto o elementos del área activa.
- **Ctrl + S:** Guardar el archivo o documento actual.
- **Ctrl + P:** Imprimir el archivo o documento actual.
- **Ctrl + F:** Buscar texto o elementos en la página o documento.
- **Ctrl + N:** Abrir una nueva ventana o documento.
- **Ctrl + W:** Cerrar la ventana o pestaña actual.

Ventana

En los sistemas operativos de interfaz gráfica, una ventana es una zona rectangular en la pantalla donde se muestra el contenido de una aplicación. El usuario puede interactuar con la aplicación a través de esta ventana. La mayoría de las ventanas tienen una barra de título en la parte superior que muestra el nombre de la aplicación y los botones para minimizar, maximizar, restaurar y cerrar la ventana. Los botones se encuentran generalmente en la esquina superior derecha de las ventanas de una aplicación en sistemas operativos como Windows y macOS. Estos botones permiten controlar el tamaño y estado de una ventana.



Botón de maximizar: Este botón tiene el símbolo de un cuadrado o dos flechas que apuntan hacia afuera. Al hacer clic en él, la ventana se expande y ocupa todo el espacio disponible en el escritorio. Esto significa que la ventana se maximiza, y se puede ver su contenido en pantalla completa sin ninguna otra ventana superpuesta.

Botón de minimizar: Este botón tiene el símbolo de un guion (–) o una flecha hacia abajo. Al hacer clic en él, la ventana se minimiza y se coloca como un botón en la barra de tareas o el *dock*, según el sistema operativo. La ventana desaparece del escritorio y se reduce a un ícono o miniatura. Se puede restaurar haciendo clic en el ícono en la barra de tareas o el *dock*.



Botón de restaurar: Este botón aparece solo cuando una ventana está maximizada. Tiene el símbolo de un cuadro con una línea en la esquina superior izquierda. Al hacer clic en él, la

ventana recupera su tamaño original antes de ser maximizada. Es decir, la ventana se reduce a su tamaño anterior y se puede mover y redimensionar libremente dentro del escritorio.

Estos botones son útiles para administrar y organizar las ventanas abiertas en tu escritorio, permitiendo cambiar rápidamente entre ellas, maximizarlas para enfocarse en su contenido o restaurarlas a su tamaño original. La funcionalidad exacta puede variar ligeramente dependiendo del sistema operativo que se esté utilizando.

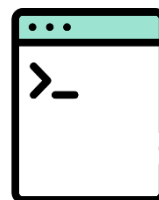
Diálogos y campos de diálogo

Un cuadro de diálogo es una pequeña ventana que se abre sobre la aplicación actual para solicitar al usuario que realice una acción o que proporcione información. Los campos de diálogo son las áreas dentro del cuadro de diálogo donde se puede introducir información. Por ejemplo, cuando se solicita guardar un documento, se abre un cuadro de diálogo que permite darle un nombre al documento e indicar dónde se debe guardar.



Terminal y consola

La terminal, o consola, es una interfaz de texto que permite interactuar con el sistema operativo y otras aplicaciones de software. Aunque puede parecer intimidante para los usuarios principiantes, aprender a usar la terminal puede ser muy útil, ya que proporciona acceso a poderosas herramientas y comandos que no siempre están disponibles a través de las interfaces gráficas.



Gestión de archivos y carpetas

La gestión de archivos y carpetas es una parte crucial de la organización del ordenador. Aprender a crear, mover, renombrar y eliminar archivos y carpetas es un aspecto importante en la programación. Además, resulta fundamental aprender a navegar por el sistema de archivos del ordenador para poder encontrar y abrir archivos y carpetas de forma rápida y eficiente.



A continuación, se muestran algunas acciones que ayudan a realizar esta gestión:

1. **Explorador de archivos:** Se debe utilizar el explorador de archivos del sistema operativo (como el Explorador de Windows en Windows o el Finder en macOS) para navegar por los archivos y carpetas.
2. **Crear carpetas:** Se pueden crear carpetas para organizar los archivos de manera lógica y coherente. Es recomendable crear carpetas principales para categorías generales (por ejemplo, "Trabajo", "Personal", "Proyectos", etc.) y subcarpetas dentro de ellas para una mayor organización.
3. **Renombrar archivos y carpetas:** Si se desea cambiar el nombre de un archivo o carpeta, se debe seleccionar el elemento, hacer clic derecho y elegir la opción "Cambiar nombre". También puede seleccionarse el archivo o carpeta y presionar la tecla "F2" en Windows o "Enter" en macOS para cambiar su nombre.
4. **Copiar y pegar:** Es posible copiar archivos o carpetas seleccionándolos y utilizando el atajo de teclado "Ctrl + C" en Windows o "Cmd + C" en macOS. Luego, se debe ir a la ubicación deseada y utilizar el atajo de teclado "Ctrl + V" en Windows o "Cmd + V" en macOS para pegar los elementos.
5. **Mover archivos y carpetas:** Para mover archivos o carpetas a una ubicación diferente, se deben seleccionar los elementos y luego arrastrarlos y soltarlos en la nueva ubicación.

También pueden utilizarse las opciones "Cortar" y "Pegar" (o sus respectivos atajos de teclado) para mover los elementos.

6. **Eliminar archivos y carpetas:** Para eliminar archivos o carpetas, basta con seleccionarlos y presionar la tecla "Supr" o "Delete" en el teclado. En general, los archivos y carpetas eliminados se envían a la papelera de reciclaje o a la papelera, en caso de que se desee restaurarlos más tarde.
7. **Búsqueda de archivos:** Se recomienda utilizar la función de búsqueda del sistema operativo para encontrar rápidamente archivos o carpetas específicos. Puede introducirse el nombre del archivo en la barra de búsqueda del explorador de archivos o utilizar filtros avanzados para refinar la búsqueda.

Estas son solo algunas acciones básicas para la gestión de archivos y carpetas en un ordenador. Cada sistema operativo y aplicación puede tener funciones adicionales y características específicas para facilitar esta gestión.

IDE (Integrated Development Environment)

Un IDE, que significa "Entorno de Desarrollo Integrado" (en inglés *Integrated Development Environment*), es una aplicación de software que proporciona facilidades integrales a los programadores para el desarrollo de software. Un IDE típicamente combina:

1. **Editor de código:** Permite a los desarrolladores escribir y modificar el código fuente de un programa. Este editor suele ofrecer características como resaltado de sintaxis y autocompletado, que facilitan la tarea de codificación.
2. **Compilador o intérprete:** Transforma el código fuente escrito por el desarrollador en un programa ejecutable o interpreta el código en tiempo real.
3. **Depurador:** Ayuda a los desarrolladores a identificar y corregir errores en el código fuente.
4. **Construcción automática:** Herramientas que automatizan procesos comunes, como la compilación y la ejecución de pruebas.
5. **Gestor de versiones:** Algunos IDE incluyen herramientas para gestionar diferentes versiones de un software, facilitando la colaboración entre desarrolladores y el seguimiento de cambios.
6. **Diseñadores gráficos:** En el caso de aplicaciones con interfaces gráficas, muchos IDE ofrecen herramientas que permiten diseñar estas interfaces de manera visual, arrastrando y soltando componentes.

Un IDE puede ser específico para un lenguaje de programación específico o puede soportar varios lenguajes. Algunos ejemplos populares de IDE son *Eclipse*, *Visual Studio*, *IntelliJ IDEA* y *NetBeans*, entre otros.

El objetivo principal de un IDE es proporcionar todas las herramientas necesarias en un solo entorno para facilitar el proceso de desarrollo, permitiendo que el programador se concentre en escribir código y desarrollar aplicaciones de manera eficiente.

Créditos: imágenes licencia de Flaticon.com

Tema 2 Conceptos básicos de programación

Programas y algoritmos

Procesamiento de la información

El diccionario de la RAE define la **informática**¹ como el "*conjunto de conocimientos científicos y técnicas que hacen posible el tratamiento automático de la información mediante ordenadores*".

Wikipedia define la **información** como un "*conjunto organizado de datos procesados, que constituyen un mensaje que cambia el estado de conocimiento del sujeto o sistema que recibe dicho mensaje*", mientras que WordReference la define como "*conjunto de datos sobre una materia determinada*".

Wikipedia define un **dato** como "*una representación simbólica (numérica, alfabética, algrítmica, espacial, etc.) de un atributo o variable cuantitativa o cualitativa*".

Por ejemplo, la información sobre una persona puede estar compuesta de datos como: nombre, apellidos, fecha de nacimiento, sexo, dirección, estado civil, ocupación, etc. Algunos datos, como la edad, pueden calcularse a partir de otros, como la fecha de nacimiento y la fecha actual.

El tratamiento de la información plantea **problemas** cuya proposición es hallar un dato desconocido a partir de otros datos conocidos. Los ordenadores se utilizan para solucionar este tipo de problemas.

Ejemplos de problemas sencillos:

- Dadas la base y la altura de un triángulo, calcular su superficie.
- A partir de los datos de partidos jugados, ganados y empatados por los equipos de una liga deportiva, listar la clasificación.

Algoritmos

Los ordenadores se utilizan para solucionar problemas de tratamiento de información. Para que un ordenador solucione un problema, hay que proporcionarle un programa que le indique con todo detalle cómo solucionarlo.

La RAE define **algoritmo**² como "*un conjunto ordenado y finito de operaciones que permite hallar la solución de un problema*". Wikipedia da una definición parecida: "*un conjunto prescrito de instrucciones o reglas bien definidas, ordenadas y finitas que permiten llevar a cabo una actividad mediante pasos sucesivos que no generen dudas a quien deba hacer dicha actividad*".

¹ El término "**informática**" es de influencia francesa: *informatique*, que reúne *información* y *automática*. El término en inglés cuyo significado más se aproxima, *computing*, no es tan usado, prefiriéndose otros términos según el enfoque o el sub-área. Aunque en inglés existe la palabra *informatics*, su uso es limitado. En el ámbito soviético, la informática se entendía como parte de la **cibernética**.

² La palabra "**algoritmo**" tiene su origen remoto en el nombre de la región de Jorasnia, de la que era oriundo un matemático persa autor de un conocido tratado medieval. La palabra "guarismo" tiene el mismo origen.

En el contexto de los problemas de tratamiento de la información, un algoritmo describe la solución de un problema en función de los datos necesarios para representar un caso concreto del problema y de los pasos requeridos para obtener el resultado deseado.

Por ejemplo, para explicar cómo multiplicar dos números (sin decimales) a una persona que solo sabe sumar, se podría usar una descripción como la siguiente:

"Dados dos números, que llamaremos multiplicando y multiplicador, el resultado se obtiene al sumar el multiplicando tantas veces como indique el multiplicador. Por ejemplo, si se quiere multiplicar 5 por 3, el resultado es: $5+5+5=15$ ".

La descripción de la multiplicación como una repetición de sumas sería el algoritmo. El ejemplo (que no es parte del algoritmo) ilustra un caso concreto del problema, representado por dos datos: multiplicando = 5 y multiplicador = 3. La aplicación del algoritmo a este caso concreto conduce al resultado de 15. Al variar el multiplicando y/o el multiplicador, se obtienen casos diferentes del problema de la multiplicación.

Nótese que los algoritmos, en sí mismos, no requieren ordenadores: un folleto para montar una estantería de Ikea es un algoritmo que se supone va a ejecutar una persona. Incluso los algoritmos de tratamiento de información, como una operación matemática o el cálculo de una nómina, pueden ser ejecutados por una persona, aunque los ordenadores suelen hacerlo más rápido.

Programa y lenguaje

Para que un ordenador solucione un problema, se le debe dar un programa que le indique cómo solucionarlo. Wikipedia define un **programa informático** como *"una secuencia de instrucciones, escritas para realizar una tarea específica en una computadora"*. Nótese el parecido con la definición de algoritmo: *"conjunto prescrito de instrucciones o reglas bien definidas, ordenadas y finitas que permiten llevar a cabo una actividad mediante pasos sucesivos que no generen dudas a quien deba hacer dicha actividad"*.

Un algoritmo se puede expresar de muchas maneras, siempre que lo entienda quien deba ejecutarlo (realizar los pasos que indica). Una de estas maneras es un programa informático, que es, básicamente, un algoritmo escrito en un lenguaje de programación.

Según Wikipedia, un **lenguaje de programación** es *"un lenguaje formal que comprende un conjunto de instrucciones que producen diversos tipos de resultados (obtener datos del usuario u otra fuente externa, realizar operaciones matemáticas, comparar datos, mostrar resultados en una pantalla, ...)"*. Los lenguajes de programación se utilizan en la programación de computadoras para implementar algoritmos.

Los algoritmos describen la solución a un problema en términos de los datos necesarios para representar un caso del problema y del conjunto de pasos necesarios para producir el resultado deseado. Los lenguajes de programación deben proporcionar una manera de representar tanto el proceso como los datos; con este fin ofrecen instrucciones básicas, estructuras de control, tipos de datos y estructuras de datos.

Los lenguajes de programación se clasifican de diferentes formas. Una de ellas consiste en atender a su nivel de abstracción, lo que permite distinguir entre lenguajes de alto nivel y lenguajes de bajo nivel, en función de su proximidad a la arquitectura de la máquina. El procesador entiende un lenguaje específico, formado por secuencias de ceros y unos, que se

conoce como **código máquina**. La dificultad que supone para las personas un nivel de abstracción tan bajo condujo a la creación de los lenguajes de ensamblador (*assembly language*), que constituyen un conjunto de mnemónicos (palabras cortas como "ADD", "SUB", "MOV", "JMP") que se corresponden directamente con instrucciones en lenguaje máquina, pero resultan más fáciles de entender y memorizar por las personas.

Los **lenguajes de alto nivel** utilizan una sintaxis próxima al lenguaje natural (generalmente con léxico inglés), pero muy simplificada, lo que implica un nivel de abstracción mucho más accesible para los programadores, y permite la realización de programas complejos.

Los programas escritos en alto nivel deben convertirse a código máquina para su ejecución, utilizando compiladores o intérpretes, que son, a su vez, programas desarrollados con ese propósito.

Un **compilador** traduce un programa completo, generando un archivo ejecutable con el programa equivalente en código máquina. Este proceso se realiza una vez, y el archivo resultante puede ejecutarse tantas veces como se quiera.

Un **intérprete**, en cambio, no genera un código máquina permanente, sino que traduce y ejecuta el programa instrucción por instrucción, a medida que se van requiriendo durante la ejecución, por lo que una nueva ejecución requiere repetir este proceso.

La compilación, por tanto, permite ejecuciones más rápidas, aunque con el coste de una traducción inicial. Además, un compilador, al intentar traducir el programa completo antes de su ejecución, es capaz de detectar errores en las instrucciones con antelación, mientras que un intérprete solo puede encontrarlos durante la ejecución.

Python

Python es un lenguaje de programación interpretado cuya filosofía hace hincapié en una sintaxis que favorezca un código legible.

Se utiliza en una gran variedad de ámbitos, desde el desarrollo web a devops (DevOps es un acrónimo inglés de *development* (desarrollo) y *operations* (operaciones), que se refiere a una metodología de desarrollo de software que se centra en la comunicación, colaboración e integración entre desarrolladores de software y los profesionales de sistemas en las tecnologías de la información), pero su crecimiento se ha acelerado notablemente por su aplicación en *machine learning* y *data science*.



Python es un lenguaje de programación potente y fácil de aprender. Cuenta con estructuras de datos de alto nivel eficientes y un enfoque simple pero efectivo para la programación orientada a objetos. Su elegante sintaxis y tipado dinámico, junto con su naturaleza interpretada, lo convierten en un lenguaje ideal para la creación de *scripts* (programas simples que sirven para realizar diversas tareas interactuando con el sistema operativo y el usuario) y el rápido desarrollo de aplicaciones en muchas áreas en la mayoría de las plataformas.

El intérprete de Python y su extensa biblioteca estándar están disponibles gratuitamente en formato fuente o binario para todas las plataformas principales desde el sitio web de Python, <https://www.python.org/>, y pueden distribuirse libremente. El mismo sitio también contiene distribuciones y enlaces a muchos módulos, programas y herramientas de Python de terceros, y documentación adicional.

¡Hola, mundo!

¡Hola, mundo!

Normalmente, al empezar a aprender un lenguaje de programación, el punto de arranque es un programa clásico conocido como "¡Hola, mundo!", que lo único que hace es mostrar un saludo en la pantalla del ordenador. En Python es realmente simple:

```
print("¡Hola, mundo!")
```

[ver ejecución >>>](#)

En Python, la función *print()* muestra ("imprime") un mensaje. El resultado de la ejecución del programa anterior es:

```
¡Hola, mundo!
```

Nótese que el mensaje a imprimir se encierra entre comillas (pueden ser dobles o simples), las cuales no forman parte del mensaje. La instrucción *print()* admite varios valores, separados por comas, que imprimirá uno detrás de otro, utilizando por defecto un espacio en blanco como separador.

```
print("¡Hola", "mundo!")
```

[ver ejecución >>>](#)

```
¡Hola, mundo!
```

Cuando se muestran múltiples valores, se puede cambiar el separador con el parámetro *sep*:

```
print("Hola", "mundo", sep="-")
```

[ver ejecución >>>](#)

```
Hola-mundo
```

La instrucción *print()* añade un salto de línea al final del mensaje mostrado:

```
print("Hola")
print("mundo")
```

[ver ejecución >>>](#)

```
Hola
mundo
```

Se puede evitar el salto de línea seleccionando un terminador con el parámetro *end*:

```
print("Hola", end = "//")
print("mundo")
```

[ver ejecución >>>](#)

```
Hola//mundo
```

Help()

Una instrucción como *print()* admite diferentes opciones, como, por ejemplo, seleccionar el separador cuando se imprimen varios valores. Para saber qué opciones ofrece una instrucción cualquiera, se puede usar la instrucción *help()*, pasándole entre paréntesis y comillas el nombre de la instrucción sobre la que se busca información:

```
help("print")
```

Resultado:

```
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file: a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.

(END)
```

Para salir y continuar, pulsar *q* (inicial de *quit*).

Si se usa *help()* sin nada entre los paréntesis, se obtiene acceso a la ayuda general de Python, donde se puede interactuar para pedir ayuda sobre una instrucción concreta:

```
help()
```

Resultado:

```
Welcome to Python 3.5's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at http://docs.python.org/3.5/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules. To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics". Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".

help> █
```

En este caso, para salir del entorno de ayuda basta pulsar la tecla *↵* (*Intro* o *Enter*).

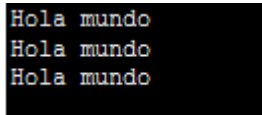
Secuencias de instrucciones

La secuencia de instrucciones es la estructura más básica de los programas. Para escribir una secuencia de instrucciones en Python, solo hay que ponerlas una tras otra, una por línea. El siguiente programa imprime tres veces el mismo mensaje:

```
print("Hola mundo")
print("Hola mundo")
print("Hola mundo")
```

[ver ejecución >>>](#)

Resultado:

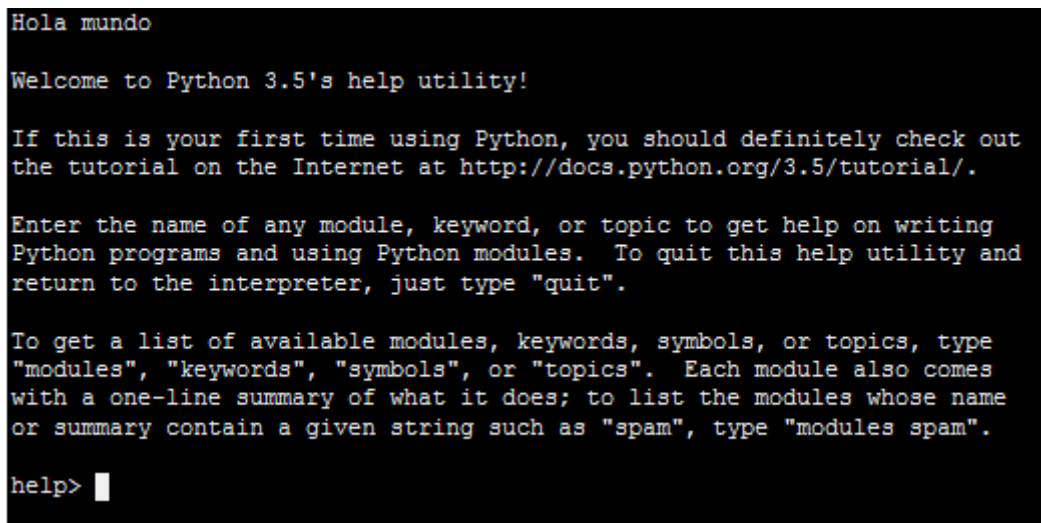


```
Hola mundo
Hola mundo
Hola mundo
```

Las instrucciones en una secuencia se ejecutan de arriba abajo en el orden en que están escritas. El siguiente programa escribe "Hola mundo" y luego muestra la ayuda general de Python:

```
print("Hola mundo")
help()
```

Resultado:



```
Hola mundo

Welcome to Python 3.5's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at http://docs.python.org/3.5/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

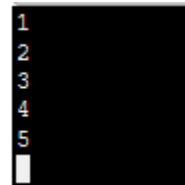
To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics".  Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".

help> 
```

Este otro escribe los números del 1 al 5:

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

[ver ejecución >>>](#)



```
1
2
3
4
5

```

Nótese que los números, a diferencia de los mensajes de texto, no se encierran entre comillas.

Comentarios

Además de las instrucciones, en un programa se pueden incluir **comentarios**, que sirvan para ayudar a comprender el código y su propósito a la persona que lo lea. En Python, el tipo básico de comentario es un texto que empieza con el símbolo #. El siguiente código incluye tres comentarios:

```
# Se muestra tres veces el texto Hola mundo
print("Hola mundo")
print("Hola mundo")
print("Hola mundo")

# Se muestran los números del 1 al 4
print(1)
print(2)
print(3)
print(4)

# Se muestra la ayuda del comando print
help("print")
```

El propósito de los comentarios no es dar información obvia, que puede deducirse fácilmente del código, como en el ejemplo anterior, sino explicar aspectos no evidentes o documentar el código de cara a su futura reutilización o mantenimiento.

En una misma línea se pueden incluir una instrucción antes del símbolo #, pero todo lo que esté después de este símbolo hasta el final de la línea es un comentario, independientemente de su contenido:

```
print("Hola, mundo") # muestra un saludo
```

Entrada de datos, variables

Entrada de datos

Un algoritmo describe la solución de un problema en función de los datos necesarios para representar un caso concreto del problema y de los pasos necesarios para obtener el resultado deseado. El tratamiento de la información plantea problemas cuyo propósito es hallar un resultado desconocido a partir de datos conocidos.

Para que un ordenador resuelva un caso de un problema, teniendo el programa para hacerlo, es necesario proporcionarle los datos del caso. Esto se puede hacer mediante instrucciones de entrada. Véase el siguiente programa:

```
print("Hola ¿Cómo te llamas?")
name = input()
print("Hola", name)
```

[ver ejecución >>>](#)

El programa usa dos veces la instrucción *print()*, que es una **instrucción de salida** (para dar información), y aparece una instrucción nueva, *input()*, que es una **instrucción de entrada** (para recoger datos que van a ser usados por el programa).

El resultado de la ejecución de este programa, suponiendo que el usuario introdujera por teclado el nombre "Pedro", sería el siguiente:

```
Hola ¿Cómo te llamas?  
Pedro  
Hola Pedro
```

También aparece el elemento *name*, que no es una instrucción, sino el nombre de una variable. Una **variable** se puede ver como una etiqueta que se usa para hacer referencia a un dato manejado por el programa.

El valor del dato referenciado por una variable puede ser distinto en diferentes ejecuciones del programa (por ejemplo, si el programa anterior se ejecuta para otro usuario llamado "Juan") e incluso puede variar durante la propia ejecución (de ahí el nombre de variable). Esta capacidad de variación de los datos es lo que permite que un programa resuelva diferentes casos de un mismo problema.

El efecto de la ejecución de la instrucción *input()* es que el programa espera a que el usuario teclee el valor del dato deseado y pulse la tecla ↵ (denominada *Intro*, o en inglés *Enter*). Una vez hecho esto, el valor se recoge y se asigna a la variable que está a la izquierda del símbolo de asignación =, tal como indica el código mostrado más arriba. Como el programa se detiene hasta que el usuario introduce el valor de un dato, es una buena práctica mostrar previo indicando qué dato es el que se espera que introduzca. Esto se hace en el ejemplo anterior con la primera instrucción *print()*, pero se puede abreviar usando la propia instrucción *input()*:

```
name = input("Hola ¿Cómo te llamas? ")  
print("Hola", name)  
ver ejecución >>>
```

El resultado de la ejecución de este programa es:

```
Hola ¿Cómo te llamas? Pedro  
Hola Pedro
```

Nótese que, en este caso, la entrada del valor se espera en la misma línea donde se muestra el mensaje que lo solicita. Por ello, en la instrucción *input* se ha añadido un espacio después del signo de interrogación '?', para evitar que el nombre tecleado por el usuario se visualice pegado al signo de puntuación. (Si el usuario tecleara un espacio antes del nombre, dicho espacio pasaría a formar parte del contenido de la variable *name*, lo cual, en principio no es deseable).

En ambas versiones del programa, la última instrucción muestra un saludo personalizado que incluye el valor del dato introducido por el usuario:

```
print("Hola", name)
```

La variable *name* no se encierra entre comillas. Las comillas sirven para indicar un valor literal tal cual (por ejemplo, la palabra "Hola" en este caso). En el caso de la variable *name*, lo que se quiere mostrar no es la palabra *name*, sino el dato referenciado por la variable (como "Pedro" en el ejemplo).

Los nombres de las variables

Los programas usan variables para hacer referencia a los datos del problema que resuelven: datos de inicio, que varían de un caso del problema a otro; datos intermedios, generados durante la resolución del problema y que pueden variar a medida que se ejecuta el programa; y datos finales, que contienen los resultados que conforman la solución del problema.

Las variables tienen un nombre y hacen referencia al valor de un dato almacenado en la memoria del ordenador, accesible durante la ejecución del programa.

Las **reglas léxicas** de Python requieren que los nombres de las variables empiecen por una letra mayúscula (A–Z), una letra minúscula (a–z)) o un guion bajo/subrayado (_), pudiendo continuar con cualquier combinación de letras, guiones bajos y dígitos (0–9):

```
nombre
NOMBRE
_nombre_1
fecha_de_nacimiento
xASFasDasd24_d
```

Las **reglas de estilo** de Python establecen que los nombres de las variables deben escribirse en minúsculas, usando guiones bajos para separar palabras cuando se trate de nombres compuestos. Además, se sugiere que sean lo más descriptivos posible respecto al dato que representan:

```
nombre
NOMBRE
_nombre_1
fecha_de_nacimiento
*ASFasDasd24_d
```

Los valores de las variables

Los programas manejan datos usando variables, las cuales tienen un nombre (identificador) y hacen referencia (contienen) al valor de un dato. Para asociar (asignar) un valor a una variable, se usa una instrucción de **asignación**, caracterizada por el símbolo =.

```
name = input() # asigna a name un valor introducido por el usuario
greeting = "Hola, mundo" # asigna a greeting un texto literal
```

Se puede definir un dato como "*una representación simbólica (numérica, alfabética, algorítmica, espacial, etc.) de un atributo cualitativo o una magnitud cuantitativa*". Existen muchos tipos de datos diferentes. En el ejemplo anterior, a las variables *name* y *greeting* se les asignan valores que representan pequeños textos. Los valores que representan texto se conocen como *strings*. Una **string** es simplemente una secuencia de caracteres. Las *strings* pueden usarse para representar cosas como nombres, notas u observaciones, entre otros.

Otros posibles tipos de datos que se pueden asociar a una variable son:

- **Números enteros** (sin parte fraccionaria):

```
age = 21
```

- **Números reales** o de punto flotante (que incluyen parte fraccionaria, separada por un punto, no por una coma):

```
temperature = 36.5
```

- **Números complejos** (con parte real y parte imaginaria; la unidad imaginaria se representa por *j*):

```
complex_number = -34+5.3j
```

- **Valores booleanos** ([Álgebra de Boole](#)), que se representan con *True* (verdadero) o *False* (falso):

```
bool_data = True
```

Los valores booleanos pueden ser el resultado de una comparación u otro tipo de operación:

```
bool_data = 8 > 5 # se asigna el valor False porque 8 < 5
```

Nótese que solo los literales de valores de tipo texto (*strings*) se representan encerrados entre comillas.

Las *strings* y los números (incluyendo los booleanos) son tipos básicos en Python. Además, Python predefine otros tipos de datos (tuplas, listas, diccionarios, etc.) o los incorpora mediante librerías externas. También ofrece al programador la posibilidad de definir nuevos tipos según las necesidades concretas del programa.

Asignación

Los programas manejan datos usando variables. Las variables tienen un nombre y hacen referencia al valor de un dato. Para asociar un (nuevo) valor a una variable, se usa una instrucción de **asignación**, caracterizada por el símbolo `=`. A la izquierda del `=` se indica una variable y, a su derecha, un valor (ya sea un literal o el resultado de un cálculo u operación).

La instrucción de asignación funciona de la siguiente manera:

1. Se obtiene el valor resultante del lado derecho del `=`.
2. Se asigna dicho valor a la variable del lado izquierdo, sustituyendo al valor que contuviera previamente.

A una variable se le puede asignar:

- **El resultado de una operación de entrada:** `var1 = input()`
- **Un valor expresado literalmente** (nótese que solo las *strings* literales se encierran entre comillas):

```
var2 = 35
var3 = "Hola"
```

- **El resultado de un cálculo** (se asigna el valor resultante):

```
var4 = 7 * 5 + 18 # a var4 se le asigna el valor 53
```

- **El valor de otra variable** (para tipos básicos de Python, se asigna una copia del valor de la variable de la derecha):

```
var5 = var4
```

- **El resultado de una expresión** que puede incluir variables:

```
var6 = var4 + var5 - 10 # Se asigna 96 a var6
```

- **La propia variable como parte de la expresión** (el mismo nombre de variable puede aparecer a ambos lados del =, pero significa cosas distintas: en el lado derecho se toma su valor actual para realizar un cálculo, mientras que en el lado izquierdo indica el destino del nuevo valor):

```
var6 = var6 + 1 # el valor asignado a var6 es 97
```

- **Formas abreviadas para expresiones comunes**, cuando la variable forma parte de la expresión:

```
var6 += 1 # equivale a var6 = var6 + 1
var6 *= 2 # equivale a var6 = var6 * 2
```

Una vez vistos los ejemplos, puede resumirse que una instrucción de asignación se compone siempre de un símbolo = con una expresión a su derecha (que puede ser tan simple como un literal o una variable, o contener operaciones), cuyo valor resultante se asigna a la variable de la izquierda.

Asignación múltiple

Los programas manejan datos usando variables. Las variables tienen un nombre y hacen referencia al valor de un dato. Para asociar un valor con una variable, se usa una instrucción de asignación, representada por el símbolo =. El funcionamiento de la asignación consiste en: primero se evalúa la expresión que está a la derecha del símbolo de asignación y, luego, el resultado de dicha evaluación se asigna a la variable que está a la izquierda del símbolo de asignación.

```
var1 = 35
```

Además de lo anterior, Python permite asignar múltiples valores a múltiples variables en una sola instrucción de asignación:

```
a, b, c = 1, 2, 3
```

A la derecha de la asignación debe haber tantas expresiones, separadas por comas, como variables haya a la izquierda. Los valores de las expresiones se asignan en paralelo ("simultáneamente") de izquierda a derecha a las variables correspondientes: el primer valor a la primera variable, el segundo valor a la segunda variable, etc. En el ejemplo anterior, el valor 1 se asigna a la variable *a*, el valor 2 a la variable *b* y el valor 3 a la variable *c*.

La asignación múltiple permite, entre otras cosas, intercambiar cómodamente los valores de dos variables con una sola instrucción (en el ejemplo *a* recibe el anterior valor de *b* y, simultáneamente, *b* recibe el anterior valor de *a*).

```
a, b = b, a
```

Esta operación, en muchos otros lenguajes, requeriría tres instrucciones y una variable adicional:

```
aux = a
a = b
b = aux
```

La variable adicional (*aux* en el ejemplo) sirve para guardar el valor de la primera variable que se va a modificar, de forma que no se pierda y pueda asignarse posteriormente a la otra.

Entrada de valores no textuales

La instrucción *input* recoge los datos, sin importar su tipo, como una secuencia de caracteres (*string*), tal como se introducen por teclado.

```
name = input("¿Cómo te llamas? ")
```

Si la secuencia de caracteres introducidos se puede interpretar como un valor válido de otro tipo primitivo (entero, real, complejo), se debe indicar explícitamente que se quiere hacer esa conversión. El caso de los booleanos es especial:

```
text = input("Dame una string formada por una secuencia de dígitos: ")
integer_number = int(input("Dame un número entero: "))
real_number = float(input("Dame un número real: "))
complex_number = complex(input("Dame un número complejo: "))
bool_value = bool(int(input("Dame un booleano (0->False, 1->True): ")))
```

[ver ejecución >>>](#)

Resultado de la ejecución (se resaltan los valores introducidos por el usuario):

```
Dame una string formada por una secuencia de dígitos: 23289
Dame un número entero: 23289
Dame un número real: 23289.0
Dame un número complejo: 14+23j
Dame un booleano (0->False, 1->True): 0
```

Nótese el uso de *int*, *float* o *complex* para indicar la conversión requerida. Las *strings* (secuencias de caracteres) se identifican como *str*, pero sería redundante indicarlo en una instrucción de entrada, dado que ese es el tipo por omisión de los datos introducidos por teclado.

El caso de *bool* es especial: no basta con teclear *False* para indicar este valor, por lo que se recurre a los *int* 0 y 1, que luego se convierten a su equivalente booleano.

En caso de que la secuencia de caracteres tecleada por el usuario no pueda interpretarse como un valor del tipo requerido, se produce un error.

Cada tipo de datos utiliza una representación interna en memoria diferente como secuencia de bits (ceros y unos), empleando en cada caso una codificación distinta. Incluso en los casos representados por las variables *text* e *integer_number* del ejemplo, que responden a la misma secuencia de caracteres de entrada ("23289"), su almacenamiento es distinto. En el primer caso, cada carácter (de "cifra") se representa en binario por separado de acuerdo con su codificación estándar (por ejemplo, cada '2' se representa como 00110010), mientras que en el segundo la secuencia completa se convierte a su valor numérico binario equivalente (en notación denominada **complemento a dos**), concretamente 0·010110101111001 (23289 en binario).

El nombre del tipo *float*, que se usa para los números reales, se debe a que utiliza una representación interna llamada *floating point* (coma flotante), análoga a la notación científica de las calculadoras (que usa un exponente). En este formato, la coma decimal (el punto) no se encuentra siempre justo a la izquierda de la parte fraccionaria (como en la *coma fija*), sino que "flota" en combinación con el exponente. Por ejemplo, 123.45 (escrito en coma fija) puede

representarse internamente como $0.12345 * 10^3$ (en realidad, algo similar pero adaptado a binario). Es decir, el punto ha "flotado" hasta justo antes de la primera cifra distinta de cero, adecuándose al mismo tiempo el exponente. De esta manera, el número queda representado por un par de enteros: en el ejemplo 12345 (la **mantisa**) y 3 (el **exponente**). El tamaño de la mantisa (y del exponente) es limitado, por lo que en muchos casos solo puede representarse un número real de forma aproximada.

Tipos de datos, operadores

Tipos de datos

Los programas manejan datos usando variables. Wikipedia define un dato como "una representación simbólica (numérica, alfabética, algorítmica, espacial, etc.) de un atributo o variable cuantitativa o cualitativa".

Existen datos de diferentes tipos: texto, como el nombre de una persona; números, como la edad de una persona; fechas, como la de nacimiento de una persona, entre otros.

Los tipos de datos estándar de Python incluyen números (enteros, reales, complejos y booleanos), *strings* (texto), tuplas, listas y diccionarios. Las *strings*, las tuplas y las listas se agrupan en la categoría de **secuencias**.

Por el momento, se abordarán solamente los tipos numéricos y las *strings*.

Números

Los números en Python pueden ser: enteros (*int*), reales (*float*) y complejos (*complex*). La siguiente tabla muestra ejemplos de literales numéricos que representan valores de cada uno de estos tipos:

int	float	complex
10	0.0	3.14j
100	15.20	45.j
-786	-21.9	9.322e-36j
0o70	32.3e+18	.876j
-0o490	-90.	-.6545+0J
-0x260	-32.54e100	3e+26J
0x69	7E-12	4.53e-7j

- En los literales de tipo *int*, los prefijos '0o' o '0O' antes de las cifras indican que el número está expresado en **base 8** (octal), en lugar de la habitual base 10 (decimal), y los prefijos '0x' o '0X' indican que está en **base 16** (hexadecimal). En cualquier caso, el valor numérico se representará internamente por su equivalente en base 2 (binario) que corresponda. Por ejemplo, los literales 10, 0o12 y 0xa representan el mismo valor entero: diez.
- En los literales *float*, la presencia de un **punto decimal** y/o una **notación exponencial** (e o E, que se significa "por diez elevado a") determina el tipo real.
- En los números complejos (*complex*), se añade la unidad imaginaria j o J, que debe ir precedida de un número (incluso 0).

Todos los tipos numéricos admiten las siguientes operaciones (en los ejemplos se usan enteros por simplicidad):

Operación	Resultado
x + y	Suma de x más y: 5 + 2 da 7

Operación	Resultado
$x - y$	Resta de x menos y: 5 - 2 da 3
$x * y$	Producto de x por y: 5 * 2 da 10
x / y	Cociente real de dividir x entre y: 5 / 2 da 2.5
$x // y$	Cociente entero de dividir x entre y: 5 // 2 da 2 <i>El número de veces que el divisor cabe entero en el dividendo: 3.14 // 0.7 da 4.0</i> <i>(Si el dividendo o el divisor son float, el resultado es float, si ambos son int, el resultado es int)</i>
$x \% y$	Resto de la división entera $x // y$: 5 % 2 da 1; 3.14 % 0.7 da 0.34 <i>Se mantiene el signo del divisor: 5 % -2 da -1</i>
$-x$	Valor negativo de x: 5 si x vale -5
$+x$	Valor de x sin cambio: -5 si x vale -5
<code>abs(x)</code>	Valor absoluto de x: abs(-5) da 5
<code>divmod(x, y)</code>	Tupla con $(x // y, x \% y)$: divmod(5, 2) da (2, 1)
<code>pow(x, y)</code>	Potencia (x elevado a y): pow(5, 2) da 25
$x ** y$	Potencia alternativa: 5 ** 2 da 25

Valores booleanos

El tipo *bool* solo dispone de dos valores posibles, representados por los literales *True* y *False*. En Python se considera un **subtipo de int**, por lo que *False* y *True* también tienen los valores numéricos 0 y cualquier valor entero distinto de cero, respectivamente.

Los siguientes valores numéricos se evalúan a *False* cuando se utilizan en contextos donde se espera un valor booleano (en caso contrario, se tomaría el valor *True*):

- El valor cero de cualquier tipo numérico: 0 (*int*), 0.0 (*float*), 0j (*complex*).
- Una secuencia vacía. Por ejemplo, la string "", compuesta de cero caracteres (aunque no por ello deja de ser una *string*).
- El valor especial representado por el literal *None*. Representa la "ausencia de valor", aunque paradójicamente es un valor válido y único del tipo *NoneType*.

En general, a cualquier valor x del tipo que sea se le puede aplicar la función *bool(x)*. Esta función devolverá un valor booleano (*True* o *False*) de acuerdo con los criterios mencionados. Por ejemplo:

Operación	Resultado
<code>bool(False)</code>	False
<code>bool(True)</code>	True
<code>bool("")</code>	False
<code>bool("Hola")</code>	True
<code>bool(0)</code>	False
<code>bool(None)</code>	False
<code>bool(3.14)</code>	True

Los operadores *or*, *and* y *not* operan normalmente sobre valores de tipo *bool*:

Operación	Resultado
<code>x or y</code>	Devuelve <i>True</i> si al menos uno de los operandos es <i>True</i> . El resultado es <i>False</i> solo si los dos son <i>False</i> . <i>Los operandos se evalúan de izquierda a derecha, de manera que si bool(x) es True, el resultado es el valor de x, no evaluándose y; en caso contrario, el resultado será el valor de y.</i>
<code>x and y</code>	Devuelve <i>True</i> solo si los dos operandos son <i>True</i> . El resultado es <i>False</i> si cualquiera es <i>False</i> . <i>Los operandos se evalúan de izquierda a derecha, de manera que si bool(x) es False, el resultado es el valor de x, no evaluándose y; en caso contrario, el resultado será el valor de y.</i>
<code>not x</code>	Negación: devuelve <i>True</i> si <i>x</i> se evalúa como <i>False</i> , y <i>False</i> si <i>x</i> se evalúa como <i>True</i> .

Este comportamiento puede aprovecharse para construir expresiones compactas y eficientes, ya que **se evita evaluar operandos innecesarios**, una técnica conocida como **evaluación en cortocircuito** (*short-circuit evaluation*).

Strings

Las *strings* representan secuencias contiguas de caracteres. Los literales de valores de tipo *string* (*str*), se encierran entre comillas dobles o simples:

```
var1 = "Hola"
var2 = 'hola'
print(var1, var2)
ver ejecución >>>
```

```
Hola hola
```

La **longitud** de una *string* (número de caracteres) se obtiene programáticamente usando la función *len()*:

```
greetings = "Hello world"
print("La longitud de '", greetings, "' es", len(greetings))
ver ejecución >>>
```

```
La longitud de 'Hello world' es 11
```

Las strings permiten acceder a sus caracteres por **índices**. Se puede acceder a un carácter concreto de una string escribiendo la *string*, o el nombre de la variable que la referencia, seguida del índice de la posición que ocupa el carácter deseado entre corchetes. El índice de la primera posición es 0.

```
greetings = "Hello world"
print(greetings[0]) # Muestra el primer carácter de greetings:
H
print(greetings[4]) # Muestra el quinto carácter de greetings:
o
ver ejecución >>>
```

También se pueden usar índices negativos, relativos a la longitud de la string:

```
greetings = "Hello world"
print(greetings[-1]) # Muestra el último carácter de greetings
d
print(greetings[-5]) # Muestra el quinto carácter, desde el final
w
```

[ver ejecución >>>](#)

Se puede obtener una *substring* (subsecuencia) de una *string* usando el operador de **segmento** o **slice** [:]. El segmento deseado se identifica indicando el índice del primer elemento y el índice de la posición siguiente al último elemento. Si se omite el segundo valor, se toma el segmento desde el primer índice hasta el final; si se omite el primero, toma desde el principio hasta la posición del índice anterior al indicado:

```
str_var = "Hello world"
print(str_var[2:5]) # Muestra los caracteres de 3° al 5°:
llo
print(str_var[2:]) # Muestra los caracteres a partir del 3°:
llo world
print(str_var[:5]) # Muestra los caracteres hasta el 5°:
Hello
```

[Ver ejecución >>>](#)

El diccionario de la RAE define **concatenar** como "unir dos o más cosas". En el caso de las *strings*, concatenar dos *strings* consiste en formar una nueva *string* juntando, en orden, dos *strings* preexistentes.

Python permite:

- **Concatenar** *strings* con el signo más (+) como operador de concatenación de strings.
- **Repetir** una *string* con el asterisco (*) como operador de repetición (equivalente a realizar una concatenación de la misma string repetidas veces).

```
greetings = "Hello world"
print('a' + 'a')
aa
print(greetings + "TEST")
Hello worldTEST
print('ab' * 5)
ababababab
print(greetings * 2)
Hello worldHello world
```

[ver ejecución >>>](#)

Se Podría decir que la repetición de *strings* es a la concatenación de *strings* lo que la multiplicación de números enteros es a la suma de números enteros. Nótese, que la concatenación junta directamente las *strings*, sin añadir ningún carácter en medio; si se desea un espacio u otro separador, debe incluirse explícitamente.

Gestión de tipos de datos

Conocer el tipo de un dato

Se puede conocer el tipo de un dato usando la operación `type()`.

```
a = 200
b = "200"
t1 = type(a)
t2 = type(b)
print(t1)
print(t2)
```

[ver ejecución >>>](#)

```
<class 'int'>
<class 'str'>
```

Nótese que el valor devuelto por `type()` es un valor de tipo `type`:

```
print(type(t1))
```

```
<class 'type'>
```

El valor devuelto, aunque se muestra con el formato de los ejemplos, es uno de los tipos de datos básicos (`int`, `float`, `str`, etc.):

```
print(t1 == int)
```

```
True
```

Si se quiere obtener una `string` que contenga únicamente el nombre del tipo, se puede consultar el atributo `__name__` (un **atributo** es una propiedad asociada a un dato):

```
typeName = t1.__name__
print(typeName)
print(type(typeName))
```

```
int
<class 'str'>
```

[ver ejecución >>>](#)

Conversión de tipos (*type casting*)

En programación, se conoce como *type casting* a la obtención, a partir de un valor de un tipo *T1*, de un valor "similar" de otro tipo *T2*, lo cual no siempre es posible. Si *T1* y *T2* son el mismo tipo, la conversión es trivial: se obtiene un valor igual de ese tipo.

En Python, la conversión se realiza escribiendo el nombre del tipo de destino seguido entre paréntesis del valor del tipo original:

- `int()` construye un número entero a partir de: un número real (truncándolo, esto es, eliminando la parte fraccionaria), o una *string* (siempre que la string represente un número entero).

```
a = int(10)      # a toma el valor entero 10
b = int(3.7)     # b toma el valor entero 3
c = int("40")    # c toma el valor entero 40
s = "25"
d = int(s)       # d toma el valor entero 25
```

[ver ejecución >>>](#)

- *float()* construye un número real a partir de: un número entero, o una *string* (siempre que la *string* represente un número real o un número entero)

```
a = float(10)      # a toma el valor real 10.0
b = float(3.5)     # b toma el valor real 3.5
c = float("40")    # c toma el valor real 40.0
s = "25.0"
d = float(s)       # d toma el valor real 25.0
```

[ver ejecución >>>](#)

- *complex()* construye un número complejo a partir de: un número entero, un número real, o una *string* (siempre que la *string* represente un número real, un número entero o un número complejo)

```
a = complex(10)    # a toma el valor complejo 10+0j
b = complex(3.5)   # b toma el valor complejo 3.5+0j
c = complex("40")  # c toma el valor complejo 40+0j
s = "25"
d = complex(s)     # d toma el valor complejo 25+0j
e = complex(1+2j)  # e toma el valor complejo 1+2j
```

[ver ejecución >>>](#)

- *str()* construye una *string* a partir de casi cualquier otro tipo de datos.

```
a = str(10)        # a toma el valor string "10"
b = str(3.5)       # b toma el valor string "3.5"
c = str("40")      # c toma el valor string "40"
s = "25"
d = str(s)         # d toma el valor string "25"
e = str(1+2j)      # e toma el valor string "1+2j"
```

[ver ejecución >>>](#)

Anotación de tipos

Python es un lenguaje de programación con **tipado dinámico**, lo que significa que el tipo de una variable (es decir, el tipo del dato asociado a la variable) se establece en el momento de asignarle un valor, y este tipo cambiará si se le asigna un nuevo valor de un tipo diferente. A diferencia de los lenguajes con **tipado estático**, en los que se requiere declarar de qué tipo es cada variable, —la cual solo podrá contener valores de dicho tipo—, en Python no se requiere dicha declaración. En estos lenguajes estáticos, la asignación de un valor de un tipo distinto puede requerir una conversión de tipo (casting); de lo contrario, se producirá un error al intentar asignarle un valor de tipo distinto.

Considérense las siguientes asignaciones en Python:

```
a = 0      # el tipo de a es int
b = 0.0    # el tipo de b es float
c = "0"    # el tipo de c es str
a = 0.0    # el tipo de a cambia a float
```

No obstante, Python permite "anotar" el tipo de una variable, lo cual solo tiene un efecto informativo, equivalente a un comentario, acerca del tipo de valor que se espera que contenga:

```
a: int = 0      # el tipo de a es (y debiera seguir siendo) int
b: float = 0.0  # el tipo de b es (y debiera seguir siendo) float
```

```
c: str = "0"      # el tipo de c es (y debiera seguir siendo) str
```

A diferencia de los lenguajes con tipado estático, en Python a una variable que haya sido anotada con un tipo se le puede asignar más tarde un valor de un tipo diferente. Sin embargo, existen herramientas que pueden analizar el código y señalar situaciones que podrían suponer un riesgo de que, durante la ejecución, el valor almacenado no se ajuste a la intención manifestada en la anotación.

Operadores

Operadores de comparación

Estos operadores son aplicables a muchos tipos de datos; comparan dos valores del mismo tipo y devuelven un valor booleano en función de la relación entre ellos, por lo que técnicamente se les denomina **operadores relacionales**.

Supóngase, en el siguiente ejemplo, que la variable *a* vale 10 y la variable *b*, 20:

operador	descripción	ejemplo
<code>==</code>	Compara si son iguales	<code>a == b</code> da <i>False</i>
<code>!=</code>	Compara si son distintos	<code>a != b</code> da <i>True</i>
<code>></code>	Compara si el primero es mayor que el segundo	<code>a > b</code> da <i>False</i>
<code><</code>	Compara si el primero es menor que el segundo	<code>a < b</code> da <i>True</i>
<code>>=</code>	Compara si el primero es mayor o igual que el segundo	<code>a >= b</code> da <i>False</i>
<code><=</code>	Compara si el primero es menor o igual que el segundo	<code>a <= b</code> da <i>True</i>

Los operadores de igualdad (`==` y `!=`) pueden comparar valores de tipos diferentes, aunque el resultado, obviamente, es siempre *False*.

Un error muy común consiste en usar un solo carácter de igualdad en lugar del operador de comparación `==`, lo que convierte la expresión en una asignación, en vez de una comparación.

Precedencia de operadores

Una **expresión** es una combinación de operandos y operadores. En el siguiente ejemplo, el símbolo `+` es un operador obviamente de adición, y los números 3 y 8 son sus operandos (aunque también podrían ser variables o cualquier otro elemento del lenguaje que produjera un valor, en este caso, aritmético):

`3 + 8`

La expresión anterior es una expresión bastante simple, ya que solo tiene un operador (incluso podría no tener ninguno y constar de un único "operando"). Sin embargo, en una expresión puede haber más de un operador. Cuando esto ocurre, las **reglas de precedencia** determinan el orden de aplicación de los distintos operadores.

Por ejemplo, la multiplicación (representada por el `*`) tiene una precedencia mayor que la adición. Por tanto, la expresión:

`3 + 8 * 2`

dará como resultado 19, y no 22, que sería el resultado si los operadores se aplicaran en el orden en que están escritos, o sea, de izquierda a derecha. Más precisamente, dicho valor 19 se obtiene aplicando en primer lugar el operador $*$, ya que tiene mayor precedencia. Se calcula primero $8 * 2$, obteniéndose el valor 16, y a continuación se aplica el operador $+$, empleando el resultado anterior como su operando por la derecha —los operandos, como se ve, pueden ser simples o resultantes de cálculos previos—. El cálculo final es $3 + 16$, que da como resultado 19. Este mismo esquema se aplicaría para los distintos operadores, de acuerdo con las reglas de precedencia, cuando se combinan en expresiones con múltiples operandos.

No obstante, el orden de las operaciones predefinido se puede modificar usando paréntesis (tantos como se considere necesario). Así, en el siguiente ejemplo se fuerza el cálculo de $3 + 8$ antes de la multiplicación, a pesar de la menor precedencia de $+$ respecto a $*$:

$(3 + 8) * 2$

El siguiente listado muestra parte del orden de precedencia predefinido de los operadores en Python. En la lista, x representa el operando en los operadores unarios, mientras que los demás son operadores binarios, es decir, requieren dos operandos (los operadores binarios típicamente son infijos, o sea, van entre sus operandos; por ejemplo: $0 \text{ not in } v$).

Cuanto más arriba esté un operador en la lista, mayor será su precedencia:

1. $**$ (Exponenciación)
2. $+x, -x, \sim x$ (Operadores unarios)
3. $*, /, //, \%$ (Multiplicación, división, módulo)
4. $+, -$ (Suma y resta binarias)
5. $<<, >>$ (Desplazamiento de bits)
6. $\&$ (AND a nivel de bits)
7. $\wedge$ (XOR a nivel de bits)
8. $|$ (OR a nivel de bits)
9. $==, !=, >, >=, <, <=, \text{is}, \text{is not}, \text{in}, \text{not in}$ (Comparaciones e identidad)
10. $\text{not } x$ (NOT booleano)
11. and (AND booleano)
12. or (OR booleano)

Obsérvese que algunos operadores comparten símbolo. Por ejemplo, el carácter $-$ puede ser unario (alta precedencia) o binario (de algo menor precedencia). Por tanto, suponiendo que la variable n tenga el valor 2, la expresión $-n-3$ se calcula aplicando primero el $-$ de la izquierda, que es unario (al no tener operando a su izquierda), por lo que tiene mayor precedencia.), obteniendo el valor -2. A continuación se aplica el $-$ binario restándole 3, resultando -5. Naturalmente, también puede escribirse $-(n-3)$. En este caso, se fuerza primero el cálculo entre paréntesis, obteniendo el valor -1, y luego se aplica el $-$ unario, resultando 1.

Las reglas de precedencia se aplican siempre. Incluso se podría escribir: $3*-n$, y como el $-$ es unario (a su izquierda no tiene operando, sino el operador $*$), tiene mayor precedencia que $*$, por lo que se obtendría primero el valor -2, realizándose a continuación la multiplicación, resultando -6.

Asociatividad de operadores

Como se puede observar, hay algunos operadores que tienen igual precedencia y que naturalmente, podrían aparecer en una misma expresión. También puede ocurrir que el mismo operador (por ejemplo, el - binario) aparezca varias veces en la expresión, lógicamente en todas las ocurrencias con su (igual) precedencia.

En estos casos de igual precedencia, dicha propiedad no resulta suficiente para determinar completamente el orden de cálculo, por lo que se recurre a las **reglas de asociatividad**. La gran mayoría de los operadores binarios son asociativos por la izquierda, o sea, dentro de un mismo nivel de precedencia se aplican empezando por el operador más a la izquierda y siguiendo hacia la derecha.

Por ejemplo, en la expresión $9 - 2 * 2 + 3$, primero se evalúa el operador '*', dando $9 - 4 + 3$; luego el operador '-', resultando $5 + 3$; y, por último, el operador '+', dando como resultado 8.

Si la asociatividad de los operadores binarios + y - fuese de derecha a izquierda, el resultado de la expresión anterior sería: $9 - 2 * 2 + 3 \rightarrow 9 - 4 + 3 \rightarrow 9 - 7 \rightarrow 2$. Por conveniencia, el operador de exponenciación (**) es asociativo por la derecha. Así: $2 ** 3 ** 2$ se interpreta que el 2 de la izquierda está elevado a "3 elevado a 2", lo que equivale a $2 ** 9$, resultando 512.

Si ** tuviera asociatividad de izquierda a derecha el resultado sería:

$$2 ** 3 ** 2 \rightarrow 8 ** 2 \rightarrow 64$$

En este caso, se obtendría un resultado diferente.

Tiene poco sentido que los operadores de comparación sean asociativos, y en Python se interpretan de forma especial. Por ejemplo, la expresión:

`a < b < c == d`

se interpreta como: `a < b and b < c and c == d`

(aunque b y c aparezcan repetidos, se evalúan una sola vez).

Obsérvese que el operador *and* tiene menor precedencia y es asociativo (por supuesto, por la izquierda).

Anexo: Resumen de reglas de estilo PEP 8 de escritura de código en Python

PEP8 es una guía de estilo oficial para escribir código en Python. A continuación, se presentan algunos de sus puntos más importantes:

Formateo de Código

- **Indentación:** Se deben utilizar 4 espacios para la indentación en lugar de tabuladores.
- **Longitud de línea:** No debe superar los 79 caracteres para el código y 72 para los comentarios y *docstrings*.
- **Líneas en blanco:** Se recomienda usar líneas en blanco para separar funciones, clases y bloques de código dentro de funciones.
- **Comentarios de línea:** Deben usarse con moderación, es decir, solo cuando el código no es autoexplicativo.
- **Comentarios:** Deben ser frases completas y servir para aclarar el código.
- Deben utilizarse *docstrings* para documentar módulos, clases y funciones.

```
"""Esto es una docstring."""  
  
if a == 0: # Esto es un comentario de línea  
    a = 5  # Instrucción indentada 4 espacios
```

Espaciado

- Se deben añadir espacios alrededor de los operadores y después de las comas.
- No deben añadirse espacios alrededor del símbolo = en argumentos de funciones y llamadas a funciones.

```
def function(arg1, kwarg1=value):  
    pass  
  
x = 1 + 2
```

Nomenclatura

- **Nombres de funciones y variables:** Se debe usar el modelo *snake_case*, todo en minúscula y las palabras separadas por guiones bajos.
- **Nombres de clases:** Se debe usar el modelo *CamelCase*, uniendo todas las palabras y comenzando en mayúscula.
- **Constantes:** Deben escribirse en *UPPERCASE_WITH_UNDERSCORES*.
- **Variables protegidas:** Debe colocarse un guion bajo antes del nombre para indicar que una variable o método es protegido (por ejemplo, *_protected_var*).
- **Variables privadas:** Deben comenzar con dos guiones bajos antes del nombre para indicar que una variable o método es privado (por ejemplo, *__private_var*).

División de una línea muy larga

En Python, hay diferentes maneras de dividir una línea que supere la longitud máxima recomendada (79 caracteres):

Uso de paréntesis

Se pueden usar paréntesis para dividir expresiones entre varias líneas:

```
result = (  
    variable1 + variable2  
    - variable3 * variable4  
)
```

**Uso de la barra inversa **

Es posible utilizar una barra inversa para indicar que la línea continúa:

```
result = variable1 + variable2 \  
    - variable3 * variable4
```

División de strings

Para strings, el intérprete concatena automáticamente las strings entre paréntesis en distintas líneas:

```
string = (  
    "This is a very long string that "  
    "needs to be divided into several lines."  
)
```

Indentación de la continuación de línea

Cuando se requiera una indentación adicional para la continuación de línea, pueden seguirse dos convenciones:

- **Alineado con el delimitador abierto:**

```
my_list = [  
    1, 2, 3,  
    4, 5, 6,  
]
```

- **Añadiendo 4 espacios de indentación adicional:**

```
argument1 = 1  
argument2 = 2  
  
if argument1 - 1 + argument2 // 8 ==\  
    some_function(argument1 + 2): # Doble sangrado  
    variable = 8 # Sangrado normal de dependencia del if
```

Expresiones

- **Comparaciones:** Se recomienda evitar comparar valores booleanos con *True* o *False* usando `==` o `!=`.

```
# Correcto
if my_var:
    ...
if not my_var:
    ...
```

```
# Incorrecto
if my_var == True:
    ...
if my_var != True:
    ...
```

Importaciones

- Las importaciones se deben colocar en la parte superior del archivo. Deben organizarse en tres grupos:
 1. Librerías estándar
 2. Librerías de terceros
 3. Módulos propios
- Cada grupo debe separarse con una línea en blanco.

```
import os # Librerías estándar
import sys

from numpy import sqrt # Librería de terceros

from my_local_module import my_function # Módulo propio
```

Otros aspectos de estilo

- **Operadores ternarios:** Si se usan operadores ternarios (a if test else b), debe garantizarse que la expresión sea fácil de leer.

```
# Correcto
x = 4 if condition else 5
```

```
# Incorrecto
x = (4
    if condition else 5)
```

Depuración de programas

Depuración

La programación es una tarea intelectual extraordinariamente compleja, en consecuencia, es frecuente que un programa no salga bien a la primera. Aunque, en general, funcione, no es raro que haya algunos casos en los que se cuelgue, de resultados erróneos, o muestre cualquier otro tipo de comportamiento inesperado.

Para descubrir estos comportamientos erróneos los programas se someten a exhaustivas baterías de pruebas con muchos conjuntos de datos diferentes de los que se conocen los resultados esperados que se conocen como casos de prueba. Aquellos casos cuyo resultado no coincida con el esperado ponen de manifiesto la existencia de un fallo o error en el programa que es necesario subsanar.

La depuración (ingles *debugging*) es la tarea de analizar el código para buscar las causas de los fallos y errores y corregirlos. Esto se podría hacer leyendo el código, pero, lo normal, es usar herramientas de depuración que existen para ayudar a realizar la tarea. Las herramientas de depuración pueden ofrecer multitud de opciones, pero la tarea en sí se basa en dos funciones principales, que estarán disponibles en todas las herramientas:

- Ejecución paso a paso
- Inspección de las variables

La ejecución paso a paso implica que las instrucciones se ejecutarán una a una a petición del depurador (humano) que está revisando el programa. Tras ejecutar cada instrucción la ejecución se detendrá y esperará la orden de ejecutar la siguiente.

La inspección de las variables implica que, cuando el programa está parado esperando la orden de ejecutar la siguiente instrucción, el depurador (humano) podrá examinar los valores de las variables del programa, y, en general de cualquier elemento de la memoria relacionado con la ejecución de este, para comprobar si siguen siendo correctos o se han modificado incorrectamente al ejecutar la última instrucción.

Cuando los programas tienen más de unas pocas líneas, ejecutarlos paso a paso para encontrar la causa de un error que podemos sospechar que se encuentra en una sección del código que se ejecutará relativamente tarde puede resultar tedioso, por ello, las herramientas de depuración ofrecen, junto a las anteriores, una tercera función:

- Establecer puntos de parada (*breakpoints*)

Esta función permite marcar cualquier línea del código para que la ejecución se detenga al alcanzarla, antes de ejecutar las instrucciones que contenga, y permita inspeccionar el estado del programa. La ejecución, en vez de ir instrucción por instrucción puede empezar desde el principio hasta el primer punto de interrupción, continuar de cada punto de interrupción al siguiente y desde el último al final, ejecutando en bloque las instrucciones intermedias cuyo efecto inmediato no se considere necesario inspeccionar.

Estas funciones de las herramientas de depuración pueden servir a un propósito distinto que localizar las causas de los fallos de un programa, por importante que esto sea: un programador novel podría usarlas en un programa correcto para observar cómo se ejecuta y comprender

mejor los intrínquilos de la programación y cómo funcionan los diferentes elementos de un programa.

En las siguientes secciones se verá como estas funciones básicas se implementan en diferentes herramientas de depuración que se podrán usar con Python y VPL.

Depuración web (Python tutor)

Inicio

Python Tutor es una web que ofrece depurar código escrito en algunos de los lenguajes: Python (<https://pythontutor.com/python-compiler.html#>), Java, C++ o JavaScript. Tiene algunas limitaciones, como la versión de Python, el tamaño máximo del programa a depurar o que éste tiene que estar contenido en un único archivo. Se accede a una página que contiene un pequeño ejemplo (Figura 1).

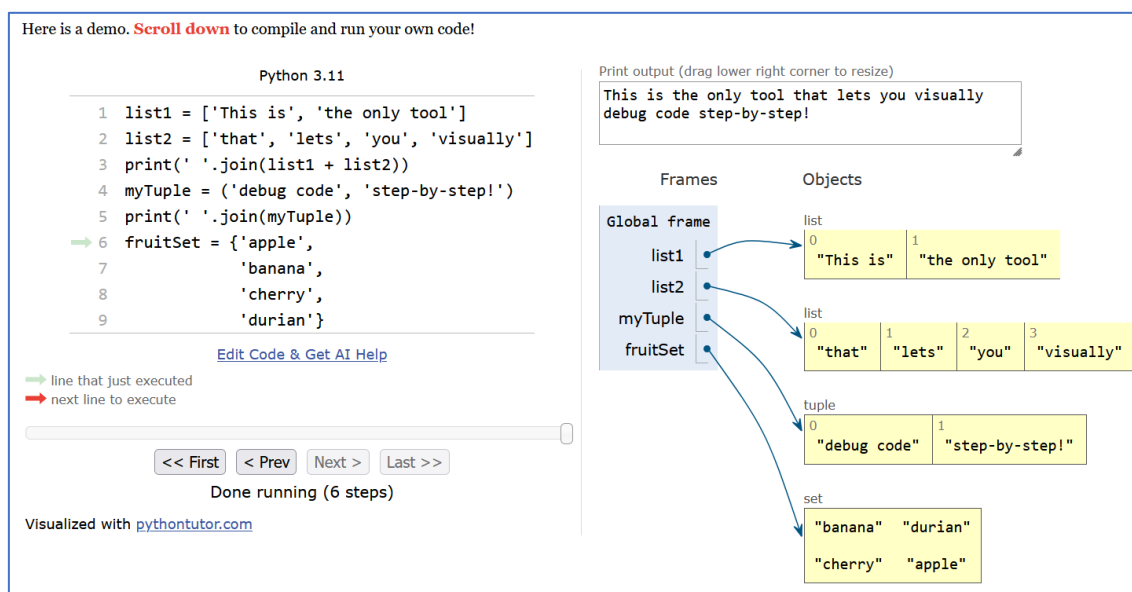


Figura 1

Pulsando en el enlace “Edit Code & Get AI Help” que está debajo del código del ejemplo, se puede editar y sustituir por nuestro propio código.

Algunas actividades VPL están configuradas para usar este depurador y, al pulsar el botón de depuración de VPL () arrancan la página con el código de la actividad ya incorporado (Figura 2).



Figura 2

Puntos de parada

Para poner un punto de parada, solo se necesita picar con el ratón sobre la línea en la que queremos situarlo, la cual cambiará de color (rojo en la Figura 3) para indicar que hay un punto de parada. Para quitarlo se hace lo mismo.

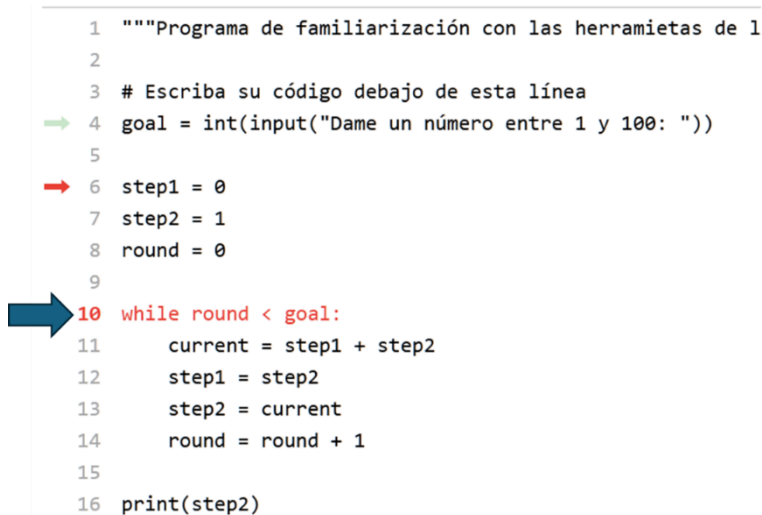


Figura 3

Ejecución paso a paso

Cuando el programa está parado, dos flechas, una gris y otra roja, nos señalan, respectivamente, la última línea ejecutada y la próxima línea que se ejecutará. Debajo de la ventana del código hay cuatro botones (Figura 4) que permiten ejecutar la siguiente línea (*Next*) retroceder a la anterior (*Prev*), ejecutar hasta el final (*Last*) o volver a empezar (*First*). Además, sobre los botones hay una barra de desplazamiento que permite mover la ejecución hacia adelante y hacia atrás.

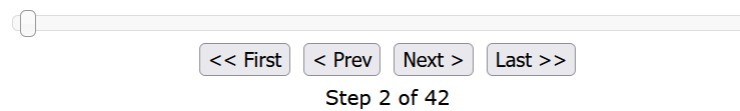


Figura 4

Si hay algún punto de parada en el código que resta por ejecutar, el botón *Next* no se detendrá en la siguiente línea, sino que continuará hasta el siguiente punto de parada. De la misma manera, el botón *Prev* retrocederá hasta el punto de parada más próximo, si lo hay.

En caso de solicitarse algún dato al usuario, debajo de los botones anteriores aparece un área para introducirlo (Figura 5). Mientras no se haga no se podrá continuar la ejecución.

Enter user input:

Dame un número entre 1 y 100:

Figura 5

Inspección

La ventana de Python Tutor (Figura 6) está dividida verticalmente en dos: a la izquierda aparece el código que se está depurando y los controles descritos en el apartado anterior, mientras que a la derecha aparece, en la parte superior, un recuadro que representa la consola del programa, donde se muestran los mensajes dirigidos al usuario, y, debajo de ésta, toda la información (variables y demás) que el programa maneja. De esta manera, la inspección del estado del programa cuando está detenido resulta muy sencilla.

Python 3.6
[known limitations](#)

```

1  """Programa de familiarización con las herramientas de 1
2
3  # Escriba su código debajo de esta línea
4  goal = int(input("Dame un número entre 1 y 100: "))
5
6  step1 = 0
7  step2 = 1
8  round = 0
9
10 while round < goal:
11     current = step1 + step2
12     step1 = step2
13     step2 = current
14     round = round + 1
15
16 print(step2)

```

[Edit this code](#)

→ line that just executed
→ next line to execute

Print output (drag lower right corner to resize)

Dame un número entre 1 y 100: 7
 21

Frames

Objects

Global frame

goal	7
step1	13
step2	21
round	7
current	21

<< First
< Prev
Next >
Last >>

Figura 6

Depuración con un depurador gráfico (Thonny)

Inicio

Thonny (<https://thonny.org/>) es un entorno integrado de desarrollo (IDE: Integrated Development Enviroment) para principiantes que se puede descargar e instalar localmente. Tiene todo lo necesario para programar, incluida una herramienta de depuración. Algunas actividades VPL están configuradas para usar la herramienta de depuración de Thonny. Para iniciar el IDE de depuración en VPL se debe pulsar el botón de depuración, en la barra de herramientas de VPL (Figura 7).

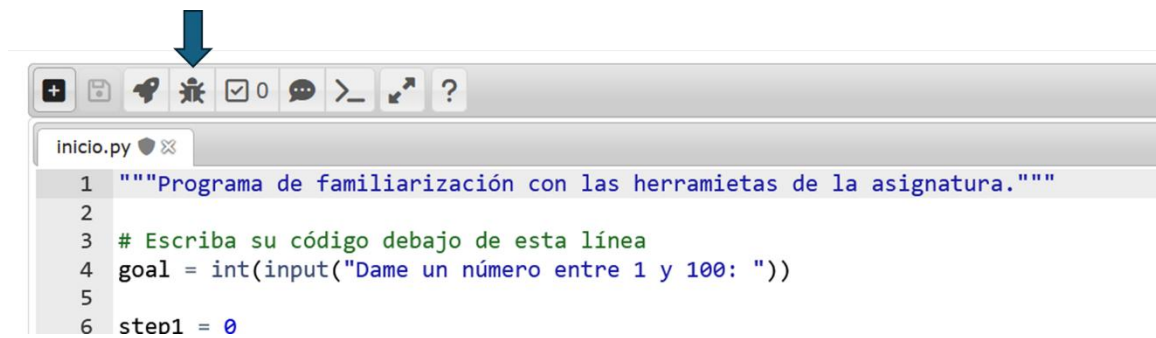


Figura 7

Se abre la terminal en modo gráfico, mostrando la carátula de arranque de Thonny. Debemos pulsar el botón etiquetado como: **Let's go!** (Figura 8).



Figura 8

Nos aparece el IDE de Thonny (Figura 9).

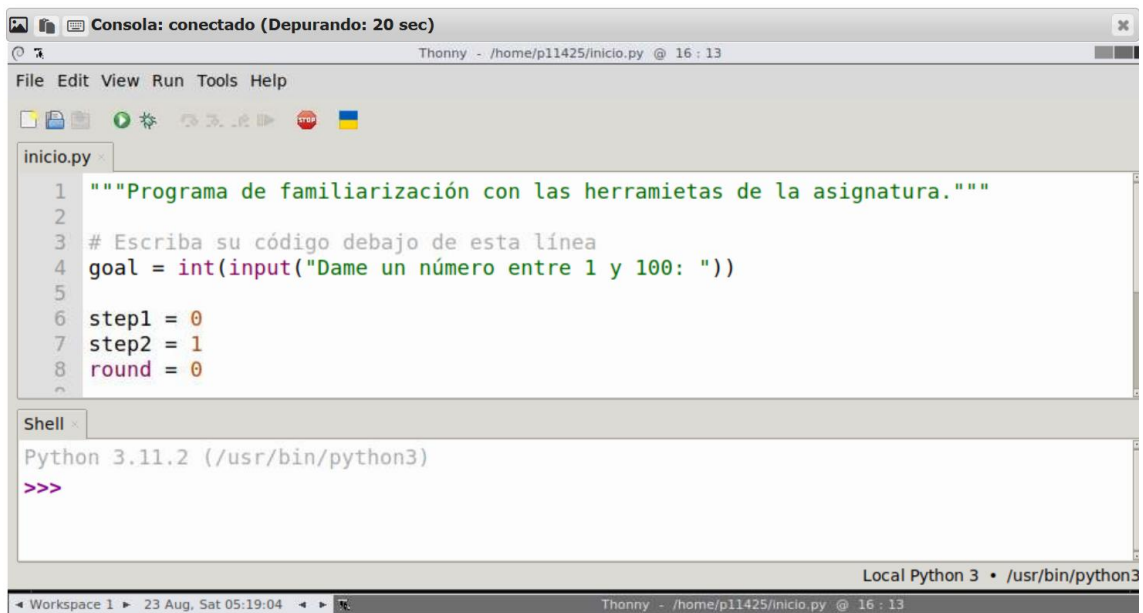


Figura 9

Para iniciar la depuración, hay que pulsar el botón de depuración de Thonny (Figura 10):



Figura 10

Puntos de parada

Se puede poner un punto de parada picando con el ratón sobre el número de la línea en la que se quiera poner. Aparecerá un punto rojo junto a la línea indicando la presencia del punto de parada (Figura 11). Para quitarlo se hace lo mismo.

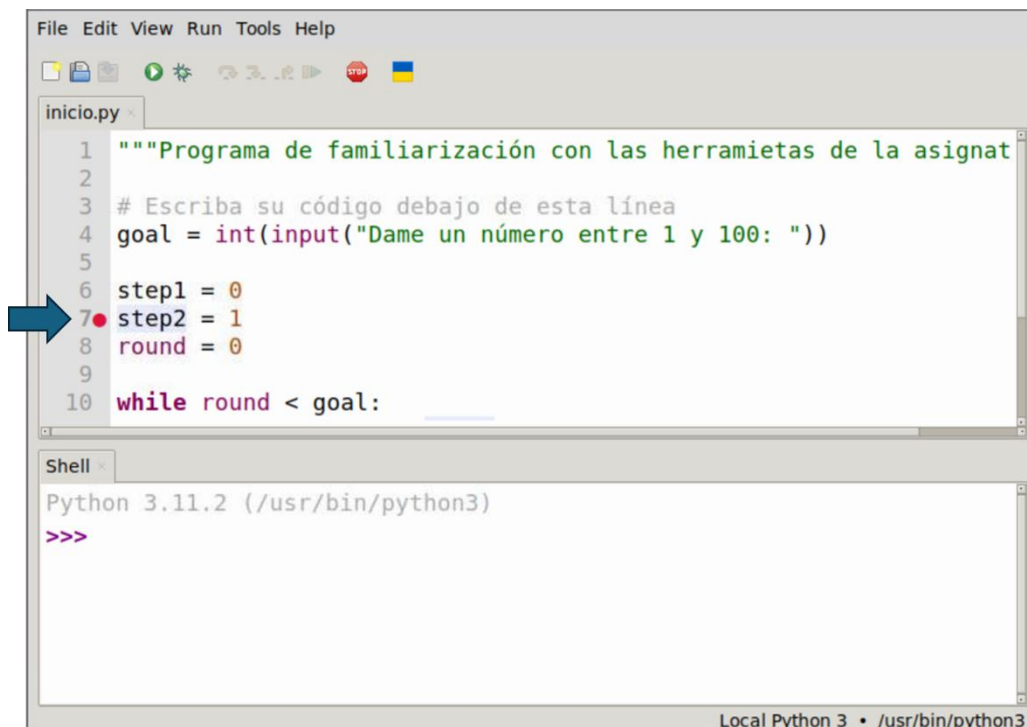


Figura 11

Ejecución paso a paso

Cuando el programa está parado, la siguiente línea que se va a ejecutar se muestra resaltada (en la Figura 12 con fondo amarillo):

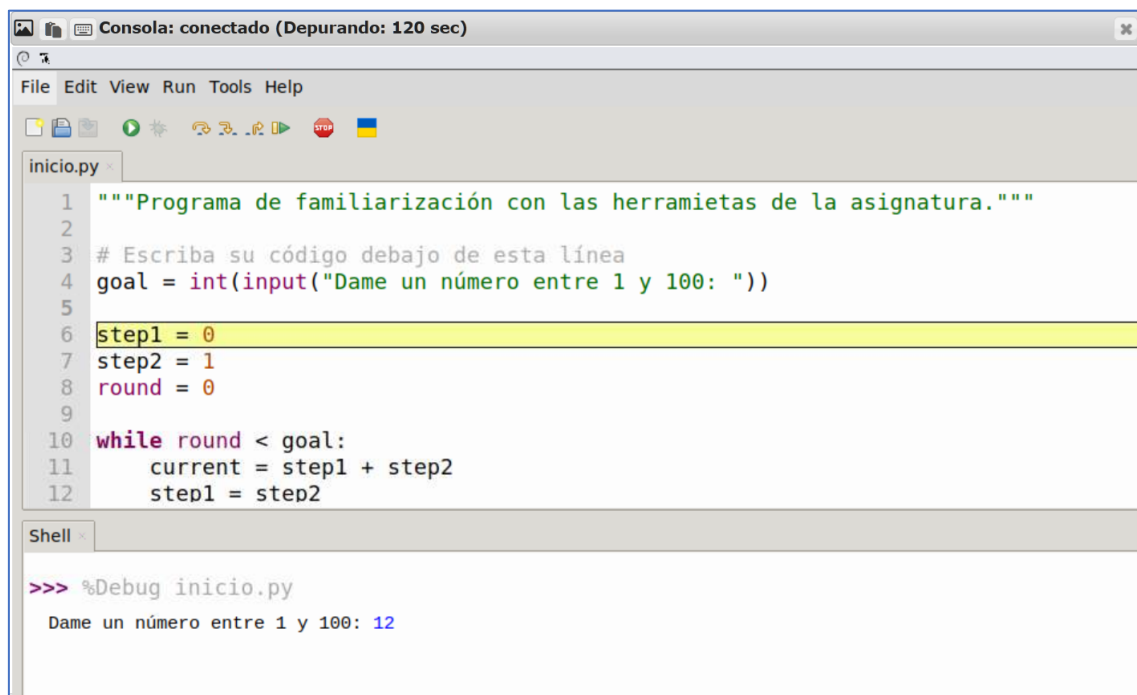


Figura 12

Al iniciar el modo depuración, en la barra de herramientas se habrán activado cuatro botones (Figura 13): *Step over*, *Step into*, *Step out* y *Resume*:



Figura 13

El botón *Step over* ejecuta la línea actual y se detiene en la siguiente. El botón *Step into* hace lo mismo, pero, si la línea actual ejecuta una rutina separada (función) se detiene en la primera línea del código de la función; si estamos ejecutando el código de una función, el botón *Step out*, ejecuta las líneas que queden de la misma y se detiene en la siguiente línea del programa, detrás de la que iniciaba la ejecución de la función.

El botón *Resume* ejecuta hasta el siguiente punto de parada, o hasta el final, si no hay ningún punto de parada en el código restante.

La interacción con el usuario se lleva a cabo en la ventana *Shell*, que, normalmente aparece debajo de la ventana del editor (Figura 14).

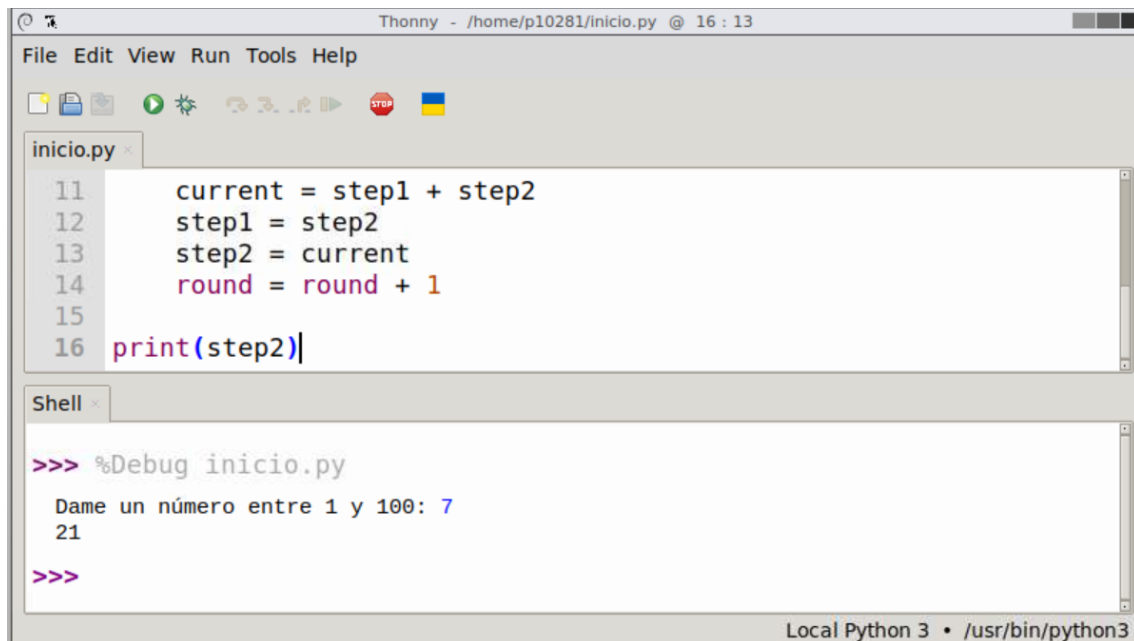


Figura 14

En esta ventana se muestran los mensajes dirigidos al usuario, y se espera la introducción de datos por parte de éste, en el caso de que se soliciten.

Inspección

La opción View del menú de Thonny permite seleccionar cualquier elemento relevante del estado del programa para visualizarlo. En la Figura 15 se ha seleccionado mostrar la ventana de Shell (está normalmente seleccionada por defecto), que aparece en la parte inferior y la ventana de variables, que aparece a la derecha. La ventana de variables muestra las variables activas del programa, con su nombre y su valor asociado.

En la Figura 16 se ha seleccionado, además, la ventana del Heap, que aparece debajo de la de variables. El Heap es la zona de la memoria donde se almacenan los datos que está usando el programa. Al mostrarse la ventana del Heap, la ventana de variables cambia y, en vez de mostrar el valor asociado a cada variable, muestra la dirección de memoria del Heap en el que se almacena. En la ventana del Heap se muestran las direcciones de memoria y los valores almacenados en ellas.

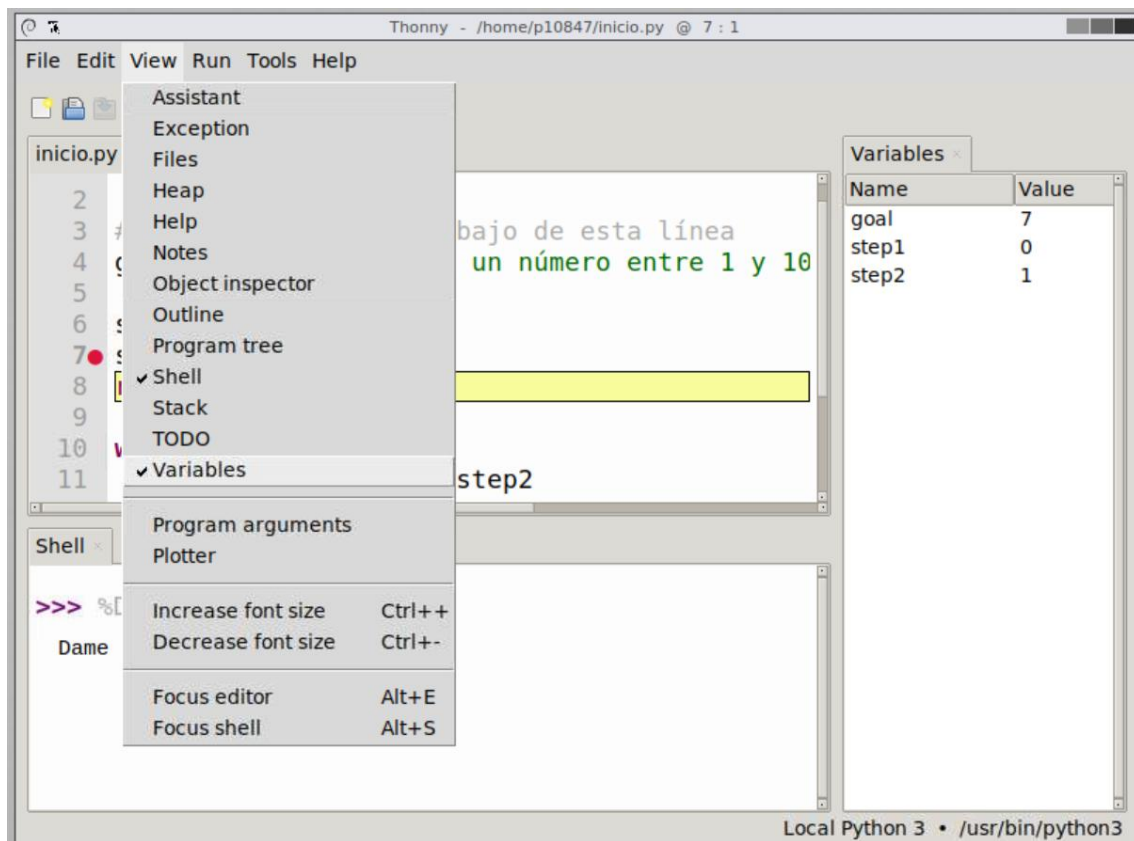


Figura 15

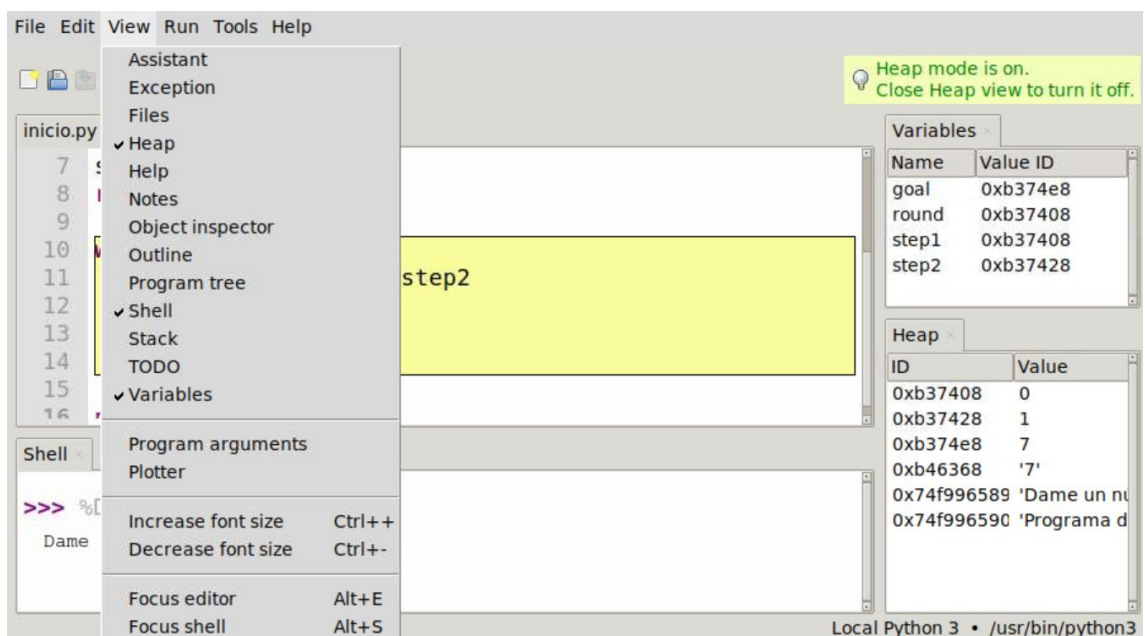
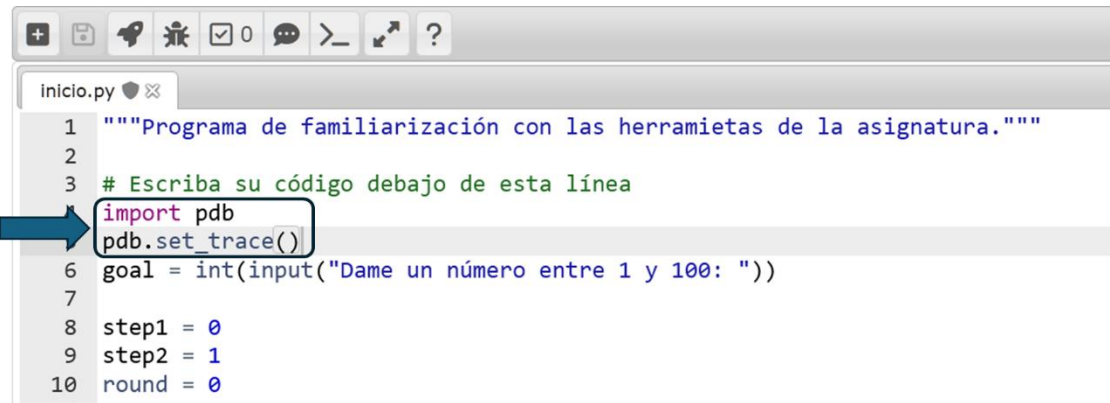


Figura 16

Depuración con el depurador integrado de Python (pdb)

Inicio

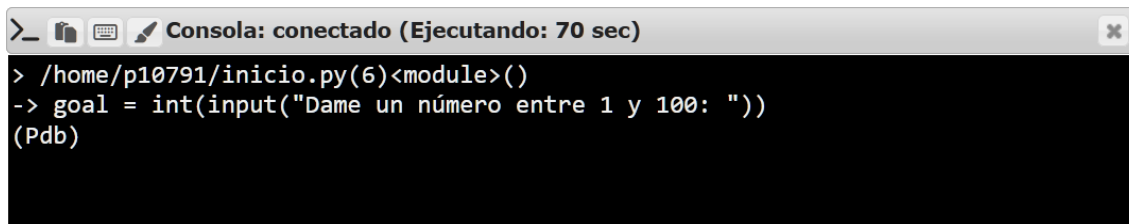
Python dispone de un módulo, llamado *pdb* (python debugger), que sirve como herramienta de depuración. Para iniciar la depuración de un programa, sólo hay que importar ese módulo y ejecutar la instrucción *pdb.set_trace()*, como se muestra en la Figura 17



```
1 """Programa de familiarización con las herramientas de la asignatura."""
2
3 # Escriba su código debajo de esta línea
4 import pdb
5 pdb.set_trace()
6 goal = int(input("Dame un número entre 1 y 100: "))
7
8 step1 = 0
9 step2 = 1
10 round = 0
```

Figura 17

Esta instrucción hace que, al ejecutar el programa, se arranque el depurador y se quede esperando órdenes para continuar la ejecución del programa, que queda detenido justo en la línea siguiente a la que contiene la instrucción *pdb.set_trace()*, como se ve en la Figura 18



```
>_ Consola: conectado (Ejecutando: 70 sec)
> /home/p10791/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb)
```

Figura 18

Obsérvese que, justo antes del prompt del depurador (*Pdb*) hay una flecha y el texto de la línea en la que el programa está parado, la línea 6 de nuestro programa de ejemplo. Un comando muy útil es este momento, es *help*, que nos mostraría todos los comandos del depurador. El comando *help* seguido de otro comando (por ejemplo: *help next*), nos da ayuda sobre ese comando en particular.

```
> _ Consola: conectado (Ejecutando: 37 sec)
> /home/p10718/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb) help

Documented commands (type help <topic>):
=====
EOF      c          d          h          list       q          rv          undisplay
a        cl        debug      help       ll         quit       s          unt
alias    clear     disable   ignore     longlist   r          source     until
args     commands display   interact   n          restart    step       up
b        condition down      j          next       return     tbreak     w
break    cont      enable    jump       p          retval     u          whatis
bt       continue exit      l          pp         run        unalias    where

Miscellaneous help topics:
=====
exec  pdb

(Pdb)
```

Figura 19

Puntos de parada

Para establecer puntos de parada usamos el comando *break*, o su versión abreviada, *b*, seguido del número de la línea en la que queremos establecerlo. En la Figura 20 se establecen dos puntos de parada, uno en la línea 10 y otro en la 16.

```
> _ Consola: conectado (Ejecutando: 28 sec)
> /home/p10510/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb) b 10
Breakpoint 1 at /home/p10510/inicio.py:10
(Pdb) b 16
Breakpoint 2 at /home/p10510/inicio.py:16
(Pdb) █
```

Figura 20

Usando el mismo comando, pero sin número de línea, obtenemos una lista de los puntos de parada establecidos en el programa (Figura 21).

```
(Pdb) b
Num Type      Disp Enb  Where
1  breakpoint  keep yes  at /home/p10510/inicio.py:10
2  breakpoint  keep yes  at /home/p10510/inicio.py:16
(Pdb)
```

Figura 21

Para quitar un punto de parada, se usa la instrucción `clear` seguida del número del breakpoint (no el número de línea), tal como muestra la Figura 22.

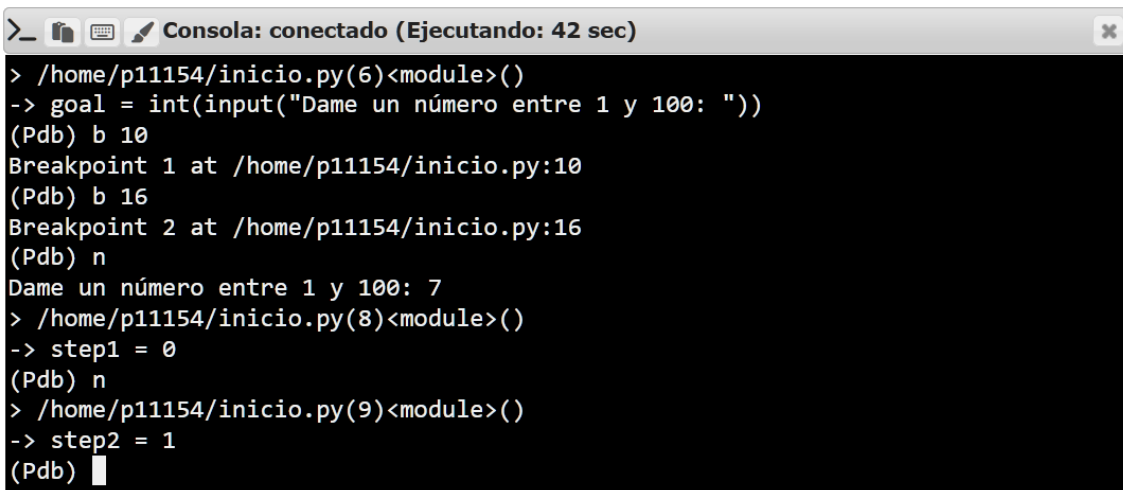


```
> /home/p10341/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb) b 10
Breakpoint 1 at /home/p10341/inicio.py:10
(Pdb) b 16
Breakpoint 2 at /home/p10341/inicio.py:16
(Pdb) b
Num Type      Disp Enb   Where
1  breakpoint keep yes   at /home/p10341/inicio.py:10
2  breakpoint keep yes   at /home/p10341/inicio.py:16
(Pdb) clear 2
Deleted breakpoint 2 at /home/p10341/inicio.py:16
(Pdb)
```

Figura 22

Ejecución paso a paso

Para ejecutar la línea en la que el programa está parado y avanzar a la siguiente se usa el comando `next`, o su versión abreviada, `n` (Figura 23).



```
> /home/p11154/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb) b 10
Breakpoint 1 at /home/p11154/inicio.py:10
(Pdb) b 16
Breakpoint 2 at /home/p11154/inicio.py:16
(Pdb) n
Dame un número entre 1 y 100: 7
> /home/p11154/inicio.py(8)<module>()
-> step1 = 0
(Pdb) n
> /home/p11154/inicio.py(9)<module>()
-> step2 = 1
(Pdb) █
```

Figura 23

Si, como en el ejemplo, en una línea se solicitan datos al usuario, la ejecución no puede avanzar hasta que se introduzcan.

El comando `continue`, o su versión abreviada, `c`, avanzan hasta el siguiente punto de parada (Figura 24).

```
> /home/p11154/inicio.py(6)<module>()
-> goal = int(input("Dame un número entre 1 y 100: "))
(Pdb) b 10
Breakpoint 1 at /home/p11154/inicio.py:10
(Pdb) b 16
Breakpoint 2 at /home/p11154/inicio.py:16
(Pdb) n
Dame un número entre 1 y 100: 7
> /home/p11154/inicio.py(8)<module>()
-> step1 = 0
(Pdb) n
> /home/p11154/inicio.py(9)<module>()
-> step2 = 1
(Pdb) c
> /home/p11154/inicio.py(10)<module>()
-> round = 0
(Pdb)
```

Figura 24

Inspección

Cuando el programa está detenido, podemos inspeccionar el estado del proceso, para averiguar si va bien o ya se ha desviado de la situación esperada, lo que significa que el fallo del programa está antes del punto actual. Una de las cosas que podemos ver es el contexto de la línea actual, usando el comando *list* o *l* (Figura 25). Se nos muestra la línea en la que el programa está parado señalada con una flecha y un conjunto de líneas anteriores y posteriores. Nótese que las líneas en las que hay un punto de parada están marcadas con una letra B.

```
(Pdb) list
 8      step1 = 0
 9      step2 = 1
10 B    round = 0
11
12      while round < goal:
13 ->        current = step1 + step2
14            step1 = step2
15            step2 = current
16 B        round = round + 1
17
18      print(step2)
```

Figura 25

Para inspeccionar el valor de una variable usamos el comando *p* (*print*) seguido del nombre de la variable (Figura 26).

```
(Pdb) p step1
0
(Pdb)
```

Figura 26

Tema 3 Introducción a la programación estructurada

Estructuras de control

La programación estructurada es un modelo de programación en el que los programas se componen de estructuras de control y subrutinas. Las subrutinas son básicamente subalgoritmos que descomponen el algoritmo principal resolviendo tareas específicas. En Python, toman la forma de funciones; por ejemplo `print()` es una función que se encarga de la tarea de mostrar información en la consola cuando un programa lo necesita.

La estructura básica de un programa es secuencial: una instrucción detrás de otra. Sin embargo, prácticamente todos los programas deben contemplar distintas situaciones o variabilidad en los datos, que requerirán realizar acciones distintas según el caso. Esto no sería posible si los programas únicamente pudieran ejecutar sus instrucciones una tras otra, de la primera a la última.

Los lenguajes de programación disponen, además de las instrucciones básicas, de **sentencias de control** que permiten reestructurar el flujo de ejecución para así adaptarse a las distintas situaciones previstas.

En particular, existen **sentencias de selección** que permiten elegir, en función de una o más condiciones, entre dos o más posibles "vías" por las que continuar la ejecución (estructura alternativa); y **sentencias de iteración o repetición** que permiten ejecutar repetidamente un mismo grupo de instrucciones, ya sea un cierto número de veces o mientras se cumpla una condición.

Condiciones

Las sentencias de control suelen decidir las acciones a ejecutar en función de **condiciones**, que son expresiones que, al ser evaluadas, devuelven un resultado booleano (*True* o *False*). Para formar condiciones se usan con frecuencia los **operadores relacionales**, que comparan los valores de sus dos operandos y devuelven un valor booleano de acuerdo con la relación entre ellos. Suponiendo, en los siguientes ejemplos, que la variable *a* tiene asociado el valor 10 y la variable *b* tiene asociado el valor 20:

Operador	Descripción	Ejemplo	Resultado
<code>==</code>	Los operandos son iguales entre sí.	<code>a == b</code>	<i>False</i>
<code>!=</code>	Los operandos no son iguales entre sí.	<code>a != b</code>	<i>True</i>
<code>></code>	El operando izquierdo es mayor que el derecho.	<code>a > b</code>	<i>False</i>
<code><</code>	El operando izquierdo es menor que el derecho.	<code>a < b</code>	<i>True</i>
<code>>=</code>	El operando izquierdo es mayor o igual que el derecho.	<code>a >= b</code>	<i>False</i>
<code><=</code>	El operando izquierdo es menor o igual que el derecho.	<code>a <= b</code>	<i>True</i>

(Nota: En los operadores formados por dos caracteres, estos deben escribirse siempre juntos y en el orden indicado).

Se pueden formar condiciones más complejas combinando el resultado de condiciones simples usando **operadores booleanos**. Suponiendo que x e y sean valores booleanos:

Operación	Resultado
x or y	Devuelve <i>False</i> si ambos son <i>False</i> ; en caso contrario, devuelve <i>True</i> .
x and y	Devuelve <i>True</i> si ambos son <i>True</i> ; en caso contrario, devuelve <i>False</i> .
not x	Devuelve <i>False</i> si x es <i>True</i> ; en caso contrario, devuelve <i>True</i> .

Ejemplos de condiciones compuestas:

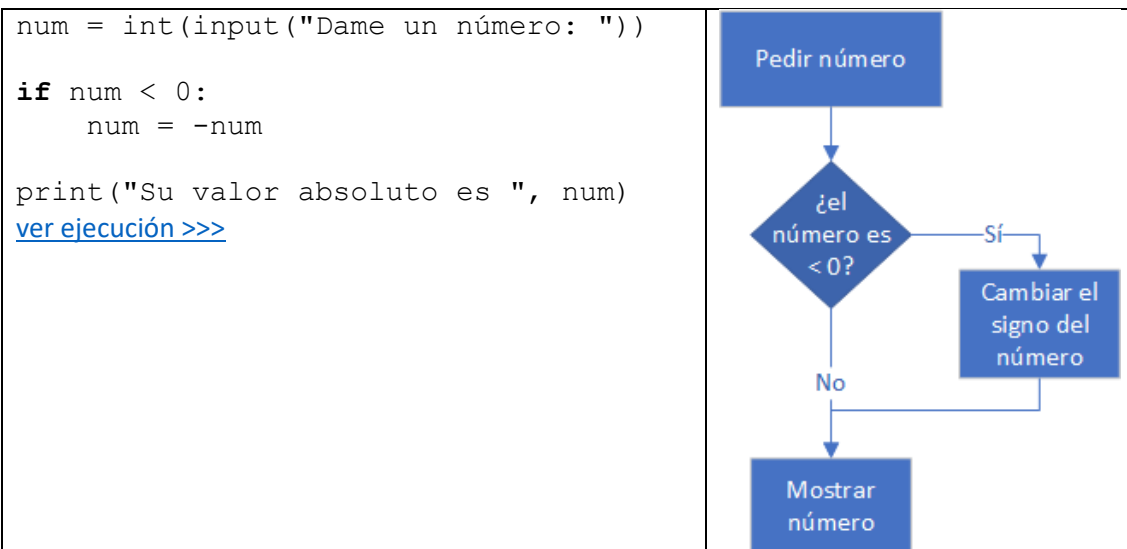
```
x > y and y < z
(x > y or x > z) and y > z
x > 2 and 0 == x % 2
```

Téngase en cuenta que, aparte paréntesis, las operaciones aritméticas tienen precedencia sobre las comparaciones, y estas sobre las operaciones booleanas. Por ejemplo, suponiendo que x vale 3 al momento de evaluar la última expresión: primero se evalúa el operando izquierdo del *and*, para ello se compara si x es mayor que 2, resultando *True*, por lo que a continuación se evalúa el operando derecho, para lo cual primero se calcula el módulo (resto de dividir x entre 2), que da 1 como resultado, valor que se compara en igualdad con el 0, lo que da *False*; por lo que el resultado del *and* y, por tanto, de la expresión completa es el valor *False*.

Ejecución alternativa

Sentencia if

La sentencia **if** permite decidir entre ejecutar, o no, una acción (o una secuencia de acciones) dependiendo de una condición.



En Python, la estructura alternativa se forma con la palabra *if* seguida de una condición y del carácter de dos puntos (:), controlando la ejecución de una o más instrucciones en las siguientes líneas. En el ejemplo, únicamente de la instrucción *num = -num*, que se ejecutará si la condición *num < 0* resulta verdadera; en caso contrario, se omitirá.

En general, la sentencia de control *if* se usa para decidir, en cada ocasión, si se ejecuta o no un bloque de instrucciones. Si la decisión fuese no ejecutar dichas instrucciones, estas se "saltan",

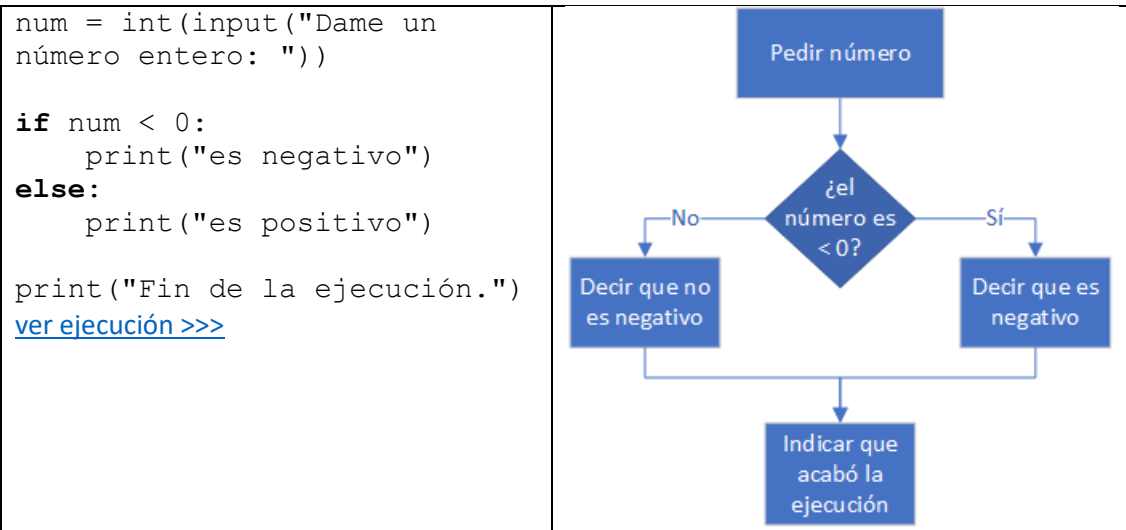
reanudándose la ejecución del programa en la instrucción siguiente a ellas; si por el contrario se ejecutaran, posteriormente se continuaría la ejecución igualmente en la instrucción siguiente a ellas.

En todas las sentencias de control de Python, las instrucciones que controla se escriben a partir de la siguiente línea y aumentando en uno el nivel de sangrado respecto a la sentencia de control (con un salto de tabulación o, preferiblemente, cuatro espacios).

Nótese, que la función `print()` del final del ejemplo, no forma parte de las instrucciones controladas por el `if`, ya que está al mismo nivel de sangrado que este, por lo que forma parte de la secuencia primaria del programa.

Cláusula `else`

Se puede añadir una cláusula `else` a una sentencia `if`. Una sentencia `if-else` se usa para decidir qué bloque de instrucciones ejecutar de entre dos posibles alternativas, reanudándose la ejecución en cualquiera de los dos casos a partir de la instrucción que sigue al segundo bloque.



En el ejemplo anterior se muestra un mensaje diferente dependiendo de si el valor numérico en la variable `num` recibido de teclado es negativo o no.

Anidamiento

Las sentencias de control se pueden "anidar", lo que significa que una (o más) de las instrucciones controladas por una sentencia de control puede ser, a su vez, otra sentencia de control que controle otra subsecuencia de instrucciones, y así sucesivamente.

```
num = int(input("Dame un número entero: "))

if num < 0:
    if num % 2 == 0:
        print(num, "es un número negativo par")
    else:
        print(num, "es un número negativo impar")
else:
    if num == 0:
        print(num, "es cero")
```

```

else:
    if num % 2 == 0:
        print(num, "es un número positivo par")
    else:
        print(num, "es un número positivo impar")

```

print("Fin de la ejecución")

[ver ejecución >>>](#)

En el ejemplo, se empieza comprobando si *num* es menor que cero; si lo es, se evalúa si es divisible por 2 (es decir, si el resto de la división es cero), mostrándose un mensaje adecuado en función de la respuesta. Si, por el contrario, el número resultase no ser menor que cero, se comprobaría a continuación si es igual a cero.

Y si al final el número resultase ser mayor que cero, se vuelve a comprobar si es par o impar. Nótese que solo se alcanzará el último de los *if* (que, por cierto, está en un segundo nivel de anidamiento y, por tanto, de sangrado) si previamente se ha comprobado que el número no es negativo ni cero, por lo que, en ese punto, se sabe, que el número no puede sino ser positivo. Y en el último de los *else* se sabe, además, que el número no es par, por lo que se puede realizar la acción correspondiente sin más comprobaciones.

Contracción *elif*

Cuando se encadenan una o más cláusulas *else* que contienen únicamente una sentencia *if*, se pueden contraer en cláusulas *elif*. Esto facilita la sintaxis para seleccionar entre múltiples alternativas excluyentes entre sí, que se colocan en el mismo nivel de sangrado que el *if* inicial:

```

num = int(input("Dame un número entero: "))

if num < 0:
    if num % 2 == 0:
        print(num, "es un número negativo par")
    else:
        print(num, "es un número negativo impar")
elif num == 0:
    print(num, "es cero")
elif num % 2 == 0:
    print(num, "es un número positivo par")
else:
    print(num, "es un número positivo impar")

```

print("Fin de la ejecución")

[ver ejecución >>>](#)

Nótese que un *if* anidado dentro de la cláusula *if* no se puede abreviar de esta manera; ni tampoco si, en una cláusula *else* a un primer *if* anidado, siguen otras instrucciones en el mismo bloque.

Obsérvese que, de la estructura *if-elif-elif-else*, solo se podrá ejecutar uno de sus bloques de instrucciones (o incluso posiblemente ninguno, si no hay un *else* al final). En particular, se ejecutará únicamente el primer bloque cuya condición se cumpla, o el del *else* si este existe y ninguna condición se cumple, reanudándose (sin comprobar las siguientes condiciones) la ejecución a continuación del último bloque.

A la hora de escribir las condiciones en los *elif*, debe tenerse en cuenta que, para que se alcance a comprobar una de ellas, todas las condiciones de los *if* y *elif* anteriores de esa misma estructura han debido resultar falsas, por lo que resultaría innecesario volver a comprobarlas. En el ejemplo, al llegar al segundo *elif*, ya se ha comprobado previamente que el número no es ni negativo ni cero, de donde se puede concluir que, alcanzado ese punto, solo puede ser positivo, y por tanto no es necesario comprobarlo de nuevo.

Ejecución repetitiva

A veces, un algoritmo requiere repetir una acción o un conjunto de acciones varias veces seguidas. Por ejemplo, para que un programa muestre cinco líneas con la palabra "Hola", una forma sencilla sería:

```
print("Hola")
print("Hola")
print("Hola")
print("Hola")
print("Hola")
ver ejecución >>>
```

La repetición o iteración es una tarea frecuente en programación, por lo que los lenguajes ofrecen estructuras para expresarla de un modo más conciso y flexible.

Sentencia while (iteración controlada por condición)

La sentencia *while* expresa la repetición de una secuencia de instrucciones mientras se cumpla una condición.

```
líneas_escritas = 0      # Inicializaciones
líneas_a_escribir = 5   #

while líneas_escritas < líneas_a_escribir: # Evaluación
    print("Hola")          # Tratamiento
    líneas_escritas = líneas_escritas + 1  # Avance
ver ejecución >>>
```

La secuencia de instrucciones asociada a la sentencia *while* sigue siempre un mismo esquema básico:

1. **Inicialización:** Se preparan las variables para la iteración.
2. **Evaluación:** La cabecera de la sentencia *while* evalúa la condición.
3. **Cuerpo del bucle:** Si la condición se cumple, se ejecutan las instrucciones del cuerpo, que se dividen en:
 - **Tratamiento:** La acción que se repite.
 - **Avance:** Una actualización que modifica la condición para que, eventualmente, deje de cumplirse. Si esto no ocurre, se entraría en un **bucle infinito**.

En el ejemplo anterior, esto se consigue incrementando la variable *líneas_escritas*, que actúa como contador de las líneas escritas. De esta forma, en algún momento llegará a igualar su valor con el de la variable *líneas_a_escribir*, que define el objetivo del problema. Entonces, la condición dejará de cumplirse y cesará la repetición, continuando la ejecución en la línea siguiente al mismo nivel que el *while*.

La flexibilidad proviene del hecho de que, al depender de una condición, el número de repeticiones necesarias no tiene por qué ser siempre el mismo ni conocerse en el momento de escribir el programa. En el ejemplo siguiente, el valor asociado a la variable *líneas_a_escribir* se solicita al usuario mediante una sentencia *input* y puede variar en cada ejecución.

```
líneas_escritas = 0
líneas_a_escribir = int(input("¿Líneas a escribir? "))

while líneas_escritas < líneas_a_escribir:
    print("Hola")
    líneas_escritas = líneas_escritas + 1
ver ejecución >>>
```

Sentencia for (iteración controlada por contador)

Los bucles controlados por la secuencia *while* se repiten mientras se cumpla una condición booleana. Aunque no siempre ocurre, es frecuente que, como sucede en los ejemplos de la sección anterior, la condición deje de cumplirse cuando se ha realizado un número de iteraciones que puede calcularse a priori con facilidad. Esto justifica la existencia de una sentencia de control específica: el bucle *for*.

```
líneas_a_escribir = int(input("¿Líneas a escribir? "))

for líneas_escritas in range(líneas_a_escribir):
    print("Hola")
ver ejecución >>>
```

En un bucle *for*, una variable de control (*líneas_escritas* en el ejemplo) va tomando secuencialmente los valores de un rango preestablecido. En el ejemplo, este rango lo proporciona la función *range()*, que se puede configurar con hasta tres parámetros:

- ***range(stop)***: Con un solo parámetro, genera una rango de valores enteros consecutivos que empieza en cero y termina en el valor anterior al del parámetro (*stop*).

```
for value in range(10):
    print(value, end=", ")
ver ejecución >>> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
```

- ***range(start, stop)***: Con dos parámetros, genera una secuencia de valores enteros consecutivos que empieza en valor del **primer** parámetro (*start*) y termina en el valor anterior al del **segundo** parámetro (*stop*).

```
for value in range(3, 8):
    print(value)
ver ejecución >>> 3, 4, 5, 6, 7,
```

- ***range(start, stop, step)***: Con tres parámetros, genera una secuencia de valores enteros que empieza en valor del **primer** parámetro (*start*) y termina en el valor anterior al del **segundo** parámetro (*stop*), con una diferencia entre cada dos valores consecutivos igual al valor del **tercer** parámetro (*step*).

```
for value in range(3, 8, 2):
    print(value)
ver ejecución >>> 3, 5, 7,
```

- El valor del tercer parámetro (*step*) puede ser negativo para una cuenta atrás si el valor del primer parámetro (*start*) es mayor que el valor del segundo parámetro (*stop*).

```
for value in range(8, 3, -2):
    print(value)
```

[ver ejecución >>>](#) 8, 6, 4,

Los valores del primer y segundo parámetro también pueden ser negativos, pero, siempre, debe cumplirse la regla: si el tercer parámetro no existe o es positivo, el segundo parámetro debe ser mayor que el primero; pero, si el tercer parámetro es negativo, el segundo parámetro debe ser menor que el primero.

Cuando el número de iteraciones se puede inferir a priori, el bucle *for* es una forma más concisa y menos propensa a errores que un *while* equivalente, ya que maneja internamente la inicialización, la condición y el avance de la variable de control.

Todo lo que se puede resolver con un *for* se puede resolver con un *while*, pero lo contrario no siempre es cierto. Por ejemplo, el bucle *while* del siguiente ejemplo no puede sustituirse fácilmente por un *for*, ya que no se conoce de antemano el número de iteraciones necesarias par que la variable *num2* llegue a cero (calcularlo costaría tanto como ejecutar el algoritmo):

```
num1 = int(input("Dame un número entero positivo: "))
num2 = int(input(
    "Dame un número entero positivo menor que el anterior: ")
)
```

```
while num2 > 0:
    mod = num1 % num2
    num1 = num2
    num2 = mod
```

```
print(num1)
```

[ver ejecución >>>](#)

Alteración del flujo de un bucle

Existen dos instrucciones que permiten alterar el flujo de ejecución de un bucle, ambas aplicables a cualquier tipo de bucle, sea *while* o *for*:

- **break**: Interrumpe la ejecución del bucle por completo, terminándolo de forma anticipada. El programa continúa en la primera instrucción después del bucle.

<pre>iteración = 0 while iteración < 10: value = int(input("Dame un número: ")) if value < 0: break print(value ** 2) iteración += 1 ver ejecución >>></pre>	<pre>for iteración in range(10): value = int(input("Dame un número: ")) if value < 0: break print(value ** 2) ver ejecución >>></pre>
--	--

- **continue**: Termina la iteración actual y pasa inmediatamente a la iteración siguiente, sin ejecutar las instrucciones restantes en el cuerpo del bucle para esa iteración.

<pre>iteración = 0 while iteración < 10: value = int(input("Dame un número: ")) if value < 0: continue print(value ** 2) iteración += 1 ver ejecución >>></pre>	<pre>for iteración in range(10): value = int(input("Dame un número: ")) if value < 0: continue print(value ** 2) ver ejecución >>></pre>
---	--

La instrucción *break* termina solo el bucle más interno en el que se encuentre. Si este está anidado dentro de otro, el bucle exterior continuaría su ejecución normalmente.

Funciones y Módulos

Funciones

Una función es un bloque de código organizado para realizar una tarea determinada y que tiene un nombre que permite llamarla (invocar su ejecución). Una función se ejecuta solo cuando se la llama, y puede ser invocada cuantas veces sea conveniente y desde múltiples puntos de un programa.

```
def greetings_func():
    """Muestra un saludo."""
    print("Hola")
```

El ejemplo anterior muestra la definición de una función en Python, que consta de una **cabecera** y de un **cuerpo**.

- La **cabecera** empieza con la palabra clave *def*, seguida del nombre de la función (*greetings_func* en el ejemplo) y paréntesis obligatorios (). La cabecera de la función finaliza con el carácter dos puntos ':
- El **cuerpo** de la función comienza en la línea siguiente a la cabecera y está indentado (desplazado un nivel de sangrado con un tabulador, o, preferiblemente, cuatro espacios respecto a la misma). Estará formado por cualquier combinación de instrucciones y sentencias de control para realizar la tarea que se desee.

Como primera línea del cuerpo de una función, justo debajo de la cabecera, es habitual poner un breve comentario llamado **docstring** (string de documentación), en forma de una string encerrada entre comillas triples. Este comentario se usa para proporcionar información básica sobre la función, que puede ser usada por herramientas de desarrollo para proporcionar ayuda automática y autocompletado de código al usar la función. Se puede acceder a la docstring de una función usando el atributo `__doc__` tal como se muestra en el siguiente ejemplo:

```
print(greetings_func.__doc__)
ver ejecución >>>
```

El resultado del código anterior es:

```
Muestra un saludo.
```

Los literales de string con comillas triples pueden ocupar varias líneas:

```
"""Esto es un ejemplo de docstring multilínea.
   Las comillas de apertura y cierre
   deben estar alineadas.
   """
```

Para usar una función hay que **invocarla** (llamarla). La llamada a una función es como una instrucción más, que se forma poniendo el nombre de la función seguido de paréntesis:

```
def greetings_func():
    """Muestra un saludo."""
    print("Hola")

print("Antes de llamar a greetings_func")
greetings_func() # Llamada a greetings_func
print("Después de llamar a greetings_func")
ver ejecución >>>
```

Al terminar la ejecución de una función, el flujo de ejecución del programa retorna justo al punto después de la llamada, reanudándose la ejecución a partir de ahí. En cierto sentido, definir una función es como si se hubiera añadido una nueva instrucción u operación al lenguaje, y la llamada sería su ejecución. El resultado del código anterior es, pues:

```
Antes de llamar a greetings_func
Hola
Después de llamar a greetings_func
```

Funciones con parámetros

Las funciones pueden tener **parámetros** que, en Python, funcionan como variables internas de la función. Los parámetros de la función se inicializan con los datos pasados en la llamada, que son los datos de entrada necesarios para realizar la tarea. Por ejemplo:

```
def greetings_func(name):
    """Muestra un saludo.
       Parameter name: str - nombre de la persona a saludar
    """
    print("Hola", name)
```

La función *greetings_func()* tiene ahora un parámetro, llamado *name*, que se usa para incluir en el saludo el nombre de la persona a la que se saluda. Una función puede tener varios parámetros, cuyos nombres se escriben dentro de los paréntesis que siguen al nombre de la función, formando la denominada **lista de parámetros formales**. Esta lista indica qué datos espera recibir la función cuando se la llame para realizar su tarea.

```
def consumo_medio(kilometros, litros):
    # ... cuerpo de la función ...
```

Al llamar a una función que tiene parámetros, se deben pasar valores para esos parámetros; estos valores se denominan **parámetros reales** (en inglés, *actual parameters*, a veces llamados también *arguments* o "argumentos"). El nombre de parámetros reales proviene del hecho de

que son los valores reales que se utilizan en la función en cada ocasión, en contraposición a los formales, que son indicadores de los parámetros que se requieren, pendientes de concretar en cada llamada:

```
name1, name2 = "Pedro", "Juan"
greetings_func(name1)
greetings_func(name2)
greetings_func("María" + " " + "Elena")
ver ejecución >>>
```

Como muestra el ejemplo, los parámetros reales pueden ser variables, valores literales o, en general, cualquier expresión que dé como resultado un valor del tipo adecuado.

En el siguiente ejemplo, la función *input()* devuelve una string (tecleada por el usuario) la cual, a su vez, es recibida por *greetings_func()* en su parámetro *name*:

```
greetings_func(input("Hola, ¿cómo te llamas? "))
ver ejecución >>>
```

Si la función tiene varios parámetros, hay que pasar valores para todos ellos. En Python, esto se puede hacer de dos formas:

- **Por posición:** El parámetro real se asocia con el parámetro formal que ocupa su misma posición en la lista de parámetros. Es el método habitual en la mayoría de los lenguajes de programación.
- **Por nombre:** El parámetro real se asocia explícitamente con el nombre de un parámetro formal.

```
def consumo_medio(kilometros, litros):
    ...

consumo_medio(100, 6) # Paso por posición -> (kilometros = 100,
litros = 6)
consumo_medio(litros=17.5, kilometros=197) # Paso por nombre
```

(Obsérvese que, tanto para los parámetros formales como para los actuales, se suele dejar un espacio después de la coma).

Los parámetros formales pueden tener **valores por omisión**, que se usan si en la llamada no se especifica un valor para ese parámetro. Los parámetros que no tienen valores por omisión son obligatorios en la llamada.

```
def consumo_medio(distancia, litros, unidades="kilometros"):
    ...
consumo_medio(103, 6.5) # distancia = 103,
                        # litros= 6.5,
                        # unidades = "kilómetros"
consumo_medio(87, 11, "millas") # distancia = 87,
                                # litros= 11,
                                # unidades = "millas"
```

Los parámetros reales que estén después de uno o más que "se pasan" por omisión tienen que pasarse obligatoriamente por nombre, ya que se pierde la correspondencia posicional entre la lista de parámetros formales y la lista de parámetros reales.

Es importante comprender la relación entre parámetros reales y formales en Python. Al llamar a una función, los valores de los parámetros reales se asignan a los parámetros formales, que actúan como variables locales (internas) de la función y desaparecen cuando esta termina. El hecho de que en una llamada se usen variables para algunos parámetros reales es irrelevante (lo que importa es el valor que contienen en el momento de la llamada). Estas variables externas no tienen relación con los parámetros formales, incluso si tuvieran el mismo nombre. Por ejemplo, en una llamada $f(x)$, x será una variable en el contexto desde el que se genera la llamada, y aunque el parámetro formal de f también se llamara x "por coincidencia", no existiría confusión ni conflicto, ya que este parámetro formal x es una variable interna a la función, distinta de la otra variable también llamada x , existente en el punto de la llamada. De este modo, incluso si durante la ejecución de la función se le asignara un nuevo valor al parámetro formal x , la otra variable x no se vería afectada.

Funciones que devuelven un resultado

Además de poder admitir datos como parámetros, las funciones pueden devolver un resultado al finalizar su tarea. Esto es coherente (además del nombre) con la idea de que las funciones resuelven problemas del tipo "hallar un dato desconocido a partir de otros datos conocidos", aplicando un algoritmo que describe la solución de un problema en función de los datos necesarios para representar un caso concreto y de los pasos requeridos para obtener el resultado deseado.

Para devolver un resultado, se utiliza la instrucción *return* seguida del valor a devolver (el cual se puede expresar en forma de un literal del tipo apropiado, una variable que lo contenga o, en general, cualquier expresión cuya evaluación proporcione el valor a devolver):

```
def suma(a, b):  
    """Suma los valores de sus parámetros."""  
    result = a + b  
    return result
```

El resultado devuelto por una función se puede usar directamente en una expresión (es como si se hubiera escrito el valor en lugar de la llamada), con la flexibilidad de que dicho valor puede variar según los valores de los parámetros en ese momento:

```
print(suma(x, y))  
...  
if 10 * suma(x, y) > 50:  
    #...
```

También se puede asignar a una variable para usarlo más tarde:

```
z = suma(x, y)  
ver ejecución >>>
```

(Recuérdese que, aunque se hayan empleado los mismos nombres para las variables usadas como parámetros reales y para los parámetros formales de *suma*, no existe relación alguna entre ellas, pues son variables distintas en ámbitos diferentes.

Dado que al ejecutarse la instrucción *return* la función finaliza su tarea, cualquier instrucción que se encuentre después del *return*, en su mismo nivel de anidamiento, no se ejecutará nunca. Por lo tanto, no deben escribirse instrucciones después de un *return*:

```
def suma(a, b):  
    """Suma los valores de sus parámetros."""  
    result = a + b  
    return result  
result = 0  
print(result)  
  
def PositivoNegativo(num):  
    """Devuelve si el número es positivo, negativo o cero."""  
    if num < 0:  
        return "negativo"  
num = 0  
print(num)  
    elif num > 0:  
        return "positivo"  
num = 0  
print(num)  
    return "cero"  
num = 0  
print(num)
```

[ver ejecución >>>](#)

En Python, todas las funciones devuelven un valor. Aquellas que no lo hacen explícitamente mediante *return* devuelven el valor *None*. *None* es el único valor del tipo de datos *NoneType* y se interpreta como "ningún valor". Por ejemplo:

```
def greetings_func():  
    """Muestra un saludo."""  
    print("Hola")  
  
result = greetings_func();  
print("Resultado devuelto por la función:", result);
```

[ver ejecución >>>](#)

El código anterior produce el siguiente resultado:

```
Hola  
Resultado devuelto por la función: None
```

La palabra "Hola" la escribe la función, la cual devuelve *None*. El valor *None* devuelto por la función es asignado a la variable *result* y mostrado por la instrucción *print()* que se ejecuta en la siguiente línea.

La instrucción *return* puede utilizarse como cualquier otra instrucción dentro de una función, esto es, puede haber varias instrucciones *return* en distintos puntos de la misma. Eso sí, una vez que se alcanza una instrucción *return*, la ejecución de la función finaliza de inmediato, por lo que carece de sentido escribir otra instrucción del mismo nivel de sangrado justo a continuación.

Funciones polimórficas

Muchos lenguajes de programación exigen que se declaren los tipos de los datos que pueden asociarse a una variable, pasarse a un parámetro formal o ser devueltos por una función. Python

tiene **tipado dinámico**, lo que significa que no requiere declaraciones de esta clase: el tipo de dato asociado a una variable lo determina el del valor que se le asigna en el momento de crearla, y un parámetro formal puede asociarse a un tipo de dato diferente en cada llamada. Esto permite que, en el caso de las funciones, estas sean **polimórficas** por definición (una función polimórfica es aquella que se puede usar con distintos tipos de parámetros y/o un número variable de ellos).

Una función en Python como la siguiente puede aplicarse sin problema a cualquier pareja de valores que admitan el uso del operador +, entre ellos:

```
def sum(a, b):  
    """Suma los valores de sus parámetros"""  
    result = a + b  
    return result  
  
print(sum(1, 3))           # Números enteros  
print(sum(3.5, 4.8))      # Números reales  
print(sum(3.5, 4))        # Un número entero con un real  
print(sum("Hola ", "mundo")) # Strings, que serán concatenadas con el +  
print(sum(3+2j, 5-1j))    # Números complejos  
ver ejecución >>>
```

Resultado de la ejecución anterior:

```
4  
8.3  
7.5  
Hola mundo  
(8+1j)
```

Anotación de los tipos de los parámetros de una función

Aunque el polimorfismo proporcionado a Python por el tipado dinámico puede ser una potente herramienta —pues permite que una función se use en situaciones muy diversas sin necesidad de reescribir múltiples versiones—, cuando no se pretende escribir una función polimórfica puede ser útil explicitar qué tipo de dato se espera que reciba un parámetro formal o qué tipo de resultado debería devolver una función. Esto se conoce como **anotación de tipos** o *type hinting*.

```
def sum(a: int, b: int) -> int:  
    """Suma los valores de sus parámetros."""  
    result = a + b  
    return result
```

Como se muestra en el ejemplo anterior, para los tipos más básicos (*int*, *float*, *bool*, *str*) basta con poner dos puntos detrás del nombre del parámetro y añadir el tipo esperado. Para otros tipos de datos, que se irán introduciendo en lecciones posteriores, puede ser necesario incluir alguna instrucción adicional. El tipo del resultado de la función se especifica después de una "flecha" (->) añadida al final de la cabecera, antes de los dos puntos de la cabecera.

El primer efecto del tipado explícito es de carácter **informativo**. Normalmente, en la *docstring* de una función se incluye información sobre sus parámetros, especificando el tipo de datos que se espera recibir, como en el siguiente ejemplo:

```

"""Suma los valores de sus parámetros.
Parámetros:
a: (int) - Primer sumando
b: (int) - Segundo sumando
"""

```

Si se especifican los tipos en la lista de parámetros formales y, además, se usan nombres de parámetros adecuados y anotaciones de tipo, la docstring puede reducirse sustancialmente, ya que mucha información relevante está incluida en la propia cabecera de la función que se vuelve autoexplicativa.

```

def sum(primer_sumando: int, segundo_sumando: int) -> int:
    """Suma los valores de sus parámetros"""
    result = primer_sumando + segundo_sumando
    return result

```

Además de esta función informativa, también se pueden usar herramientas de análisis estático que, basándose en la información proporcionada por el tipado explícito, son capaces de detectar por anticipado posibles errores que, de otro modo, solo se manifestarían durante la ejecución del programa.

Módulos y paquetes

Dónde escribir una función

En Python, una función se puede escribir en el mismo archivo que el código que la utiliza:

archivo my_program.py

```

def my_func(a: int, b: float) -> bool:
    return a > b

num1 = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(my_func(num1, num2))

```

Sin embargo, un código también puede llamar a una función escrita en otro archivo. Para ello, debe incluir previamente una cláusula de **importación**:

archivo my_program.py

```

import functions

num1 = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(functions.my_func(num1, num2))

```

archivo `functions.py`

```
def my_func(a: int, b: float) -> bool:
    return a > b
```

Esta segunda aproximación independiza a la función del código que la usa, lo que permite que sea reutilizada desde diferentes piezas de código escritas en distintos archivos.

Módulos

La programación modular es una técnica de diseño de software que enfatiza la separación de la funcionalidad de un programa en módulos independientes e intercambiables, de manera que cada uno contiene todo lo necesario para ejecutar solo un aspecto de la funcionalidad deseada. Por ejemplo, en un juego, se podría tener, entre otros, un módulo para controlar la lógica del juego, otro para interactuar con el usuario y un tercero para mostrar el escenario de juego en la pantalla.

La programación modular permite gestionar la complejidad, dividiendo una funcionalidad compleja en otras más simples, y favorece la **reutilización del código**, ya que los módulos deben ser independientes e intercambiables. Al ser los módulos independientes, uno desarrollado inicialmente para un programa podría usarse en otro, y un programa que usa un módulo podría cambiarlo por otro mejor sin afectar al resto del sistema.

En Python, cada fichero con extensión `.py` alberga un módulo, que se llama como el archivo que lo contiene (sin la extensión). Para usar las funcionalidades ofrecidas por un módulo desde un módulo distinto, solo hay que poner su nombre en una cláusula de importación:

módulo `my_program.py`

```
import functions

num1 = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(functions.my_func(num1, num2))
print(functions.pi)
```

módulo `functions.py`

```
pi = 3.14

def my_func(a: int, b: float) -> bool:
    return a > b
```

El módulo *functions* del ejemplo anterior ofrece dos elementos que se pueden utilizar: la variable *pi* y la función *my_func*. Ambos, se pueden utilizar, una vez importado el módulo, escribiendo su nombre precedido del nombre del módulo separado por un punto.

```
print(functions.my_func(num1, num2))
print(functions.pi)
```

Un módulo proporciona un **espacio de nombres** (*namespace*) en el que se pueden declarar elementos sin entrar en conflicto con los de otros módulos, aunque tengan el mismo nombre. Se diferenciarían al prefijarlos con el nombre del módulo correspondiente (por ejemplo, se podría usar en el mismo programa dos módulos que declarasen una función llamada *my_func* sin que se confundan —*module1.my_func()* y *module2.my_func()*—).

En el caso de que un módulo ofrezca varios elementos, pero sólo se necesiten usar algunos, es posible importar individualmente aquellos que se deseen con la sintaxis "*from ... import ...*", como muestra el siguiente ejemplo:

módulo my_program.py

```
from functions import my_func

num1 = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(my_func(num1, num2))
```

Los elementos importados directamente no pueden ser prefijados con el nombre del módulo a la hora de usarlos. Para evitar conflictos en caso de querer usar elementos con el mismo nombre proveniente de distintos módulos, se puede optar por importar los módulos completos de modo que los elementos puedan ser prefijados con el nombre del módulo (*module1.suma* y *module2.suma*), o bien usar **alias** con la palabra reservada **as**:

```
from module1 import suma as suma1
from module2 import suma as suma2
```

Asimismo, se pueden usar alias al importar un módulo completo, generalmente para disponer de un nombre más corto y manejable:

```
import functions as funcs
```

El módulo `__main__`

Un módulo puede incluir secuencias de instrucciones que no forman parte de ninguna función:

módulo functions.py

```
def my_func(a: int, b: float) -> bool:
    return a > b

print("Esto no forma parte de una función")
```

Estas instrucciones "libres" se ejecutan en el momento en que se carga el módulo (la primera vez que es importado), a diferencia de las funciones, que solo se ejecutan cuando se las llama:

módulo my_program.py

```
from functions import my_func

radi = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(my_func(num1, num2))
```

Resultado:

```
Esto no forma parte de una función
Entra un número entero: 1
Entra un número entero: 2
False
```

Frecuentemente, no se desea que este código "libre" se ejecute cuando el módulo es importado, sino solo cuando es el módulo "principal" (inicial) del programa (es decir, el archivo que se ejecuta directamente). Para lograr esto, se comprueba si el nombre del módulo es "`__main__`":

módulo functions.py

```
def my_func(a: int, b: float) -> bool:
    return a > b

if __name__ == "__main__":
    print("Esto no forma parte de una función")
```

El nombre `__main__` (con dos guiones bajos a cada lado) es asignado automáticamente por Python al módulo que inicia la ejecución de un programa. De esta manera, el código controlado por esa sentencia *if* solo se ejecuta si el módulo es el punto de partida, pero no cuando es importado por otro que se empezó a ejecutar primero.

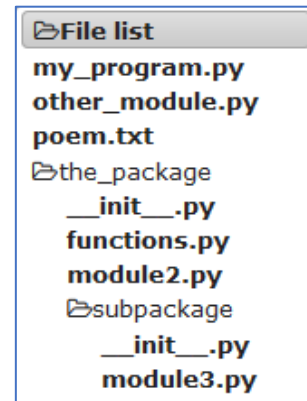
Paquetes

En programación modular, el concepto "**paquete**" (en inglés, *package*) se usa para referirse a una agrupación lógica de módulos. Distintas funcionalidades se implementan en diferentes módulos, que pueden incluir funciones, variables, y otros elementos relacionados, y un conjunto de módulos con funcionalidades relacionadas se agrupan en un paquete.

En Python, al igual que un módulo se implementa en un archivo, un paquete, —que es un conjunto de módulos y, por tanto, implica un conjunto de archivos— se corresponde, físicamente, con una **carpeta** o directorio en el sistema de ficheros.

Para que un directorio sea un paquete por Python, debe incluir un "archivo de configuración" llamado `__init__.py`. Este archivo puede estar vacío, y sirve para inicializar el paquete y controlar el acceso a sus módulos desde módulos externos.

La figura muestra una estructura formada por dos módulos (*my_program* y *other_module*) que no forman parte de ningún paquete, más un paquete llamado *the_package*, que contiene otros dos módulos (*functions* y *module2*) y un subpaquete llamado *subpackage*, el cual contiene, a su vez, un único módulo (*module3*).



Para importar un módulo que pertenece a un paquete, se debe especificar la ruta completa desde la raíz del proyecto, usando puntos como separadores, como se muestra en el siguiente ejemplo:

módulo `my_program.py`

```
from the_package.functions import my_func

num1 = int(input("Entra un número entero: "))
num2 = int(input("Entra un número entero: "))
print(my_func(num1, num2))
```

Algoritmos y Estructuras de Datos

Tema 4 Secuencias

Tipos de datos secuencia. Contenedores secuenciales

Secuencias y contenedores secuenciales

Una **secuencia** es una colección de elementos **organizados de manera lineal**; es decir, alineados de forma que hay un elemento que es el primero y cada uno, excepto el último, tiene un único sucesor. *Por ejemplo, los meses del año forman una secuencia en la que enero es el primero, le sigue febrero, a este le sigue marzo, y así sucesivamente hasta llegar a diciembre.*

Al estar alineados, a los elementos de una secuencia se le pueden asignar números de orden de acuerdo con su posición en la misma. Así, una fecha como "16 de julio de 1969", se suele expresar como "16/07/1969", usando la posición ordinal de julio en la secuencia de meses del año.

Los lenguajes de programación, como Python, ofrecen tipos de datos para representar o almacenar secuencias. Estos tipos de datos son **tipos de datos compuestos**, ya que su valor se compone de un conjunto de valores individuales (los elementos de la secuencia), a diferencia de los **tipos de datos simples**, como *int* o *float*, que representan valores unitarios.

Los principales tipos de datos secuencia en Python son: string de caracteres (*str*), también conocido como ristra o cadena de caracteres, lista (*list*) y tupla (*tuple*). Python considera estos tipos como **contenedores secuenciales**. En general, un **contenedor** se define como una estructura diseñada para *almacenar, organizar y gestionar un conjunto de datos* en memoria.

Una **string** (cadena de caracteres) puede considerarse un tipo especial de contenedor en muchos lenguajes de programación. Aunque no se suele pensar en ella de la misma manera que en las listas o las tuplas, cumple con las características básicas de un contenedor, aunque está especializada en almacenar exclusivamente caracteres. En cambio, otros contenedores tienen un carácter más general, ya que pueden almacenar diversos tipos de datos. Por otra parte, mientras que las operaciones de un contenedor general se centran en la gestión de los elementos almacenados, las operaciones más relevantes de las strings están orientadas al **procesamiento de textos**, más que al almacenamiento de caracteres individuales.

Valores de tipo str, list y tuple

Los valores de tipo *str* (**strings**) se crean escribiendo una secuencia de caracteres encerrada entre comillas. En Python, las comillas pueden ser indistintamente dobles o simples:

```
greetings = "Hello World"
name = 'John Smith'
empty_string = ""
```

[ver ejecución >>>](#)

Una **tupla** (*tuple*) se forma enumerando, entre paréntesis, la secuencia de elementos que la componen, separados por comas:

```
name = ("John", "Smith")
age = 21
code = "X928476"
person = (name, age, code) # (("John", "Smith"), 21, "X928476")
empty_tuple = ()
one_element = (10,) # La coma indica una tupla de un solo elemento
ver ejecución >>>
```

Nótese que, para formar una tupla con un solo elemento, es necesario colocar una coma después de este (como en la asignación a *one_element* del ejemplo anterior). De lo contrario, se considerará una expresión entre paréntesis en lugar de una tupla.

Una **lista** (*list*) se forma enumerando, entre corchetes, la secuencia de elementos que la forman, separados por comas:

```
primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
names = ["John", "Anne", "Philip", "Albert", "Mary"]
empty_list = []
ver ejecución >>>
```

Operaciones básicas de los contenedores secuenciales.

Cardinalidad

Todos los contenedores, secuenciales o no, almacenan una colección de elementos. Por ello, es útil disponer de una operación que permita conocer cuántos elementos contiene un contenedor en un momento dado. En Python, esta operación se realiza con la función *len()*:

```
len(container) # Devuelve el número de elementos
```

Ejemplos:

```
num_of_chars = len("Hola mundo") # num_of_chars toma el valor 10

primes = [2, 3, 5, 7, 11, 13, 17, 19]
print(len(primes)) # Muestra 8 en la consola

date_len = len((16, 7, 1969)) # date_len toma el valor 3
ver ejecución >>>
```

Acceso a los elementos individuales de un contenedor secuencial

El acceso a los elementos individuales de un contenedor secuencial se hace especificando el **índice** de su posición entre **corchetes**. En Python, los índices pueden seguir una **secuencia incremental** (empezando en cero para la primera posición) o **decremental** (empezando en -1 para la última posición, lo que equivale a *len(seq) - 1*, siendo *seq* la secuencia):

<i>negative index</i>	-5	-4	-3	-2	-1
<i>positive index</i>	0	1	2	3	4
lst=	[10,	20,	30,	40,	50]

Ejemplos:

```
greetings = "Hello World"
primes = [2, 3, 5, 7, 11, 13, 17, 19]
```

```
moon_date = (16, 7, 1969)
```

Operación	Resultado
<code>char = greetings[6]</code>	char vale 'W'
<code>fifth_prime = primes[4]</code>	fifth_prime vale 11
<code>year = moon_date[2]</code>	year vale 1969

[ver ejecución >>>](#)

Slices

Un *slice* es una **copia** de una subsecuencia de elementos de un contenedor lineal. Se indica con los índices de inicio, fin y, opcionalmente, el intervalo entre los elementos a seleccionar, entre corchetes y separados por dos puntos:

```
container[start:end:step]
```

Nótese en los siguientes ejemplos, que cualquiera de los indicadores antedichos se puede omitir, usándose, en ese caso, su correspondiente valor por omisión: 0 para *start*, -1 para *end* y 1 para *step*.

Ejemplos:

<pre> -5 -4 -3 -2 -1 0 1 2 3 4 lst = [10, 20, 30, 40, 50]</pre>	
Operación	var
<code>var = lst[1:3]</code>	[20, 30]
<code>var = lst[-3:-1]</code>	[30, 40]
<code>var = lst[:3]</code>	[10, 20, 30]
<code>var = lst[3:]</code>	[40, 50]
<code>var = lst[:-1]</code>	[10, 20, 30, 40]
<code>var = lst[1:-1]</code>	[20, 30, 40]
<code>var = lst[::2]</code>	[10, 30, 50]
<code>var = lst[::-1]</code>	[50, 40, 30, 20, 10]
<code>var = lst[::-2]</code>	[50, 30, 10]
<code>var = lst[:]</code>	[10, 20, 30, 40, 50]

[ver ejecución >>>](#)

<pre>012345678901234567890123 text = "En un lugar de la Mancha"</pre>	
Operación	var
<code>var = text[1:3]</code>	"n "
<code>var = text[-3:-1]</code>	"ch"
<code>var = text[:3]</code>	"En "
<code>var = text[3:]</code>	"un lugar de la Mancha"
<code>var = text[:-1]</code>	"En un lugar de la Manch"
<code>var = text[1:-1]</code>	"n un lugar de la Manch"
<code>var = text[::2]</code>	"E nlgrd aMnh"
<code>var = text[::-1]</code>	"ahcnaM al ed ragul nu nE"
<code>var = text[::-2]</code>	"aca le au un"
<code>var = text[:]</code>	"En un lugar de la Mancha"

[ver ejecución >>>](#)

Concatenación de secuencias

La operación de **concatenación** consiste en formar una nueva secuencia juntando dos secuencias del mismo tipo. En Python, se expresa mediante el operador +.

```
primera = "Hola"
segunda = "Mundo"
total = primera + " " + segunda # "Hola Mundo"
```

```
primera = (10, 20)
segunda = (30, 40)
total = primera + segunda # (10, 20, 30, 40)
```

```
primera = [10, 20]
segunda = [30, 40]
total = primera + segunda # [10, 20, 30, 40]
```

[ver ejecución >>>](#)

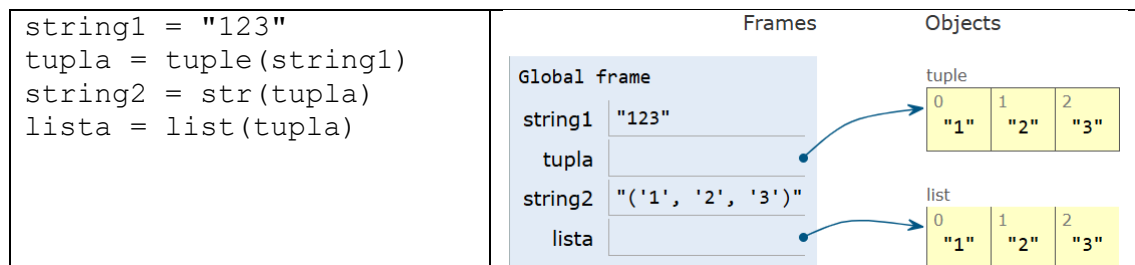
El operador de concatenación, +, sería equivalente, para las secuencias, al operador de suma aritmética para los números. De la misma manera, existe un operador de multiplicación para las secuencias, *, que indica la repetición de una secuencia un número determinado de veces:

```
string = 'a' * 5 # "aaaaa"
tupla = (1,) * 5 # (1, 1, 1, 1, 1)
lista1 = [0] * 5 # [0, 0, 0, 0, 0]
lista2 = [1, 2, 3] * 2 # [1, 2, 3, 1, 2, 3]
```

[ver ejecución >>>](#)

Creación de una secuencia a partir de otra

Es posible crear una secuencia a partir de otra usando el nombre del tipo de secuencia como **función constructora** a la que se le pasa como parámetro la secuencia origen. Por ejemplo:



[ver ejecución >>>](#)

Nótese que, como una string es una secuencia de caracteres, al **crear la tupla a partir de la string pasada como parámetro** cada carácter se convierte en un elemento de esta. Por otra parte, al **crear una string a partir de una tupla**, lo que se guarda es la representación visual de la tupla como string, que incluye los paréntesis delimitadores y las comas y espacios de separación visual como caracteres, y cualquier carácter ligado a la representación de los elementos individuales, como, en el ejemplo, las comillas simples.

Empaquetado y desempaquetado

Como ya se ha visto, es posible crear una tupla o una lista poniendo entre los delimitadores correspondientes una secuencia de valores, literales o variables, como en el siguiente ejemplo

```
name = ("John", "Smith")
age = 21
code = "X928476"
person = (name, age, code) # (("John", "Smith"), 21, "X928476")
```

A esta acción la podemos llamar **empaquetado** de datos. La acción inversa, el **desempaquetado**, se puede hacer asignando a una secuencia de variables un dato de tipo secuencia:

```
firstname, lastname = name
nombre, edad, código = person
```

[ver ejecución >>>](#)

Es importante tener en cuenta que el número de variables a la izquierda de la asignación tiene que ser igual al número de componentes del objeto de la derecha; el desempaquetado se suele aplicar con secuencias que tienen pocos elementos.

Esquemas de iteración secuencial

Esquema general de recorrido

Cuando se tiene una secuencia de valores, por ejemplo, guardada en una lista:

```
my_list = [10, 12, 23, 35, 45, 67, 89]
```

Es frecuente que se tenga que repetir un mismo tratamiento para cada elemento de la secuencia, según el siguiente esquema, que se llama **esquema de recorrido**:

Esquema	Ejemplo
<pre>para cada <i>item</i> en <i>secuencia</i>: hacer algo con <i>item</i></pre>	<pre>def show(seq): for item in seq: print(item, end=", ") show(my_list) # 10, 12, 23, 35, 45, 67, 89, </pre>

[ver ejecución >>>](#)

La sentencia *for* crea un bucle que itera (iterar=repertir) sobre la secuencia, repitiendo el mismo tratamiento para cada elemento. Cada vuelta (repetición) del bucle se conoce como una **iteración**.



Recorrido de un rango

Un esquema de recorrido también se puede aplicar a un rango de valores consecutivos no almacenados previamente en un contenedor. En Python, esto se puede hacer usando la función *range()*, como en el siguiente ejemplo.

```
def show_up_to(number):  
    """Muestra los números en el rango [0..number-1]"""  
    for current in range(0, number):  
        print(current, end=", ")
```

```
show_up_to(10) # ||0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ||
```

[ver ejecución >>>](#)

Cuando el rango empieza en cero, como en el ejemplo anterior, el cero se puede omitir:

```
for current in range(number):
```

También se puede especificar un comienzo distinto de cero:

```
def show_up_to(start, end):  
    """Muestra los números en el rango [start..end-1]"""  
    for current in range(start, end):  
        print(current, end=", ")
```

```
show_up_to(5, 10) # ||5, 6, 7, 8, 9, ||
```

[ver ejecución >>>](#)

Recorrido en rango con salto

Se puede especificar que el rango sea discontinuo, especificando un intervalo entre valores. Igual que en los slices, la función *range()* admite hasta tres parámetros (*range(start, stop, step)*), siendo *start* el valor inicial de la iteración, *stop* el valor siguiente al último iterado y *step* el intervalo entre valores a iterar:

```
def show_evens_up_to(number):  
    """Muestra los números en el rango [0..number-1]"""  
    for current in range(0, number, 2):  
        print(current, end=", ")
```

```
show_evens_up_to(10) # ||0, 2, 4, 6, 8, ||
```

[ver ejecución >>>](#)

También se puede recorrer un rango descendente, especificando un final de rango menor que el inicio y un intervalo negativo:

```
def show_from(number):  
    """Muestra los números en el rango [number..1]"""  
    for current in range(number, 0, -1):  
        print(current, end=", ")
```

```
show_from(10) # ||10, 9, 8, 7, 6, 5, 4, 3, 2, 1, ||
```

[ver ejecución >>>](#)

Si los valores de inicio y fin están **cruzados** y no se pone un intervalo negativo, se considera un **rango vacío**, por lo que el bucle no ejecuta ninguna iteración.

Recorrido de una secuencia almacenada usando índices

La función `range()` también se puede usar para implementar una variación del recorrido de una secuencia almacenada de tal manera que el bucle está controlado por la secuencia de valores de índices y no directamente por la secuencia de elementos:

```
my_list = [10, 12, 23, 35, 45, 67, 89]

def show(seq):
    for idx in range(len(seq)):
        print(seq[idx], end=" ")

show(my_list) # ||10, 12, 23, 35, 45, 67, 89, ||
ver ejecución >>>
```

Esto es especialmente útil cuando se necesita modificar los valores de la lista:

```
my_list = [10, 12, 23, 35, 45, 67, 89]

def grew_up(seq):
    for idx in range(len(seq)):
        seq[idx] += 1

grew_up(my_list)
print(my_list) # ||[11, 13, 24, 36, 46, 68, 90]||
ver ejecución >>>
```

Si el recorrido es por elemento y no por índice, la lista **no se modificará**:

```
my_list = [10, 12, 23, 35, 45, 67, 89]

def grew_up(a_list):
    for item in a_list:
        item += 1

grew_up(my_list)
print(my_list) # ||[10, 12, 23, 35, 45, 67, 89]||
ver ejecución >>>
```

Recorrido basado en una condición

La sentencia `for` está pensada para recorrer secuencias preestablecidas, almacenadas o no, donde se sabe de antemano cuántos elementos hay. Sin embargo, en algunos casos solo se sabrá que la secuencia a recorrer ha terminado cuando se encuentre su final (por ejemplo, alguna marca o valor especial). El siguiente ejemplo lee números del teclado hasta que el usuario teclea un valor negativo:

```
def read_numbers():
    numbers = []
    current = int(input("Dame un número (negativo termina): "))

    while current >= 0:
        numbers.append(current)
        current = int(input("Dame otro (negativo termina): "))

    return numbers
```

```
list_of_numbers = read_numbers()
ver ejecución >>>
```

El esquema general del recorrido basado en una condición es:

preparar el primer elemento

mientras *se cumpla una condición:*
 hacer algo
 avanzar *al siguiente elemento*

Obviamente, cualquier cosa que se pueda hacer con una sentencia *for* también se puede hacer con una sentencia *while*, ya que siempre se puede encontrar una condición booleana equivalente a que se haya completado la secuencia.

Con sentencia <i>for</i>	Con sentencia <i>while</i>
<pre>def show(seq): for item in seq: print(item, end=", ") show(my_list)</pre> ver ejecución >>>	<pre>def show(seq): idx = 0 while idx < len(seq): item = seq[idx] print(item, end=", ") idx += 1 show(my_list)</pre> ver ejecución >>>

Recorrido con tratamiento selectivo (esquema de selección)

En algunos casos, se requiere aplicar un tratamiento solo a ciertos elementos de una secuencia que cumplan una determinada condición. Igualmente se tendrá que recorrer la secuencia completa, pero, al acceder a cada elemento, se deberá evaluar la condición para saber si se tiene o no que aplicar el tratamiento en cuestión a ese elemento concreto. La función del siguiente ejemplo muestra en pantalla solo los elementos de la secuencia que sean **múltiplos de un valor pasado** a la función como segundo parámetro:

```
def show(seq, divisor):
    for item in seq:
        if item % divisor == 0:
            print(item, end=", ")

my_list = [10, 12, 23, 35, 45, 67, 89]
show(my_list, 3)
ver ejecución >>>
```

Se observa que el esquema general de recorrido con tratamiento selectivo es:

para cada *item en secuencia:*
 si *item cumple la propiedad X:*
 hacer algo con *item*

Exactamente igual que el esquema general de recorrido, pero añadiendo una pregunta antes de tratar el elemento.

Búsqueda lineal (esquema de búsqueda)

Saber si un valor está en una secuencia

En este tipo de problema se quiere saber si un valor determinado, o uno que cumpla una determinada condición, se encuentra en una secuencia. La solución es muy similar al esquema de recorrido con tratamiento selectivo (esquema de selección), con la diferencia de que, una vez que se ha encontrado algún elemento que cumpla el criterio de búsqueda, no habrá que continuar recorriendo hasta el final de la secuencia.

El siguiente ejemplo busca si hay en la secuencia algún elemento que sea múltiplo del valor pasado como segundo parámetro:

```
def find(seq, divisor):  
    for item in seq:  
        if item % divisor == 0:  
            return True  
  
    return False  
  
my_list = [10, 11, 12, 23, 35, 45, 67, 89]  
found = find(my_list, 3) # True
```

[ver ejecución >>>](#)

Si se llega a recorrer toda la secuencia, significa que no hay ningún elemento que cumpla la condición buscada, por lo que se devuelve *False* al finalizar el bucle. En ningún caso se puede devolver *False* como alternativa (*else*) a la pregunta dentro del bucle, ya que, en ese caso, solo se evaluaría el primer elemento de la secuencia:

```
def find(seq, divisor):  
    for item in seq:  
        if item % divisor == 0:  
            return True  
        else:  
            return False
```



```
my_list = [10, 11, 12, 23, 35, 45, 67, 89]  
found = find(my_list, 3) # False
```

[ver ejecución >>>](#)

Cuando se busca un valor que cumpla una condición, en lugar de un valor concreto, puede ser más útil devolver el valor encontrado en vez de simplemente *True* o *False*.

```
def find(seq, divisor):  
    for item in seq:  
        if item % divisor == 0:  
            return item  
  
    return False  
  
my_list = [10, 11, 12, 23, 35, 45, 67, 89]  
found = find(my_list, 3) # 12
```

[ver ejecución >>>](#)

Búsqueda de la posición de un valor en una secuencia

Si interesase saber en qué posición de una secuencia está el primer elemento que tenga un determinado valor o que cumpla una determinada propiedad, se necesitarían los índices de la secuencia:

```
def find(seq, divisor):
    for idx in range(len(seq)):
        item = seq[idx]

        if item % divisor == 0:
            return idx

    return False
```

```
my_list = [10, 11, 12, 23, 35, 45, 67, 89]
found = find(my_list, 3) # 2
```

[ver ejecución >>>](#)

Adicionalmente, es posible devolver una tupla con el valor y su posición, aunque, dado que se tiene la posición, se puede acceder al valor cuando se requiera.

En Python existe la alternativa de usar una función llamada *enumerate()* que devuelve parejas (índice, valor), en lugar de basar el recorrido en el rango de índices:

```
def find(seq, divisor):
    for idx, item in enumerate(seq):
        if item % divisor == 0:
            return idx

    return False
```

```
my_list = [10, 11, 12, 23, 35, 45, 67, 89]
found = find(my_list, 3) # 2
```

[ver ejecución >>>](#)

Operador de pertenencia y funciones de búsqueda

El operador *in* de Python no solo sirve para expresar la iteración de un bucle, sino también como operador de pertenencia para saber si un elemento se encuentra en un contenedor:

```
languages = ['C', 'Java', 'Python']
print('Pascal' in languages) # False
print('Python' in languages) # True
print('Java') not in languages # False

string = "En un lugar de La Mancha..."
print("La Mancha" in string) # True
print(['C', 'Java'] in languages) # False
```

[ver ejecución >>>](#)

Nótese que, en el caso de las strings, se puede evaluar la pertenencia no solo de un carácter individual, sino de una secuencia cualquiera de caracteres. En otros contenedores, en cambio, solo se evalúa la pertenencia de elementos individuales (en el ejemplo, la lista *['C', 'Java']* tendría que ser un elemento de la lista *languages*).

Python también ofrece el método `.index()`, que devuelve el índice de la primera aparición de un valor en una secuencia. Sin embargo, si el valor no está en la secuencia, produce un error. Para evitarlo, se recomienda comprobar primero si el valor está presente usando el operador `in`:

```
languages = ['C', 'Java', 'Python']
string = "En un lugar de La Mancha..."

print(languages.index('Java')) # 1
print(string.index("La Mancha")) # 15
ver ejecución >>>
```

Las strings también tienen el método `.find()`, que devuelve -1 si la substring buscada no se encuentra en la string de búsqueda:

```
string = "En un lugar de La Mancha..."
print(string.find("La Mancha")) # 15
print(string.find("la Mancha")) # -1
ver ejecución >>>
```

Es importante tener en cuenta que, en general, cada una de estas operaciones implementa internamente un esquema de búsqueda, por lo que su coste de ejecución es equivalente.

Búsqueda del máximo/mínimo de una secuencia

Si se tiene una secuencia de elementos entre los que se puede establecer una relación de orden total (es decir: $\forall x, y \in S, (x \leq y) \vee (x \geq y)$), el problema consiste en encontrar cuál es el valor más grande entre los elementos de la secuencia. A diferencia del esquema de búsqueda, aquí siempre habrá que recorrer la secuencia completa, ya que no se puede saber cuál es el valor máximo hasta que se hayan visto todos. Se empieza inicializando una variable con el primer valor de la secuencia. Esta variable servirá para ir recordando el valor máximo encontrado hasta un punto de la secuencia. Se tiene que actualizar cada vez que se encuentre un valor más grande, de forma que, al terminar el recorrido tendrá el valor máximo de toda la secuencia; de esta manera, se puede considerar un caso particular del esquema de selección: la condición para ser seleccionado es ser mayor que el último candidato seleccionado y la acción es convertirse en el nuevo candidato.

```
def máximo(seq):
    candidato = seq[0]

    for item in seq:
        if item > candidato:
            candidato = item

    return candidato

my_list = [12, 13, 23, 56, 48, 56, 21, 56, 56, 12]
found = máximo(my_list) # 56
ver ejecución >>>
```

La búsqueda del mínimo sería igual, solo habría que cambiar la comparación, sustituyendo `>` por `<`.

En la solución mostrada, la primera comparación es entre el *item* de la primera posición (*seq[0]*) y *candidato*, que se ha inicializado con *seq[0]*. Esta comparación de un elemento consigo mismo se puede evitar empezando el bucle en el segundo elemento, para lo que se usa un recorrido basado en índices:

```
def máximo(seq, divisor):
    candidato = seq[0]

    for idx in range(1, len(seq)):
        if seq[idx] > candidato:
            candidato = seq[idx]

    return candidato
```

```
my_list = [12, 13, 23, 56, 48, 56, 21, 56, 56, 12]
found = máximo(my_list, 3) # 56
```

[ver ejecución >>>](#)

Hay que recordar que la opción de que el bucle recorra el slice de la secuencia desde la segunda posición (**for** item **in** seq[1:]) no es adecuada porque un slice implica la copia del trozo de la secuencia especificado.

Esquema acumulador

Los esquemas presentados hasta aquí tratan cada elemento por separado, sin relacionarlos con los demás, salvo en el caso de la búsqueda del máximo/mínimo. El propósito de un esquema acumulador es obtener un valor que resulta de "acumular" de algún modo el conjunto de valores de la secuencia. Por ejemplo, el siguiente código suma los valores de una secuencia:

```
def suma(seq):
    acumulador = 0

    for item in seq:
        acumulador += item

    return acumulador
```

```
my_list = [12, 13, 23, 56, 48, 56, 21, 56, 56, 12]
total = suma(my_list) # 353
```

[ver ejecución >>>](#)

El esquema general de acumulación sería:

inicializar un acumulador con
el elemento neutro de la operación de acumulación

para cada item **en** secuencia:
 acumular item en el acumulador

Se pueden hacer acumulaciones con cualquier operación que tenga elemento neutro y sea asociativa. En el ejemplo anterior se usa la suma, cuyo elemento neutro es el cero. Otros ejemplos de operaciones acumulables pueden ser: la multiplicación (elemento neutro 1) o la concatenación de secuencias (elemento neutro "" para las strings, () para las tuplas y [] para las listas).

Acumulador selectivo

En el ejemplo anterior, se hace un recorrido general para acumular todos los elementos de la secuencia, pero también podría hacerse un recorrido con tratamiento selectivo, por ejemplo, para sumar solo los valores de la secuencia que sean impares:

```
def suma_impares(seq):
    acumulador = 0

    for item in seq:
        if item % 2 != 0:
            acumulador += item

    return acumulador

my_list = [12, 13, 23, 56, 48, 56, 21, 56, 56, 12]
total = suma_impares(my_list) # 296
```

[ver ejecución >>>](#)

Los esquemas de recorrido general y con tratamiento selectivo, sirven de base para cualquier otro esquema. Por ejemplo, de la búsqueda del máximo, se podría tener una versión que buscase el máximo valor de entre los de la secuencia que cumplen una determinada condición.

Esquema contador

El propósito es contar todos los elementos de una secuencia que cumplen una determinada condición. Básicamente, es una modificación del esquema acumulador en el que, en lugar de acumular el valor del elemento, se suma 1 por cada elemento objetivo:

```
def contar_impares(seq):
    contador = 0

    for item in seq:
        if item % 2 != 0:
            contador += 1

    return contador

my_list = [12, 13, 23, 56, 48, 56, 21, 56, 56, 12]
total = contar_impares(my_list) # 3
```

[ver ejecución >>>](#)

Se puede generalizar el esquema contador sumando cualquier cantidad fija, con lo que el valor resultante será un múltiplo del número de elementos contados. De esta forma, la diferencia entre un esquema acumulador y un esquema contador es que en el primero se acumula un valor dependiente de cada elemento y en el último un valor fijo.

Mutable e inmutable

Operaciones que modifican listas

Las listas son **mutables**, a diferencia de las strings y las tuplas, que son **inmutables**. Eso significa que, en el momento de crear una string o una tupla, hay que conocer todos los caracteres o elementos que la forman, ya que no podrán añadirse, borrarse o actualizarse posteriormente (las strings y las tuplas no pueden mutar).

Por el contrario, las listas pueden mutar, lo que significa que se les pueden añadir, borrar o actualizar elementos después de ser creadas.

Modificación de los elementos de una lista

Las listas, al ser secuencias mutables, ofrecen además operaciones para modificar, añadir y quitar elementos. Para modificar el valor que ocupa una determinada posición en una lista, solo hay que asignar a la lista, indicando el índice de la posición entre corchetes, el nuevo valor que sustituirá al original:

```
lst = [10, 20, 30, 40, 50]
lst[3] = 28    # lst == [10, 20, 30, 28, 50]
lst[-4] = 25   # lst == [10, 25, 30, 28, 50]
```

[ver ejecución >>>](#)

También se puede modificar un *slice*:

```
lst[1:4] = [14, 15]    # lst = [10, 14, 15, 40, 50]
```

Nótese que, en el ejemplo, los valores de la lista [14, 15] han sustituido en la lista *lst* a los del slice [1:4] ([10, 20, 25, 28, 40, 50]).

Añadir elementos a una lista

Python ofrece tres métodos para añadir elementos a una lista: *.append()*, *.insert()* y *.extend()*. El método *.append()* añade un valor al final de la lista, mientras que al método *.insert()* se le debe indicar la posición en la que se quiere insertar el nuevo valor:

```
lista = [1, 2, 3]
lista.append(5)           # [1, 2, 3, 5]
lista.insert(len(lista), 6) # [1, 2, 3, 5, 6]
lista.insert(-1, 7)       # [1, 2, 3, 7, 5, 6]
lista.insert(0, 8)         # [8, 1, 2, 3, 7, 5, 6]
lista.insert(2, 9)         # [8, 1, 9, 2, 3, 7, 5, 6]
```

[ver ejecución >>>](#)

Nótese que, para añadir un valor al final de la lista usando *.insert()* hay que indicar como índice de la posición de inserción la longitud de la lista; si se indica -1, se añadirá el nuevo valor en la que actualmente es la última posición, desplazando al elemento que la ocupa. Toda inserción en una posición que no sea al final desplaza los elementos desde el que actualmente ocupa esa posición hasta el final una posición a la derecha para hacer hueco al nuevo elemento.

Por su parte, el método *.extend()* requiere que el parámetro sea una secuencia y lo que hace es añadir los elementos de la misma al final de la lista:

```
lista = [1, 2, 3]
tupla = (5, 6, 7)
lista.extend(tupla)    # [1, 2, 3, 5, 6, 7]
```

[ver ejecución >>>](#)

Nótese que, si se usa *.append()* con el mismo parámetro, se añade la secuencia (completa) pasada como parámetro como último elemento de la lista:

```
lista = [1, 2, 3]
tupla = (5, 6, 7)
lista.append(tupla)    # [1, 2, 3, (5, 6, 7)]
```

[ver ejecución >>>](#)

Quitar elementos de una lista

Para eliminar un valor de una lista, se puede especificar su posición con la operación *del*:

```
del lst[3] # lst == [10, 14, 15, 50] (desapareció el 40)
```

También se puede eliminar un slice:

```
del lst[1:3] # lst == [10, 50] (desaparecieron el 14 y el 15)
```

El método *.pop()* quita de la lista y devuelve el elemento indicado por el índice que se pasa como parámetro. Si no se pasa ninguno, elimina el último elemento:

```
lst = [10, 20, 30, 40, 50, 60]
item = lst.pop(3) # item == 40 y lst == [10, 20, 30, 50, 60]
item = lst.pop() # item == 60 y lst == [10, 20, 30, 50]
```

También se pueden quitar el primer elemento que tenga un determinado valor con el método *.remove()*:

```
lst = [123, 34, 10, 2, 10, 14, 76]
lst.remove(10) # lst == [123, 34, 2, 10, 14, 76]
```

Importante: la operación *del* elimina físicamente el elemento, no solo lo quita de la lista. Puede verse la diferencia ejecutando el siguiente ejemplo:

```
lst = [
    (1, 1), (1, 2), (1, 3),
    (2, 1), (2, 2), (2, 3),
]
del lst[2]
item = lst.pop()
```

[ver ejecución >>>](#)

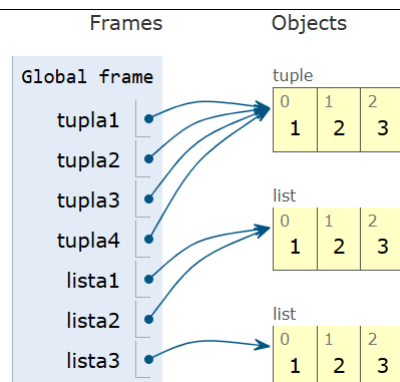
Consecuencias de la mutabilidad

Alias

Las variables en Python son referencias a valores (objetos) almacenados en memoria. Cuando se realiza una asignación, como, en el siguiente ejemplo, *tupla2 = tupla1*, ambas variables referencian a la misma tupla en memoria, igual que *lista1* y *lista2* referencian a la misma lista. Se dice que ambas variables son **alias** del mismo objeto (una lista, en este caso).

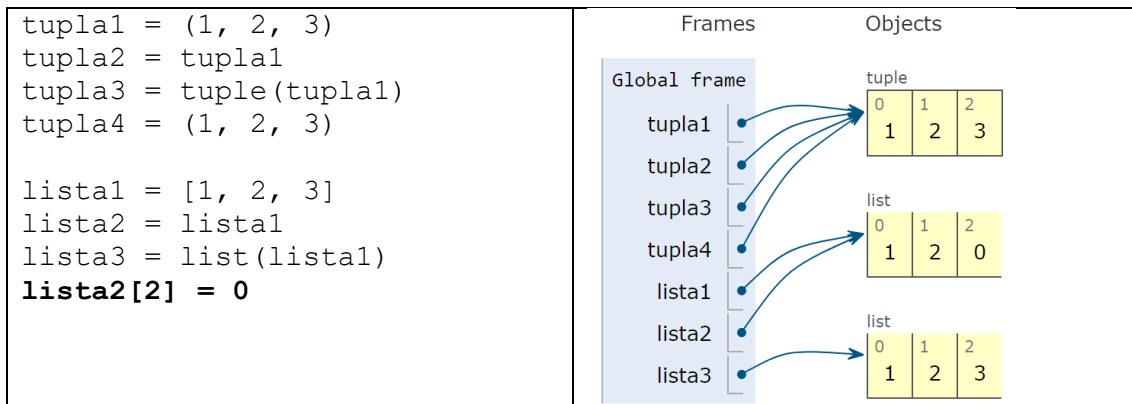
```
tupla1 = (1, 2, 3)
tupla2 = tupla1
tupla3 = tuple(tupla1)
tupla4 = (1, 2, 3)

lista1 = [1, 2, 3]
lista2 = lista1
lista3 = list(lista1)
```



[ver ejecución >>>](#)

Esto significa que si se modifica el objeto referenciado por una de las variables (la lista en cuestión), el objeto referenciado por la otra (que es el mismo) se vería asimismo modificado. En el siguiente ejemplo, tras asignar `lista2[2] = 0`, se vería que el elemento `lista1[2]` pasaría a valer 0, y no 2, como originalmente.



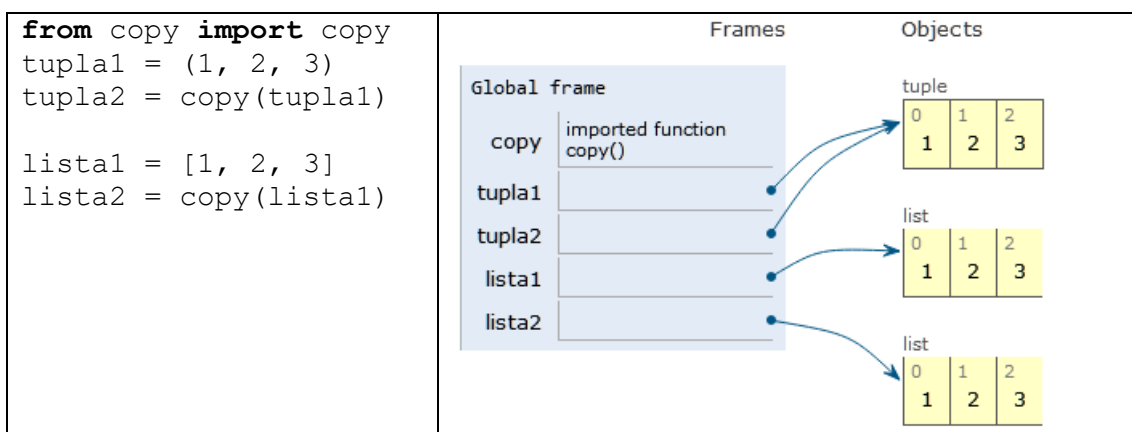
[ver ejecución >>>](#)

Nótese que el alias se rompe si se hace una asignación a una de las variables que referencian a un objeto; esto no afectará al resto de las variables que referencian a ese mismo objeto. Los alias solo son un problema cuando se usan operaciones de modificación en objetos mutables referenciados por varias variables diferentes.

Shallow copy y Deep copy

Si los alias son un problema, habrá que hacer una copia del objeto a asignar. En los ejemplos de la sección anterior se ha visto que se puede obtener una copia de una lista usando la función constructora `list()`. No sucede lo mismo con la función constructora de las tuplas, `tuple()`, que sigue generando un alias. La idea es que, puesto que los tipos inmutables no pueden modificarse, los alias no supondrían, en principio, un problema. De hecho, en el caso de los valores de clases inmutables, podría bastar con que existiese una única instancia para cualquier uso.

En Python, existe una función llamada `copy()` (del módulo `copy`) que permite hacer una copia de cualquier contenedor. Sin embargo, como se ve en el siguiente ejemplo, no copia las tuplas.



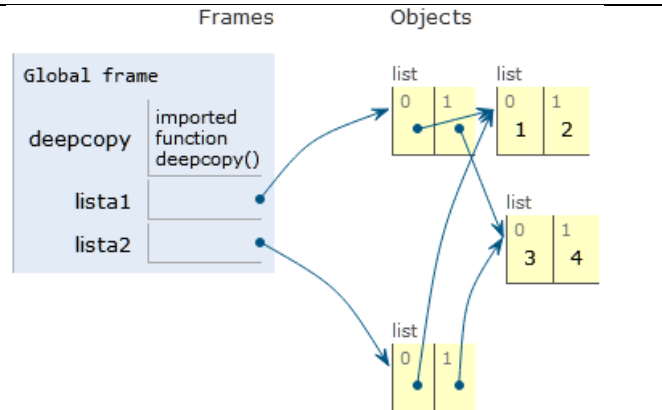
[ver ejecución >>>](#)

El problema es que un contenedor puede contener elementos de cualquier tipo, incluidos tipos mutables, creando estructuras multinivel, y la función `copy()` solo copia el primer nivel, realizando una copia superficial o *shallow copy*.

```

from copy import copy
lista1 = [
    [1, 2],
    [3, 4]
]
lista2 = copy(lista1)

```



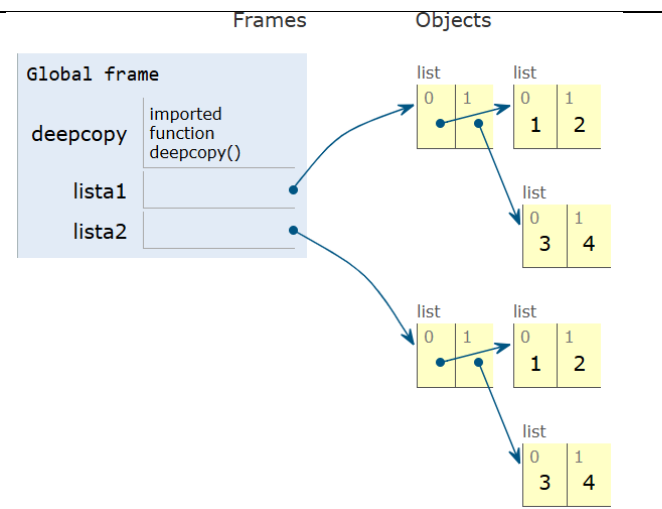
[ver ejecución >>>](#)

Para resolver este problema, existe la función *deepcopy()*, que itera por la estructura copiando todos los niveles.

```

from copy import deepcopy
lista1 = [
    [1, 2],
    [3, 4]
]
lista2 = deepcopy(lista1)

```



[ver ejecución >>>](#)

Listas y funciones

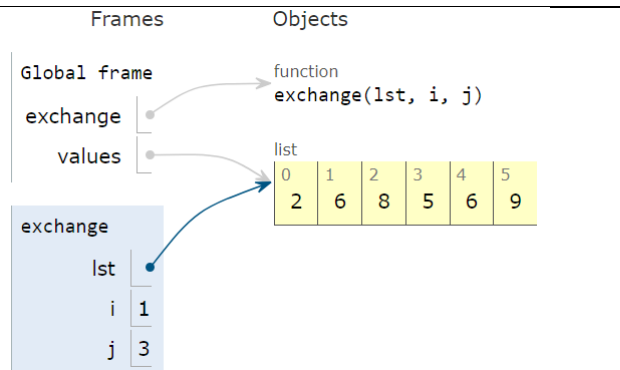
Cuando hay que procesar una lista, frecuentemente se pasa como parámetro a una función. En estos casos, el parámetro formal de la función referenciará a la misma lista que se pasa en la llamada (es un alias). En el siguiente ejemplo, la función *exchange()* intercambia dos elementos de una lista:

```

def exchange(lst, i, j):
    temp = lst[i]
    lst[i] = lst[j]
    lst[j] = temp

values = [2, 6, 8, 5, 6, 9]
exchange(values, 1, 3)

```



[ver ejecución >>>](#)

En consecuencia, hay que tener presente que si se quiere pasar una lista como parámetro a una función teniendo la seguridad de que no pueda modificarla, hay que realizar una copia de la misma.

Más esquemas de tratamiento

Creación de una lista

La siguiente función crea una lista extrayendo los números pares de la lista que se le pasa como parámetro.

```
def get_odds(lst):
    odds = []

    for item in lst:
        if item % 2 == 0:
            odds.append(item)

    return odds

my_list = [12, 31, 56, 7, 98, 72]
odds_list = get_odds(my_list)
```

[ver ejecución >>>](#)

En este caso, se usa claramente un esquema acumulador selectivo, en el que la lista vacía actúa como elemento neutro para la operación *append()*. También podría usarse un esquema acumulador no selectivo, pero siempre se empieza creando una lista vacía, y se utiliza una operación que permita añadir elementos a la lista (*append()*, *insert()* o *extend()*).

Creación de una lista usando list comprehension

Una función como la del ejemplo anterior se puede implementar en Python de forma más concisa usando *list comprehension*:

```
def get_odds(lst):
    return [item for item in lst if item % 2 == 0]

my_list = [12, 31, 56, 7, 98, 72]
odds_list = get_odds(my_list)
```

[ver ejecución >>>](#)

El mecanismo de *list comprehension* de Python proporciona una forma concisa de crear nuevas listas. Consiste en una expresión, seguida de una cláusula *for*, seguida a su vez de cero o más cláusulas *for* o *if*; todo ello entre corchetes. Por ejemplo:

```
[expression for element in list]
[expression for element in list if condition]
```

Con esta notación, se obtiene una nueva lista, compuesta de los valores que resulten de evaluar repetidamente la expresión (que puede ser compleja) en el contexto de las cláusulas *for* e *if* que la siguen. Aquí hay algunos ejemplos; estudie atentamente los resultados:

```
list1 = [i for i in range(10)] # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
list2 = [i ** 2 for i in range(5)] # [0, 1, 4, 9, 16]
```

```
import random
rand_list = [random.randint(0, 9) for i in range(10)]
# [4, 4, 9, 7, 4, 1, 0, 0, 3, 6]
```

(La función `random.randint()` genera un número al azar en el rango indicado; la salida será diferente en diferentes ejecuciones)

```
words = ['this', 'is', 'a', 'list', 'of', 'words']
iniciales = [word[0].upper() for word in words]
# ['T', 'I', 'A', 'L', 'O', 'W']
```

(El método `upper()` devuelve la conversión a mayúsculas de la string asociada)

```
things = ['pen', 'pencil', 'rubber', 'paper']
large_things = [thing for thing in things if len(thing) > 5]
# ['pencil', 'rubber']
```

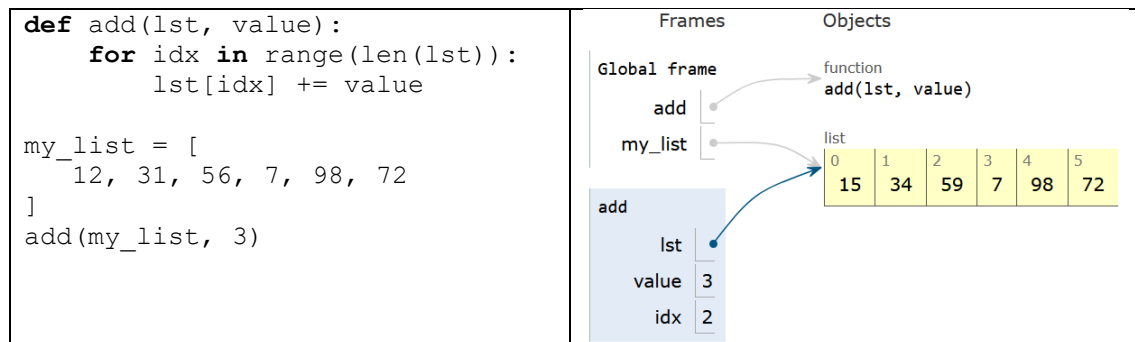
```
from math import pi
pi_rounds = [str(round(pi, i)) for i in range(1, 6)]
# ['3.1', '3.14', '3.142', '3.1416', '3.14159']
```

(La función `round()` redondea el `float` pasado como primer parámetro al número de decimales indicado por el segundo)

[ver ejecución >>>](#)

Modificación de una lista

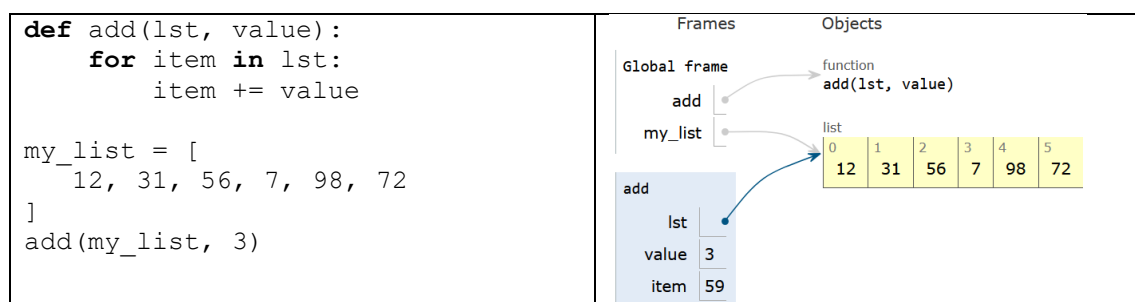
La siguiente función suma un valor a cada elemento de la lista pasada como parámetro:



[ver ejecución >>>](#)

Al modificarse directamente la lista pasada como parámetro, no es necesario crear una nueva lista, y la función no tiene que devolverla al final.

Nótese que se usa un recorrido basado en índices para modificar la lista, en vez de un recorrido por elementos. Por ejemplo, la siguiente función:



[ver ejecución >>>](#)

no modifica los elementos de la lista, sino el valor de la variable *item*, que es independiente de los elementos originales de la lista.

Esquemas de procesamiento sincronizado de varias secuencias

Procesamiento sincronizado de varias secuencias

En muchas ocasiones, se necesita procesar dos o más secuencias simultáneamente. El siguiente ejemplo intercala los elementos de dos listas, generando una nueva:

```
def interleave(list1, list2):
    result = []

    for idx in range(min(len(list1), len(list2))):
        result.append(list1[idx])
        result.append(list2[idx])

    return result

numbers = [10, 20, 30]
chars = ['A', 'B', 'C']
both = interleave(numbers, chars) # [10, 'A', 20, 'B', 30, 'C']
```

[ver ejecución >>>](#)

Como se observa en el ejemplo, no hace falta un bucle independiente para cada lista, ya que el objetivo es recorrerlas al mismo tiempo, tomando un elemento de cada una para pasarlo al resultado. Es decir, 1 recorrido (de 1, 2, o más listas a la vez) == 1 bucle.

Nótese que en el rango se usa el valor mínimo de las longitudes de las listas a tratar. No se puede usar la longitud de una cualquiera de las listas, a menos que se tenga la certeza de que son exactamente del mismo tamaño. De lo contrario, si se elige la más larga, se producirá un error al intentar acceder a un elemento de la más corta más allá de su rango de índices (*index out of range*).

Procesamiento sincronizado usando zip

Si no es necesario acceder a los índices, una forma de procesar dos o más listas al mismo tiempo es usar la función *zip()*:

```
def interleavel(list1, list2):
    result = []

    for item1, item2 in zip(list1, list2):
        result.append(item1)
        result.append(item2)

    return result

list1 = [10, 20, 30, 40, 50]
list2 = ['A', 'B', 'C']
both = interleavel(list1, list2) # [10, 'A', 20, 'B', 30, 'C']
```

[ver ejecución >>>](#)



La función predefinida *zip()* permite iterar por una secuencia de tuplas, formadas de tal manera que el *i*-ésimo elemento de esa secuencia es una tupla compuesta por los *i*-ésimos elementos de cada una de las listas (o, en general, secuencias) que se le pasen como parámetros.

En el ejemplo, el primer elemento devuelto por *zip()* es la tupla (10, 'A') formada por el primer elemento de *list1* y el primer elemento de *list2*; como segundo elemento la tupla (20, 'B') y como tercer y último elemento la tupla (30, 'C').

Nótese que en cada iteración del bucle, cada elemento/tupla de la secuencia *zip()* es disgregado en las variables *item1* e *item2*. Para la tupla (10, 'A'), *item1* tomará el valor 10 e *item2* el valor 'A'.

Cuando las listas tienen diferentes longitudes, *zip()* solo procesa hasta la longitud de la más corta.

Procesamiento hasta la longitud de la más larga usando *zip_longest*

La función *zip()* permite recorrer simultáneamente dos o más listas, pero solo hasta el tamaño de la más corta. Si se quiere recorrer hasta el tamaño de la más larga, se puede usar la función *zip_longest()* del módulo *itertools*:

```
from itertools import zip_longest

def interleave2(list1, list2):
    result = []

    for item1, item2 in zip_longest(list1, list2, fillvalue='-'):
        result.append(item1)
        result.append(item2)

    return result
```

```
list1 = [10, 20, 30]
list2 = ['A', 'B', 'C', 'D']
both = interleave2(list1, list2)  # [10, 'A', 20, 'B', 30, 'C', '-', 'D']
```

[ver ejecución >>>](#)

Los elementos que faltan en la lista más corta se sustituyen con el valor especificado en el parámetro *fillvalue* (el carácter '-' en el ejemplo). Si se omite este parámetro, se usa el valor *None*.

Procesamiento hasta la longitud de la más larga usando índices

Si se quiere recorrer dos listas de diferentes tamaños a la vez sin usar `zip_longest()`, es conveniente, como en el siguiente ejemplo, separar la iteración en tres bucles, de los cuales solo se ejecutan, como máximo, dos: el primero, mientras ambas listas tienen elementos, y uno de los otros dos, para terminar de procesar los elementos de la lista más larga.

```
def interleave3(list1, list2):
    result = []
    min_len = min(len(list1), len(list2))

    for idx in range(min_len):
        result.append(list1[idx])
        result.append(list2[idx])

    for idx in range(min_len, len(list1)):
        result.append(list1[idx])
        result.append('-')

    for idx in range(min_len, len(list2)):
        result.append('-')
        result.append(list2[idx])

    return result
```

[ver ejecución >>>](#)

Alternativamente, se pueden usar sentencias *while*, aprovechando el valor del índice a la salida del primer bucle.

Si se hiciera un solo bucle hasta la longitud de la más larga, sería necesario incluir comprobaciones dentro del bucle para verificar si alguna lista ha terminado, lo que incrementaría el coste de ejecución.

```
def interleave3b(list1, list2):
    result = []

    for idx in range(max(len(list1), len(list2))):
        if idx < len(list1):
            result.append(list1[idx])
        else:
            result.append('-')

        if idx < len(list2):
            result.append(list2[idx])
        else:
            result.append('-')

    return result
```

[ver ejecución >>>](#)

Estructuras multidimensionales

Secuencias de secuencias

Un contenedor puede almacenar elementos de cualquier tipo, incluidos otros contenedores. Así, se pueden crear tuplas de tuplas, tuplas de listas, listas de tuplas, listas de listas, etc., dando lugar a estructuras de datos complejas que se extienden en varios niveles o dimensiones.

El siguiente ejemplo representa una estructura de datos para el menú de un restaurante. El primer nivel es una lista de categorías. Cada categoría es una tupla con dos elementos: el nombre de la categoría y una lista de platos de esa categoría. Los elementos de las listas de platos son, a su vez, tuplas compuestas por el nombre de un plato y su precio, conformando así una estructura jerárquica de categorías y platos.

```
menú = [  
    ("Entrantes calientes", [  
        ("Sopa de tomate", 4.50),  
        ("Croquetas de jamón", 6.50),  
        ("Calamares a la romana", 9.50)  
    ]),  
    ("Entrantes fríos", [  
        ("Ensalada César", 7.50),  
        ("Gazpacho", 5.00),  
        ("Carpaccio de salmón", 10.00)  
    ]),  
    ("De cuchara", [  
        ("Lentejas", 6.00),  
        ("Sopa de mariscos", 9.00),  
        ("Crema de calabaza", 5.50)  
    ])  
]
```

[ver ejecución >>>](#)

Para acceder a un elemento específico, se usa una secuencia de índices, tantos como es necesario para alcanzar el nivel de detalle requerido. Por ejemplo, la instrucción:

```
print(menú[0][1][2][1])
```

[ver ejecución >>>](#)

muestra 9.5, el precio de los "Calamares a la romana". La variable *menú* referencia la estructura completa, *menú[0]* referencia la primera tupla, *menú[0][1]* referencia la lista de platos de esa tupla, *menú[0][1][2]* referencia la tupla del tercer plato de esa lista y *menú[0][1][2][1]* referencia el precio de ese plato.

Tablas y matrices

A menudo, se necesita implementar una estructura bidimensional, como una tabla con filas y columnas (una matriz de valores desde un punto de vista matemático). Dicha tabla de valores se puede crear utilizando listas de listas en Python:

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

o, en una forma más legible:

```
m = [  
    [1, 2, 3],
```

```

        [4, 5, 6],
        [7, 8, 9]
    ]

```

Cada uno de los elementos $m[0]$, $m[1]$, $m[2]$ referencia a una lista de 3 elementos. Otra forma de crear la misma estructura:

```

row1 = [1, 2, 3]
row2 = [4, 5, 6]
row3 = [7, 8, 9]
m = [row1, row2, row3]

```

El número de filas o columnas se puede conocer usando la función `len()`.

```

print(len(m))    # 3 - número de filas

```

La longitud de cualquier elemento, por ejemplo $m[0]$, da el número de columnas.

```

print(len(m[0])) # 3 - número de columnas

```

Los elementos de la tabla se pueden acceder mediante una pareja de índices. La expresión $m[i][j]$ representa el valor del elemento que está en la columna j de la fila i . El siguiente ejemplo cambia el valor de la primera columna de la segunda fila de la tabla m y muestra la tabla:

```

m[1][0] = 0
print(m)    # [[1, 2, 3], [0, 5, 6], [7, 8, 9]]

```

Tratando secuencias de secuencias

A la hora de manejar una estructura multidimensional, se pueden distinguir dos "casos": uno en el que cada "fila" se pueda procesar independientemente y luego combinar su resultado con el del resto de las "filas" para obtener el resultado final, y otro en el que se requieran simultáneamente elementos de diferentes "filas". Al primero se le llama "secuencias de secuencias" y al segundo, "matrices".

La siguiente función devuelve el nombre y el precio del plato más barato de un menú (si hay varios con el mismo precio mínimo, el del primero que encuentre):

```

def expensive_dish(menú):
    expensive = (None, -1.0)

    for category_name, dishes in menú:
        expensive_name, expensive_price = expensive
        cat_expensive_name, cat_expensive_price = \
            category_expensive(dishes)

        if cat_expensive_price > expensive_price:
            expensive = (cat_expensive_name, cat_expensive_price)

    return expensive

```

Se trata de un esquema de búsqueda del máximo, pero en lugar de plantear la búsqueda del máximo entre todos los platos, lo que se hace es elegir el máximo entre los máximos de cada categoría. Para ello, se utiliza una función aparte que sigue el mismo esquema de búsqueda del máximo:

```
def category_expensive(dishes):
    expensive = dishes[0]

    for name, price in dishes:
        expensive_name, expensive_price = expensive

        if price > expensive_price:
            expensive = name, price

    return expensive
```

De esta manera, se conservan en cada nivel los esquemas básicos lineales de tratamiento de secuencias ya estudiados, en contraste con la alternativa de aplicar un esquema nuevo, bidimensional, que no es más que el resultado de anidar explícitamente los esquemas correspondientes a cada nivel:

```
def expensive_dish(menú):
    expensive = (None, -1.0)

    for category_name, dishes in menú:
        expensive_name, expensive_price = expensive

        for name, price in dishes:
            if price > expensive_price:
                expensive = name, price

    return expensive
```

Como nota al margen, obsérvese cómo en todos los ejemplos anteriores se desempaquetan las tuplas en sus componentes para evitar trabajar con múltiples índices y dejar más claro con qué elemento se está trabajando en cada momento.

Tratando matrices

El siguiente ejemplo verifica si una matriz cuadrada de números enteros (representada como una lista de listas) es simétrica:

```
def is_symetric(mat):
    for row_idx in range(1, len(mat)):
        for col_idx in range(row_idx - 1):
            if mat[row_idx][col_idx] != mat[col_idx][row_idx]:
                return False

    return True
```

[ver ejecución >>>](#)

Se trata de un esquema de búsqueda bidimensional. En este caso, no se tiene la opción de aislar el tratamiento de cada fila, ya que se requiere comparar elementos de filas diferentes.

Cuestión para reflexionar: ¿Por qué en los bucles del ejemplo se usan los rangos que se usan?

Se puede implementar una versión multidimensional para cualquiera de los esquemas lineales de tratamiento de secuencias estudiados. Para ello, será necesario anidar un bucle por cada dimensión que se tenga que recorrer, con el fin de acceder a los elementos individuales requeridos.

El siguiente ejemplo muestra dos formas de obtener la suma de los elementos de una estructura bidimensional de números enteros. A la de la izquierda, se presenta un esquema acumulador bidimensional, mientras que, a la derecha, se emplean dos esquemas acumuladores monodimensionales:

<pre>def mat_sum(mat): sum = 0 for row in mat: for item in row: sum += item return sum ver ejecución >>></pre>	<pre>def mat_sum(mat): sum = 0 for row in mat: sum += row_sum(row) return sum def row_sum(row): sum = 0 for item in row: sum += item return sum ver ejecución >>></pre>
---	---

ANEXO: Operaciones de secuencias, contenedores y listas

Operaciones generales de contenedores

<pre>lst1 = [20, 10, 30, 10, 40] lst2 = [60, 70] lst3 = [0, 1, 2, 3] lst4 = [0, 0, 0, 0]</pre>		
Operación	Uso	Resultado
len()	<code>x = len(lst1)</code>	<code>x == 5</code>
min()	<code>x = min(lst1)</code>	<code>x == 10</code>
max()	<code>x = max(lst1)</code>	<code>x == 40</code>
sum()	<code>x = sum(lst1)</code>	<code>x == 110</code>
in	<code>x = 30 in lst1</code>	<code>x == True</code>
	<code>x = 70 in lst1</code>	<code>x == False</code>
enumerate()	<pre>seq = list(enumerate(lst1))</pre>	<code>seq == [(0, 20), (1, 10), (2, 30), (3, 10), (4, 40)]</code>
zip()	<pre>seq = list(zip(lst1, lst2))</pre>	<code>seq == [(20, 60), (10, 70)]</code>
zip_longest() (importar de itertools)	<pre>from itertools import zip_longest seq = list(zip_longest(lst1, lst2))</pre>	<code>seq == [(20, 60), (10, 70), (30, None), (10, None), (40, None)]</code>
all()	<code>x = all(lst1)</code>	<code>x == True</code>
	<code>x = all(lst3)</code>	<code>x == False</code>
any()	<code>x = any(lst3)</code>	<code>x == True</code>
	<code>x = any(lst4)</code>	<code>x == False</code>

<pre>lst1 = [[20, 10, 30], [10, 40]]</pre>		
Operación	Uso	Resultado
copy() (importar de copy)	<pre>from copy import copy seq = copy(lst1)</pre>	Frames Global frame lst1 copy imported function copy() seq Objects list 02011030 list 011040 list 01

<pre>lst1 = [[20, 10, 30], [10, 40]]</pre>		
Operación	Uso	Resultado
deepcopy() (importar de copy)	from copy import deepcopy seq = deepcopy(lst1)	Frames Objects

Operaciones de contenedores secuenciales

(tuplas, strings, listas)

<pre>lst1 = [20, 10, 30, 10, 40] lst2 = [60, 70]</pre>		
Operación	Uso	Resultado
reversed()	lst3 = reversed(lst1)	lst3 == [40, 10, 30, 10, 20]
.index()	x = lst1.index(10)	x = 1
.count()	c = lst1.count(10)	c == 2
+	lst3 = lst1 + lst2	lst3 == [20, 10, 30, 10, 40, 60, 70]
*	lst3 = lst1 * 3	lst3 == [60, 70, 60, 70, 60, 70]

Operaciones de listas

<pre>lst1 = [20, 10, 30, 10, 40] lst2 = [60, 70]</pre>		
Operación	Uso	Resultado
.append()	lst1.append(99)	lst1 == [20, 10, 30, 10, 40, 99]
.extend()	lst1.extend(lst2)	lst1 == [20, 10, 30, 10, 40, 60, 70]
.insert()	lst1.insert(3, 99)	lst1 == [20, 10, 30, 99, 10, 40]
.remove()	lst1.remove(10)	lst1 == [20, 30, 10, 40]
.pop()	lst1.pop()	lst1 == [20, 10, 30, 10]
	lst1.pop(3)	lst1 == [20, 10, 30, 40]
.sort()	lst1.sort()	lst1 == [10, 10, 20, 30, 40]
	lst1.sort(reverse=True)	lst1 == [40, 30, 20, 10, 10]
.reverse()	lst1.reverse()	lst1 == [40, 10, 30, 10, 20]

Tema 5 Recursividad

Solución de problemas mediante diseño descendente

Habitualmente, la forma de resolver un problema es dividirlo en subproblemas más pequeños que puedan resolverse por separado y luego combinar las soluciones para obtener la solución al problema completo. Por ejemplo, el siguiente conjunto de funciones resuelve el problema de calcular cuántos elementos de una secuencia de números están por encima de la media:

<pre>def greater_than_average(seq): avg = calculate_average(seq) gt_count = greater(seq, avg) return gt_count</pre>	<pre>def calculate_sum(seq): accumulator = 0 for item in seq: accumulator += item return accumulator</pre>
<pre>def calculate_average(seq): addition = calculate_sum(seq) average = addition / len(seq) return average</pre>	<pre>def greater(seq, value): count = 0 for item in seq: if item > value: count += 1 return count</pre>

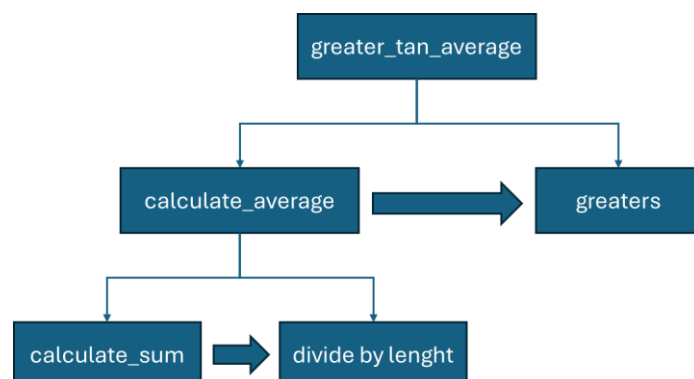
[ver ejecución >>>](#)

La función *greater_than_average()* es la que resuelve el problema de calcular cuántos elementos de la secuencia pasada como parámetro son mayores que el valor medio de la secuencia. Para hacerlo, divide el problema en dos subproblemas:

1. Calcular el valor medio de la secuencia.
2. Contar cuántos elementos de la secuencia son mayores que ese valor.

Para calcular el valor medio de la secuencia se usa la función *calculate_average()*, que, a su vez, divide el problema en dos subproblemas: sumar los elementos de la secuencia y dividir dicha suma por el número de elementos de la secuencia.

El primer subproblema se resuelve llamando a la función *calculate_sum()*, que implementa un esquema acumulador. Se podía haber usado directamente la función *sum()* de Python, pero se ha hecho así para enfatizar su naturaleza como problema separado.



Finalmente, el problema de contar los elementos de la secuencia que son mayores que la media se encarga a la función *greater()*, que implementa un esquema contador.

Solución de problemas mediante recursividad

¿Qué es la recursividad?

La recursividad es una técnica de solución de problemas que se basa en dividir un problema en versiones más pequeñas de sí mismo, que se resuelven por separado para luego combinar las soluciones y obtener la solución del problema completo. A menudo, los problemas que pueden resolverse recursivamente presentan una estructura recursiva en su propia definición.

Por ejemplo, el factorial de un número natural se define habitualmente como:

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n * (n - 1)! & \text{si } n > 0 \end{cases}$$

El factorial de un número, n , mayor que 0 se define en función del factorial de un número más pequeño, $n - 1$.

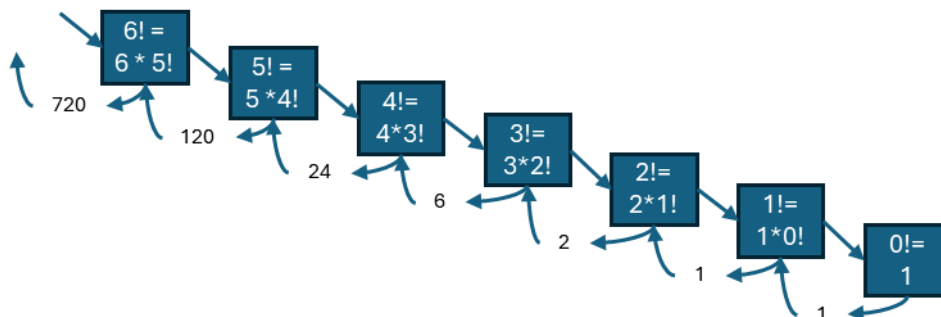
Esta definición se puede trasladar directamente a la implementación de una función:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

[ver ejecución >>>](#)

En la técnica de resolución de problemas por diseño descendente, cuando se identifica un subproblema más pequeño, generalmente se usa una función para resolverlo por separado. En este caso, el subproblema más pequeño $((n - 1)!)!$ es un caso más pequeño del mismo problema, calcular el factorial de un número natural. Por lo tanto, la función adecuada para resolverlo es la misma que la que resuelve el problema general.

El efecto no debe interpretarse como que "se vuelve a llamar a la misma función en un punto determinado". Lo que realmente ocurre es que se llama a una instancia (copia) de la misma función, que se ejecuta desde cero con unos parámetros diferentes. Funciona igual que si se llamara a una función distinta.



La solución se compone "hacia arriba", a medida que las instancias que calculan el factorial de números más pequeños van retornando.

Caracterización de un problema recursivo

La capacidad de encontrar una solución recursiva a un problema depende, en gran medida, de cómo se conciba el problema. Supóngase que, en lugar de definir el número factorial como se hizo en el apartado anterior, el problema se expresara de la siguiente manera:

"El factorial de cero es 1. El factorial de cualquier número natural mayor que cero se calcula multiplicando todos los número naturales desde 1 hasta n ".

Esta definición conduce, de forma natural, a la implementación de un esquema acumulador iterativo con la operación producto:

```
def factorial(n):  
    fact = 1  
  
    for num in range(1, n + 1):  
        fact *= num  
  
    return fact
```

Cuando se intenta resolver un problema aplicando la técnica de diseño descendente, se busca expresarlo en función de subproblemas más pequeños. En este caso, se expresa el factorial como una secuencia de multiplicaciones, lo que lleva al esquema acumulador.

Si se busca encontrar una solución recursiva, es necesario intentar expresar el problema de manera que algunos de los subproblemas sean versiones mas pequeñas del mismo problema. Esto es precisamente lo que se hace en la sección anterior al expresar el factorial de n en función de una multiplicación y el factorial de $n - 1$.

Para lograr expresar un problema en forma recursiva, se deben considerar una serie de cuestiones:

- ¿Qué factores determinan el tamaño del problema?
- ¿Cómo se pasa de una versión del problema a otra(s) más pequeña?
- ¿En qué casos el problema no puede dividirse de forma que contenga versiones más pequeñas de sí mismo?
- ¿Cómo se combinan las soluciones de los subproblemas para encontrar la solución global?

Identificación de los factores que determinan el tamaño de un problema

El tamaño de un problema se asocia con el trabajo que hay que realizar para resolverlo (por ejemplo, número de pasos a ejecutar). Un problema es "mayor" cuanto más trabajo se requiera para resolverlo. En el caso del factorial, solo hay un parámetro, n , que determina el tamaño del problema; a mayor valor de n , más multiplicaciones se deben realizar:

```
1! = 1 * 1  
2! = 2 * 1 * 1  
3! = 3 * 2 * 1 * 1
```

Considérese otro problema: calcular la potencia $base^{exponente}$, siendo $base$ un número y $exponente$ un número natural. La potencia se calcula multiplicando la $base$ por sí misma tantas veces como

indique el exponente, salvo que el exponente sea cero, que es un caso particular en el que la potencia es igual a 1.

$$a^0 = 1; a^n = \prod_{i=1}^n a$$

$3^1 = 3$	$1^3 = 1 * 1 * 1 = 1$
$3^2 = 3 * 3 = 9$	$2^3 = 2 * 2 * 2 = 8$
$3^3 = 3 * 3 * 3 = 27$	$3^3 = 3 * 3 * 3 = 27$
$3^4 = 3 * 3 * 3 * 3 = 81$	$4^3 = 4 * 4 * 4 = 64$

Aunque en este caso el problema tiene dos parámetros, es evidente que el valor de la base no afecta al número de operaciones realizadas, como sí lo hace el valor del exponente, que es quien determina el tamaño del problema de la potencia.

Considérese un tercer problema: contar el número de elementos con valor par en una lista de números. Para resolverlo, se deben examinar todos los elementos de la lista, evaluando si son pares o no. El número de evaluaciones a realizar no depende ni del valor de los elementos ni de cuántos de ellos sean pares; es el tamaño de la lista el que determina el tamaño del problema en este caso.

Paso del problema a una versión más pequeña

Es necesario identificar la presencia de una versión más pequeña del mismo problema dentro del problema general. Una estrategia útil es analizar ejemplos concretos de diferentes casos del problema con tamaño creciente, como se hace a continuación con el problema del cálculo de la potencia:

$3^0 = 1$
$3^1 = 3 * 1 = 3 * 3^0$
$3^2 = 3 * 3 * 1 = 3 * (3 * 1) = 3 * 3^1 = 9$
$3^3 = 3 * 3 * 3 * 1 = 3 * (3 * 3 * 1) = 3 * 3^2 = 27$
$3^4 = 3 * 3 * 3 * 3 * 1 = 3 * (3 * 3 * 3 * 1) = 3 * 3^3 = 81$
$3^5 = 3 * 3 * 3 * 3 * 3 * 1 = 3 * (3 * 3 * 3 * 3 * 1) = 3 * 3^4 = 243$

No es difícil darse cuenta de que cada caso, salvo cuando el exponente es cero, contiene el caso anterior. De aquí se obtiene la siguiente relación de recurrencia:

$$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a * a^{n-1} & \text{si } n > 0 \end{cases}$$

La relación de ocurrencia puede no ser única, por lo que es conveniente explorar varias opciones para elegir la que parezca más apropiada. Por ejemplo, en el caso de la potencia, otra división posible del problema sería:

$3^0 = 1$
$3^1 = 3 * 1 = 3$
$3^2 = 3 * 3 * 1 = 3^1 * 3^1 = 9$
$3^3 = 3 * 3 * 3 * 1 = (3) * (3 * 3) = 3^1 * 3^2 = 27$
$3^4 = 3 * 3 * 3 * 3 * 1 = (3 * 3) * (3 * 3) = 3^2 * 3^2 = 81$
$3^5 = 3 * 3 * 3 * 3 * 3 * 1 = (3 * 3) * (3 * 3 * 3) = 3^2 * 3^3 = 243$

En este caso, aprovechando las propiedades de la multiplicación, se ha intentado una división "más equilibrada" del problema en dos mitades con aproximadamente el mismo número de multiplicaciones de la base, lo que da la siguiente relación de recurrencia:

$$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a & \text{si } n = 1 \\ a^{\lfloor n/2 \rfloor} * a^{\lfloor n/2 \rfloor} & \text{si } n > 1 \end{cases}$$

Al desarrollar la relación de recurrencia, se obtiene una definición particionada del problema³, en la que se separan diferentes casos, lo que automáticamente identifica los casos del problema que no admiten subdivisiones, que se conocen como **casos base**, en oposición al **caso general**.

Para el problema del cálculo de la potencia, la primera relación de recurrencia da que el caso base es cuando el exponente es igual a cero; los problemas de cualquier tamaño acaban subdividiéndose recursivamente hasta alcanzar este valor. La segunda relación de recurrencia da lugar a dos casos base: exponente cero, y exponente 1, dado que este último no puede dividirse en dos partes tal como se pretende.

En todo caso, la fórmula, o fórmulas, del caso general deben ser convergentes a un caso base. La fórmula del caso general incluye la respuesta a la pregunta de cómo se combinan las soluciones de los subproblemas para encontrar la solución del problema general: multiplicándolas por la base en la primera recurrencia y multiplicándolas entre sí en la segunda.

Implementación de la solución

Una vez que se tiene la recurrencia, es relativamente fácil trasladarla a una función en un lenguaje como Python.

Dado que la definición recursiva del problema estará particionada en caso general y casos base, el esquema general de una función que implemente una solución recursiva estará dominado por una estructura de selección alternativa que discrimina los distintos casos.

A continuación, se muestran la solución iterativa del cálculo de la potencia, con un esquema acumulador, y las soluciones recursivas correspondientes a las dos relaciones de recurrencia planteadas.

$a^0 = 1; a^n = \prod_{i=1}^n a$	<pre>def pow(base, exp): result = 1 for idx in range(exp): result *= base return result</pre> ver ejecución >>>
$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a * a^{n-1} & \text{si } n > 0 \end{cases}$	<pre>def pow(base, exp): if exp == 0: return 1 else: return base * pow(base, exp - 1)</pre> ver ejecución >>>

³ En los problemas que admiten una solución recursiva, incluso la definición no recursiva suele estar particionada, a poco que se analice detenidamente el problema.

$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a & \text{si } n = 1 \\ a^{\lfloor n/2 \rfloor} * a^{\lfloor n/2 \rfloor} & \text{si } n > 1 \end{cases}$	<pre>def pow(base, exp): if exp == 0: return 1 elif exp == 1: return base else: first = pow(base, exp // 2) second = pow(base, (exp + 1) // 2) return first * second</pre> ver ejecución >>>
--	--

Análisis de las soluciones recursivas

Profundidad de la recursión

En las soluciones recursivas planteadas para el problema del cálculo de la potencia, el código desarrollado a partir de la primera relación de recurrencia parece, aparentemente, más sencillo que el desarrollado a partir de la segunda. Sin embargo, conviene examinarlo en mayor detalle.

La recursividad de la primera solución es lineal: cada instancia de la función correspondiente al caso general produce una única llamada recursiva, hasta llegar al caso base, desde donde comienza a retornarse la solución siguiendo el camino inverso.

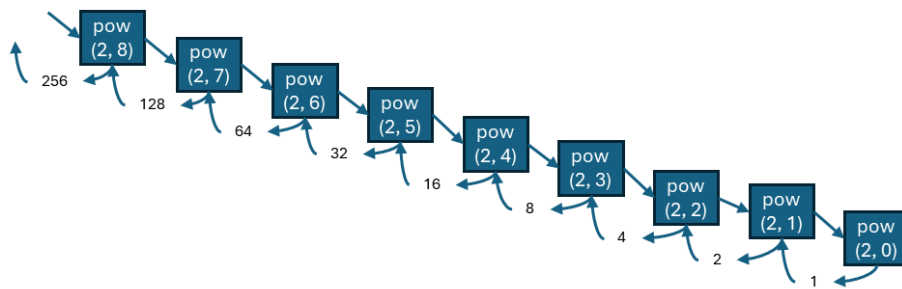


Figura 27 Profundidad de la recursión del cálculo de la potencia (Solución 1)

La recursividad de la segunda solución es no lineal: cada instancia de la función correspondiente a un caso general genera dos ramas de recursión. Primero desciende por una, hasta llegar a un caso base desde donde se retorna para descender a continuación por la segunda rama y se devuelve la solución una vez que ambas ramas se han completado.

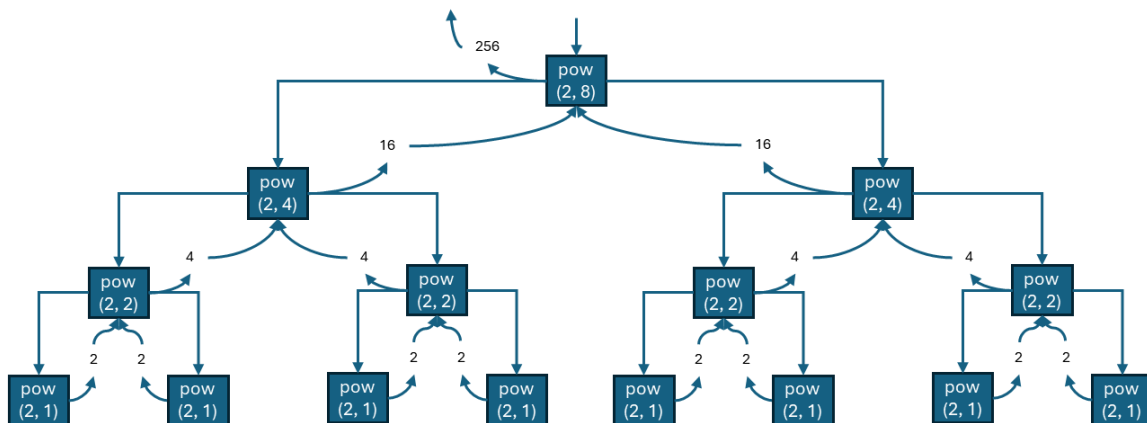


Figura 28 Profundidad de la recursión del cálculo de la potencia (Solución 2)

Esta diferencia de comportamiento afecta claramente a la **profundidad de la recursión**. La profundidad de la recursión es el número de instancias activas hasta alcanzar un caso base. En la primera solución, es directamente proporcional al exponente. Para el caso de las figuras, con exponente 8, se hacen 8 llamadas recursivas hasta alcanzar el caso base. Para la segunda solución, es proporcional a logaritmo en base 2 del exponente. Para exponente 8, por cualquier rama se hacen como mucho tres llamadas recursivas para alcanzar el caso base. Esto se debe a que en la primera solución el tamaño del problema decrece en 1 en cada llamada, mientras que en la segunda solución se divide por dos en cada llamada.

$$\log_2 n = x \rightarrow 2^x = n$$

La cantidad de recursos de máquina (por ejemplo, memoria RAM) usados en un momento dado es proporcional al número de instancias activas, por lo que la primera solución requiere disponer de más recursos que la segunda, que usa una cantidad menor, aunque por un tiempo mayor. Como los recursos son limitados, en principio, la segunda solución podría resolver problemas de mucho más tamaño que la primera.

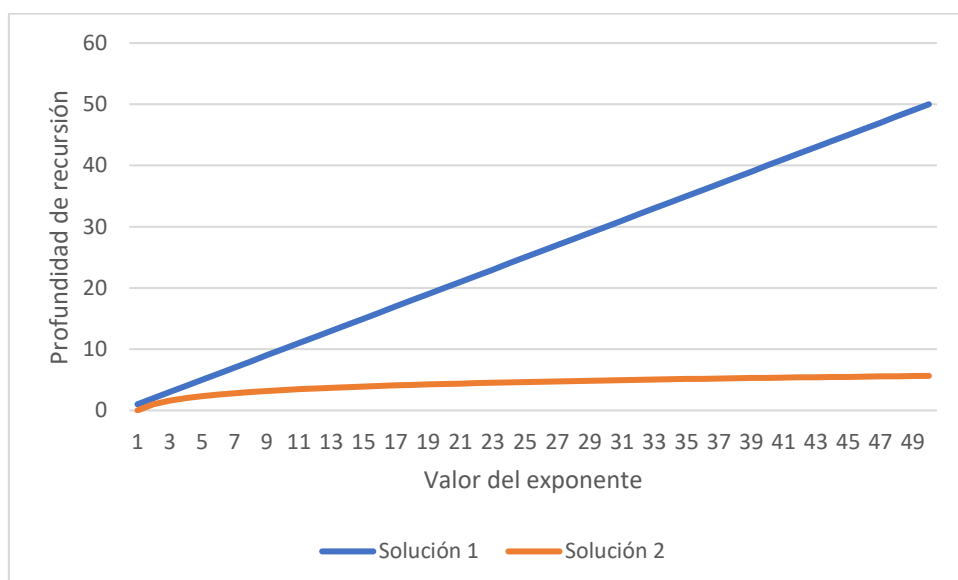


Figura 29

Eficiencia de la recursión

Además de la profundidad de recursión, la Figura 27 y la Figura 28 ponen de manifiesto otra diferencia entre las dos soluciones al problema del cálculo de la potencia: el número de instancias que llegan a ejecutarse en total. En la primera solución, este número es proporcional al valor del exponente, al igual que la profundidad de la recursión, mientras que, en la segunda, es exponencial con respecto al valor del exponente, ya que en cada nivel de recursión el número de llamadas se multiplica por 2:

$$n^{\circ} \text{ llamadas} = 2^0 + 2^1 + 2^2 + 2^3 \dots + 2^{\log_2 \text{exp}} = \sum_{n=0}^{\log_2 \text{exp}} 2^n$$

Para un exponente de valor 8, la primera solución realiza la llamada inicial más 8 llamadas recursivas, mientras que la segunda hace la llamada inicial más 14 llamadas recursivas. El número de llamadas realizadas, aunque no estén todas activas y consumiendo recursos

simultáneamente, influye directamente en el tiempo de ejecución necesario para completar la resolución del problema.

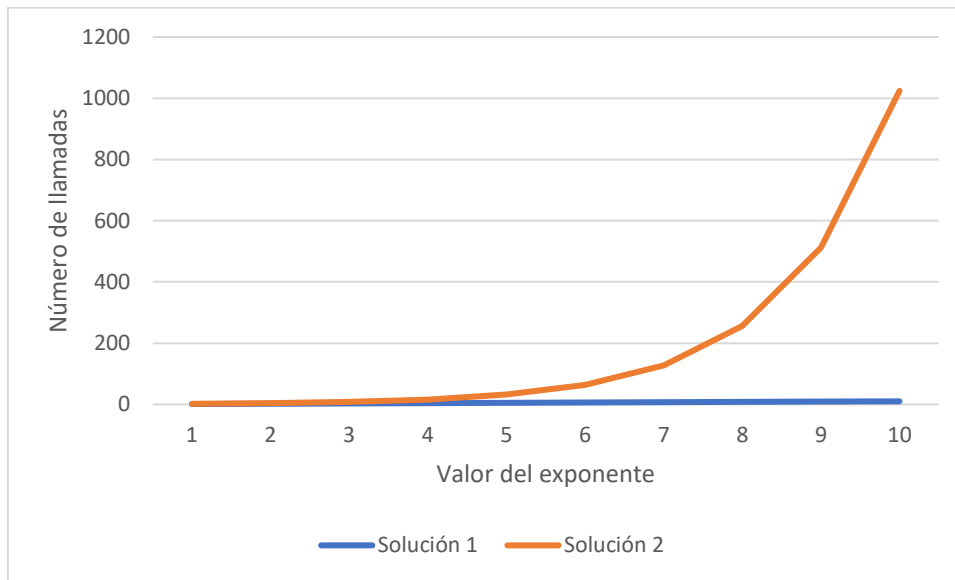


Figura 30

La Figura 28 pone en evidencia otro problema: no es solo que se realicen muchas llamadas, sino que la mayoría están resolviendo el mismo cálculo. Ocho llamadas calculan el caso base 2^1 , cuatro llamadas calculan 2^2 y dos llamadas calculan 2^4 . Esto ocurre porque se emplea la siguiente fórmula de recurrencia:

$$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a & \text{si } n = 1 \\ a^{\lfloor n/2 \rfloor} * a^{\lfloor n/2 \rfloor} & \text{si } n > 1 \end{cases}$$

Sin embargo, dado que:

$$a^{\lfloor n/2 \rfloor} = \begin{cases} a^{\lfloor n/2 \rfloor} & \text{si } n \text{ es par} \\ a^{\lfloor n/2 \rfloor} & \text{si } n \text{ es impar} \end{cases}$$

la relación de recurrencia podría escribirse:

$$a^n = \begin{cases} 1 & \text{si } n = 0 \\ a & \text{si } n = 1 \\ a^{\lfloor n/2 \rfloor} * a^{\lfloor n/2 \rfloor} & \text{si } n \text{ es par} \\ a^{\lfloor n/2 \rfloor} * a^{\lfloor n/2 \rfloor} * a & \text{si } n \text{ es impar} \end{cases}$$

Esto permitiría implementar la solución sin repetir el cálculo de $a^{\lfloor n/2 \rfloor}$, de la siguiente manera:

```
def pow(base, exp):
    if exp == 0:
        return 1
    elif exp == 1:
        return base
    else:
        result = pow(base, exp // 2)
        result *= result
        return result if exp % 2 == 0 else result * base
```

[ver ejecución >>>](#)

Con esta modificación, se obtiene una solución lineal con una profundidad de recursión de orden logarítmico, lo que reduce significativamente el consumo de recursos computacionales en comparación con la primera solución y disminuye considerablemente el tiempo de ejecución en relación con la segunda, como se observa en la Figura 31.

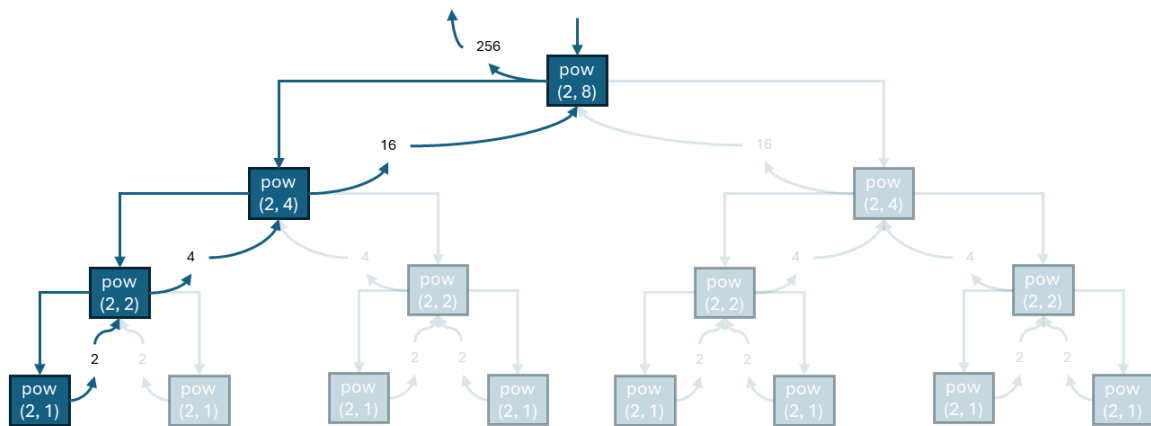


Figura 31 Recursión para la tercera solución

Estructuras de datos recursivas

Vista recursiva de una estructura de datos

Supóngase que se dispone de una lista ordenada de números y se desea saber si un número concreto se encuentra en ella. Al plantear una solución recursiva, se observa que el tamaño del problema viene determinado por el tamaño de la lista: cuanto más pequeña sea la lista, menos comparaciones serán necesarias en el peor caso. La lista más pequeña posible es la lista vacía. Ahora bien, ¿cómo se puede "visualizar" una lista como formada por listas más pequeñas?

Una posible forma es considerar que una lista no vacía está formada por un elemento seguido de una lista:

$[12, 13, 20, 23, 34, 45, 67, 89, 93] = 12 \rightarrow [13, 20, 23, 34, 45, 67, 89, 93]$

```
def find(lst, value):
    if len(lst) == 0:
        return False
    elif value == lst[0]:
        return True
    elif value > lst[0]:
        return find(lst[1:], value)
    else:
        return False
```

[ver ejecución >>>](#)

Esta solución es lineal y presenta una profundidad de recursión, en el peor caso, proporcional al tamaño de la lista. No obstante, al aprovechar el hecho de que la lista está ordenada, se introduce la posibilidad de finalizar la búsqueda antes de comparar con todos los elementos.

Otra forma de estructurar una lista es considerarla como dos sublistas unidas por un elemento central:

$[12, 13, 20, 23, 34, 45, 67, 89, 93] = [12, 13, 20, 23] \leftarrow 34 \rightarrow [45, 67, 89, 93]$

```
def find(lst, value):
    if len(lst) == 0:
        return False
    else:
        mid = len(lst) // 2

        if lst[mid] == value:
            return True
        elif lst[mid] > value:
            return find(lst[0:mid], value)
        else:
            return find(lst[mid + 1:], value)
```

[ver ejecución >>>](#)

Esta segunda solución, conocida como **búsqueda dicotómica**, es lineal con profundidad logarítmica, ya que cada llamada recursiva divide el espacio de búsqueda en dos y continúa buscando en la sublista donde se pueda encontrar el valor, dado que la lista está ordenada.

Cualquier secuencia se puede ver de alguna de estas maneras para encontrar soluciones recursivas. La más habitual consiste en considerar que una secuencia no vacía se compone de un elemento seguido de otra secuencia. Este enfoque ha sido tan utilizado que se ha convertido en el paradigma de diseño de lenguajes funcionales como LISP (por **LISt Processing**).

Estructuras de datos construidas recursivamente

Aparte de "ver" estructuras de datos secuenciales de manera recursiva, como en el apartado anterior, es posible construir estructuras de datos que sean explícitamente recursivas.

Por ejemplo, inspirándose en la definición de una lista no vacía como formada por un elemento seguido de una lista, se puede crear una estructura utilizando tuplas:

```
empty = () # una lista vacía se representa con una tupla vacía
lklst = (23, empty) # (23, ())
lklst = (12, lklst) # (12, (23, ()))
lklst = (42, lklst) # (42, (12, (23, ())))
lklst = (16, lklst) # (16, (42, (12, (23, ())))))
```

[ver ejecución >>>](#)

Este tipo de estructura se conoce como **lista encadenada**, ya que cada tupla actúa como un eslabón (nodo) que contiene un elemento y una referencia que enlaza con el siguiente nodo/eslabón, formando así una cadena.

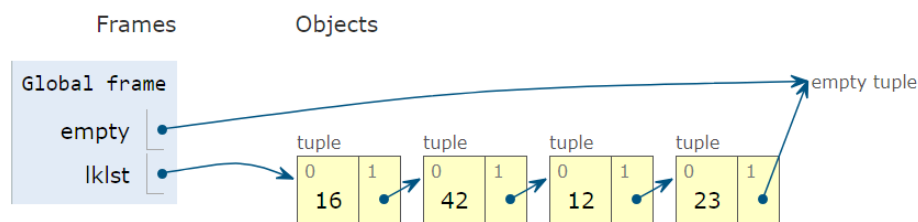


Figura 32 Lista encadenada

En este esquema, la variable que referencia a la lista, referencia, en realidad al primer nodo de esta. Dicho nodo, a su vez, se compone de un valor y una referencia al primer nodo de una lista más pequeña que le sigue. Esto evidencia el carácter recursivo de la. Como tal, es ideal para soluciones recursivas:

```
def find(lst, value):
    if lst == ():
        return False
    else:
        node_value, next = lst

        if value == node_value:
            return True
        else:
            return find(next, value)
```

[ver ejecución >>>](#)

Sin embargo, esta estructura presenta una desventaja: no permite el acceso directo a un elemento por su posición. Su naturaleza es estrictamente secuencial, lo que implica que para acceder a un nodo determinado es necesario pasar por todos los anteriores.

Este comportamiento se ilustra en la siguiente función iterativa, que obtiene el elemento de la lista *lst* que ocupa la posición indicada por *pos*:

```
def get(lst, pos):
    current = lst
    idx = 0

    while current != () and idx < pos:
        current = current[1]
        idx += 1

    return None if current == () else current[0]
```

[ver ejecución >>>](#)

Nótese que emplea un recorrido basado en una condición, ya que no se dispone de un rango de índices.

Inmersión recursiva

Retomando el algoritmo de la búsqueda dicotómica visto anteriormente:

```
def find(lst, value):
    if len(lst) == 0:
        return False
    else:
        mid = len(lst) // 2

        if lst[mid] == value:
            return True
        elif lst[mid] > value:
            return find(lst[0:mid], value)
        else:
            return find(lst[mid + 1:], value)
```

[ver ejecución >>>](#)

Lo que no se consideró anteriormente es que se usaron *slices* (`lst[0:mid]`, `lst[mid + 1:]`), y, en Python, cada *slice* supone una copia superficial de la lista. Para eviatarlo, se puede redefinir el problema a resolver, *"Determinar si un valor está contenido en una lista"*, como *"Determinar si un elemento está contenido en una zona de una lista especificada por una pareja de índices"*:

```
def find_range(lst, value, inf, sup):
    if inf >= sup:
        return False
    else:
        mid = inf + (sup - inf) // 2

        if lst[mid] == value:
            return True
        elif lst[mid] > value:
            return find_range(lst, value, inf, mid)
        else:
            return find_range(lst, value, mid+1, sup)
```

[ver ejecución >>>](#)

Para mantener la interfaz original, se puede redefinir la función *find* para que simplemente llame a la función *find_range*, tomando como rango de búsqueda inicial toda la lista:

```
def find(lst, value):
    return find_range(lst, value, 0, len(lst))
```

[ver ejecución >>>](#)

Esta estrategia se conoce como resolver el problema por inmersión: cuando se enfrenta un problema que es difícil/costoso/ineficiente de resolver usando recursividad, se puede buscar un problema más general, que sea más apropiado para una solución recursiva y que contenga al problema original como un caso particular. Es decir, el problema original está inmerso en el conjunto de variantes del problema más general, de ahí el concepto "resolución por inmersión".

En este caso, el problema general es *"Determinar si un elemento está contenido en una zona de una lista especificada por una pareja de índices"*, y el problema original, *"Determinar si un valor está contenido en una lista"*, está inmerso en él como una variante particular: *"Determinar si un elemento está contenido en **la** zona de una lista especificada por **la** pareja de índices (0, len(lst))"*.

Tema 6 Contenedores asociativos

Un **contenedor asociativo** es un tipo de estructura de datos en informática que organiza y almacena elementos de manera que se pueda acceder rápidamente a ellos a través de una clave asociada, en lugar de un índice posicional como en los arrays o listas.

Diccionarios

Los diccionarios, conocidos en algunos lenguajes como arrays asociativos, son colecciones de datos similares a las secuencias, pero, a diferencia de estas, no se indexan usando un rango de números (0, 1, 2, ...), sino que usan una clave, la cual puede ser de cualquier tipo inmutable (típicamente, strings o números). Un diccionario (tipo *dict*) es una colección no ordenada de parejas clave-valor.

Un diccionario inicialmente vacío se crea con claves vacías:

```
d = {}  
print(type(d))  
<class 'dict'>  
print(len(d))  
0
```

También se puede crear un diccionario vacío utilizando el constructor *dict()* sin argumentos:

```
d = dict()
```

Un diccionario de Python se puede crear inicializándolo con uno o más elementos en la forma *clave:valor*, separados por comas. El siguiente es un (mini) diccionario inglés-español, donde tanto las claves como los valores son *strings*, las primeras representan palabras en inglés y los segundos, su traducción al español:

```
d = {  
    'mother': 'madre', 'father': 'padre',  
    'house': 'casa', 'car': 'coche'  
}
```

Al constructor *dict()* se le puede pasar una secuencia de pares clave:valor. El siguiente ejemplo inicializa un (mini) diccionario inglés-eslovaco:

```
my_list = [  
    ('mother', 'matka'), ('father', 'otec'),  
    ('house', 'dom'), ('car', 'auto')  
]  
d = dict(my_list)
```

También se puede usar *comprehension (dictionary comprehension)* para crear diccionarios:

```
d = {x: x**2 for x in (2, 4, 6)}
```

Cuando las claves son *strings*, se pueden usar como "parámetros por nombre" en *dict()*:

```
d = dict(Michael=77, Peter=66, Patrick=82)
```

Operaciones en diccionarios

Las principales operaciones que se pueden realizar en un diccionario son:

- Almacenar un valor asociado a una clave.
- Obtener el valor asociado a una clave.
- Borrar un par clave:valor.

```
d = {'mother': 'madre', 'father': 'padre'}  
d['child'] = 'hijo'
```

En el ejemplo anterior, la instrucción `d['child'] = 'hijo'` añade un nuevo par clave:valor al diccionario *d*. Si se usa una clave ya existente, el nuevo valor sustituye al almacenado previamente:

```
d['child'] = 'hija'
```

Para acceder a un valor, se usa su clave:

```
print(d['mother'])  
madre
```

Intentar acceder al diccionario con una clave que no existe genera un error:

```
v = d['dog']  
Traceback (most recent call last): File "<input>", line 7, in <module>  
KeyError: 'dog'
```

Para prevenir este error, se puede utilizar el operador *in*, que permite saber si una clave existe:

```
if 'dog' in d:  
    print(d['dog'])  
else:  
    print('La clave "dog" no existe en el diccionario')
```

Eliminar una pareja clave:valor de un diccionario

A veces, es necesario eliminar una pareja clave:valor de un diccionario. Esto puede hacerse con el método *pop()*.

```
d = {'mother': 'madre', 'father': 'padre', 'child': 'hijo'}  
value = d.pop('mother')  
print(d)  
{'father': 'padre', 'child': 'hijo'}
```

El método `pop()` elimina el par clave:valor asociado a la clave pasada como parámetro y devuelve el correspondiente valor. Si no se necesita conservar el valor, puede llamarse al método `pop()` sin asignar su resultado a ninguna variable:

```
d.pop('mother')
```

También se puede usar la instrucción `del` con diccionarios (obsérvese la sintaxis):

```
d = {'mother': 'madre', 'father': 'padre', 'child': 'hijo'}
del d['mother']
print(d)
{'father': 'padre', 'child': 'hijo'}
```

En ambos casos, se produce un error (`KeyError`) si la clave en cuestión no existe en el diccionario, por lo que puede ser conveniente usar previamente el operador `in` para asegurarse.

Para vaciar un diccionario se usa el método `clear()`:

```
d = {'mother': 'madre', 'father': 'padre', 'child': 'hijo'}
d.clear()
print(d)
{}
```

El método `get()`

A veces, es más conveniente utilizar el método `get()` para recuperar el valor asociado a una clave:

```
capitals = {'Slovakia': 'Bratislava', 'Austria': 'Vienna'}
print(capitals['Slovakia'])
Bratislava
print(capitals.get('Slovakia'))
Bratislava
```

Si la clave no está en el diccionario, el comportamiento varía dependiendo del método usado:

```
capitals = {'Slovakia': 'Bratislava', 'Austria': 'Vienna'}
print(capitals.get('Hungary'))
None
print(capitals['Hungary'])
Traceback (most recent call last):
  File "my_program.py", line 2, in <module>
    print(capitals['Hungary'])
KeyError: 'Hungary'
```

El método `get()` no genera un error cuando la clave no se encuentra en el diccionario; en su lugar, se devuelve el valor `None`, a menos que se especifique un valor por defecto como segundo parámetro en la llamada:

```
print(capitals.get('Hungary'))
print(capitals.get('Hungary', '???'))
None
???
```

Diccionarios como valores de diccionarios

Un diccionario puede almacenarse como valor dentro de otro diccionario:

```
s1 = {
    'name': 'Paul Carrot',
    'address': {'Street': 'Long', 'Number': 13, 'City': 'Nitra'}
}
s2 = {
    'name': 'Peter Pier',
    'address': {'Street': 'Short', 'Number': 21, 'City': 'Nitra'}
}
s3 = {
    'name': 'Patrick Nut',
    'address': {'Street': 'Deep', 'Number': 77, 'City': 'Nitra'}
}
students = [s1, s2, s3]
```

En el ejemplo anterior, se crea una lista de tres diccionarios. El elemento `students[0]` es un diccionario con dos parejas *clave:valor*:

```
print(student[0]['name']) # Paul Carrot
print(student[0]['address'])
{'Street': 'Long', 'Number': 13, 'City': 'Nitra'}
```

El valor asociado a la clave *'address'* es, a su vez, un diccionario:

```
print(student[0]['address']['Street'])
print(student[0]['address']['Number'])
print(student[0]['address']['City'])
Long
13
Nitra
```

Conjuntos

El tipo *set* de Python se usa para representar colecciones de elementos sin orden y sin repetición; básicamente, implementa el concepto matemático de conjunto. Un conjunto vacío se crea usando la función constructora *set()*:

```
vacío = set()
```

Un conjunto no vacío se puede crear escribiendo sus elementos encerrados entre llaves, separados por coma:

```
elementos = {"agua", "aire", "fuego", "tierra"}
print(elementos)
{'agua', 'tierra', 'aire', 'fuego'}
```

>>> *Nótese que el orden de los elementos no está definido y que, por tanto, su visualización no tiene por qué coincidir con el orden en que se escribieron al crear el conjunto.*

Los elementos no pueden ser de un tipo mutable. En el siguiente ejemplo, el tercer elemento es una lista, lo que supone un error:

```
elementos = {"agua", "aire", ["fuego", "tierra"]}
```

```
Traceback (most recent call last):
  File "/home/p11049/p1.py", line 1, in <module>
    elementos = {"agua", "aire", ["fuego", "tierra"]}
TypeError: unhashable type: 'list'
```

Otra forma de crear un conjunto es pasando una colección de elementos (tupla, lista, string, diccionario u otro conjunto) a la función constructora:

```
elementos = set(["agua", "aire", "fuego", "tierra"])
print(elementos)
caracteres = set("esto es una string")
print(caracteres)
{'agua', 'fuego', 'aire', 'tierra'}
{'a', 's', 'r', 'e', 'g', ' ', 't', 'o', 'i', 'n', 'u'}
```

>>> *Nótese que en el conjunto caracteres no hay elementos repetidos, a pesar de que sí los había en la string que se pasó como parámetro para crearlo.*

En caso de que se pase un diccionario, se crea un conjunto con las claves de este (no con las parejas clave:valor). Si se pasa un conjunto, se crea una copia de este.

Otra forma de copiar un conjunto es usando el método `.copy()`:

```
elementos = set(["agua", "aire", "fuego", "tierra"])
copia = elementos.copy()
```

La cardinalidad (número de elementos de un conjunto) puede averiguarse usando la función `len()`, como en cualquier otra colección de Python:

```
elementos = set(["agua", "aire", "fuego", "tierra"])
print(len(elementos))
4
```

Modificación de conjuntos

Los conjuntos de Python son mutables. A un conjunto existente se le pueden añadir elementos individuales usando la operación `.add()`:

```
elementos = {"agua", "aire", "fuego", "tierra"}
elementos.add("eter")
print(elementos)
{'tierra', 'eter', 'agua', 'aire', 'fuego'}
```

También se le pueden añadir elementos de una colección usando la operación `.update()`:

```
elementos = {"agua", "aire"}
elementos.update(["fuego", "tierra"])
print(elementos)
{'fuego', 'aire', 'tierra', 'agua'}
```

Por supuesto, ni `.add()` ni `.update()` añaden elementos repetidos.

Para eliminar un elemento concreto, se pueden usar las operaciones `.remove()` o `.discard()`:

```
elementos = {'fuego', 'aire', 'tierra', 'agua'}
elementos.remove("agua")
print(elementos)
{'tierra', 'fuego', 'aire'}
```

```
elementos.discard("aire")
print(elementos)
{'tierra', 'fuego'}
```

La diferencia entre `.remove()` y `.discard()` es que el primero produce un error si el elemento a eliminar no está en el conjunto, mientras que el segundo lo ignora y no hace nada:

```
elementos.remove("eter")
```

```
Traceback (most recent call last):
  File "p1.py", line 6, in <module>
    elementos.remove("eter")
KeyError: 'eter'
```

El método `.pop()` elimina un elemento al azar:

```
elementos = {'fuego', 'aire', 'tierra', 'agua'}
elementos.pop()
print(elementos)
{'agua', 'tierra', 'fuego'}
```

El método `.clear()` vacía el conjunto en cuestión, eliminando todos sus elementos:

```
elementos = {'fuego', 'aire', 'tierra', 'agua'}
elementos.clear()
print(elementos)
set()
```

Obsérvese la especial visualización de un conjunto vacío. Nótese que `{}` representaría un diccionario vacío.

Pertenencia e inclusión

Para comprobar si un valor pertenece a un conjunto (es uno de sus elementos) se usa el operador *in*:

```
elementos = {'fuego', 'aire', 'tierra', 'agua'}  
print('aire' in elementos)
```

```
True
```

```
print('eter' in elementos)
```

```
False
```

Los operadores relacionales $<$, $<=$, $>$, $>=$ permiten saber si un conjunto está incluido en otro (es un subconjunto del otro):

```
alfabeto = set("abcdefghijklmnopqrstuvwxyz")  
vocales = set("aeiou")  
print(vocales < alfabeto)
```

```
True
```

Los operadores $<$ y $>$ requieren que sea un subconjunto estricto (se corresponden con las relaciones matemáticas $\subset$ y $\supset$), mientras que $<=$ y $>=$ (que se corresponden con $\subseteq$ y $\supseteq$) admiten también los "subconjuntos" que son iguales al conjunto que los "contiene" (subconjuntos impropios).

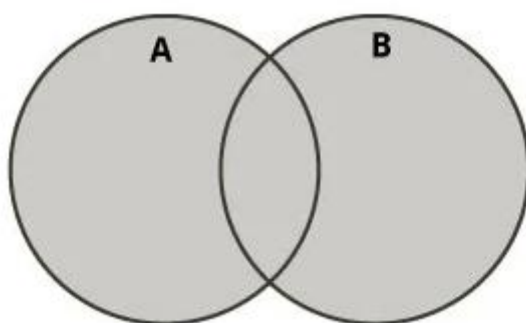
Entre conjuntos también se pueden usar los operadores relacionales $=$ y $\neq$, que determinan si dos conjuntos son iguales o distintos. Dos conjuntos son iguales si tienen exactamente los mismos elementos (recuérdese que los elementos no tienen orden).

Álgebra de conjuntos

Los conjuntos disponen de cuatro operaciones básicas: unión, intersección, diferencia y diferencia simétrica.

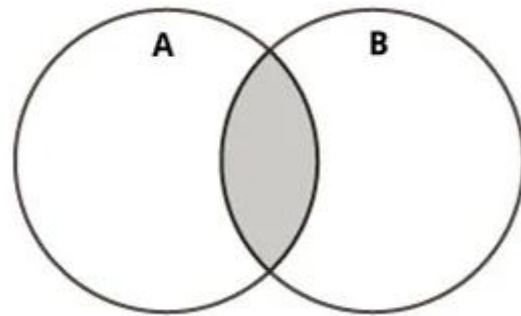
La **unión** de dos conjuntos resulta en un conjunto que tiene todos los elementos que estén en alguno(s) de los conjuntos originales, sin repetición. En Python, la unión se hace con el operador $|$ (barra vertical; para escribirla, normalmente se compone con la tecla del 1):

```
set1 = {1, 2, 3}  
set2 = {3, 4, 5}  
unión = set1 | set2  
print(unión)  
{1, 2, 3, 4, 5}
```



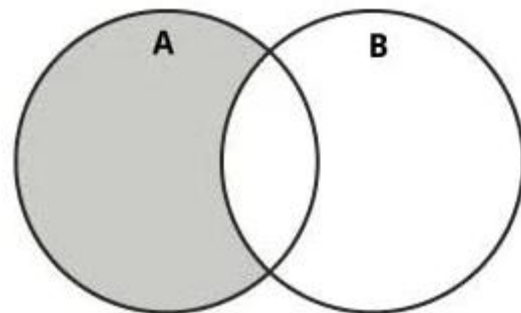
La **intersección** contiene únicamente los elementos comunes a ambos conjuntos y se hace con el operador &:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
intersección = set1 & set2
print(intersección)
{3}
```



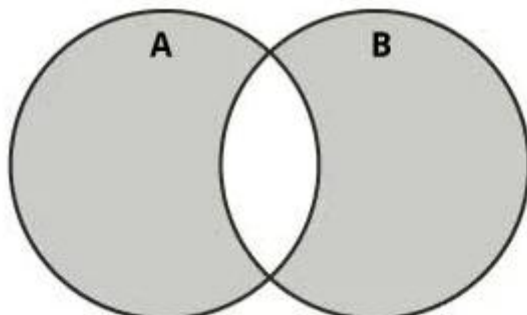
Para la **diferencia** se utiliza el operador -, cuyo resultado es el conjunto formado por todos los elementos del primer conjunto que no están en el segundo:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
diferencia = set1 - set2
print(diferencia)
{1, 2}
```



La **diferencia simétrica** comprende todos los elementos del primer conjunto que no están en el segundo y los del segundo que no están en el primero (los elementos no comunes), y se realiza con el operador ^:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
diferencia = set1 ^ set2
print(diferencia)
{1, 2, 4, 5}
```



Iterando en contenedores asociativos

Aparte de obtener un valor usando una clave, es posible obtener todas las claves, todos los valores, o todos los pares *clave:valor* de un diccionario:

```
dct = {'A': 10, 'B': 9, 'C': 9, 'D': 9, 'E': 4, 'FX': 2}
print(dct.keys())
print(dct.values())
print(dct.items())
```

```
dict_keys(['A', 'B', 'C', 'D', 'E', 'FX'])
dict_values([10, 9, 9, 9, 4, 2])
dict_items([('A', 10), ('B', 9), ('C', 9), ('D', 9), ('E', 4), ('FX', 2)])
```

Los cuales son todos ellos objetos iterables:

```
for key in dct.keys():
    print(key, end=' ')
```

```
A B C D E F X
```

```
for value in dct.values():
    print(value, end=' ')
```

```
10 9 9 9 4 2
```

```
for key, value in dct.items():
    print(key, value)
```

```
A 10
B 9
C 9
D 9
E 4
FX 2
```

Si no se especifica nada, el recorrido es por claves:

```
for key in dct:
    print(key, dct[key])
```

```
A 10
B 9
C 9
D 9
E 4
FX 2
```

Esto también se puede aplicar para recorrer un conjunto:

```
elementos = {'fuego', 'aire', 'tierra', 'agua'}
```

```
for item in elementos:
    print(item)
```

```
tierra
agua
aire
fuego
```

[ver ejecución del código de esta sección >>>](#)

Los objetos resultantes de llamar a los métodos *keys()*, *values()* o *items()* pueden convertirse a su vez en listas. Por ejemplo, la expresión:

```
list(d.keys())
```

devuelve una lista con todas las claves del diccionario `d`, en un orden arbitrario (recuérdese que un diccionario es una colección no ordenada de parejas clave:valor). Si se necesita tener las claves ordenadas, se debe usar en su lugar la expresión:

```
sorted(d.keys())
```

Tratamiento de contenedores asociativos

El punto fuerte de los contenedores asociativos es que, dado un valor de clave es posible determinar si está en el contenedor o, en el caso de los diccionarios, recuperar su valor asociado de forma muy eficiente sin tener que efectuar ningún proceso de búsqueda del tipo que se aplicaría en una secuencia. Por lo tanto, solo se deben aplicar los esquemas de iteración sobre contenedores asociativos cuando se enfrenten a un tipo de problema en el que no se tenga ningún valor de clave y se requiera realizar algún tratamiento con aquellos valores que cumplan determinada condición, si los hubiera. Esto se puede ilustrar con un ejemplo.

Supóngase una estructura de datos como la siguiente:

```
academic_data = (
    {
        "40951": ("Álgebra Y Geometría", 1, 6),
        "40953": ("Fundamentos De Programación I", 1, 6),
        "40957": ("Física", 2, 6),
        "40961": ("Métodos estadísticos", 3, 6),
        "40964": ("Periféricos E Interfaces Obligatoria", 3, 6),
        "40968": ("Bases De Datos I", 4, 6),
        "40973": ("Producción De Software", 6, 9),
        "40990": ("Prácticas Externas", 7, 12),
        "40991": ("Trabajo Fin De Grado", 8, 12),
    },
    {
        "40444444X": ("Pepito Grillo", ["40951", "40961"]),
        "40555555Y": ("María López", ["40968", "40957"]),
        "41444444Z": (
            "Juan García",
            ["40973", "40964", "40964"]
        ),
        "40555566A": ("Ana Stasia", []),
        "4035553B": ("Fer Nando", ["40991", "40973"]),
    }
)
```

Se trata de una tupla que contiene dos diccionarios. El primero tiene como claves códigos de identificación de asignaturas y como valores tuplas con información sobre las asignaturas: nombre, semestre y créditos. El segundo tiene como claves DNIs de estudiantes y como valores tuplas con el nombre del estudiante y la lista de asignaturas en las que está matriculado.

La siguiente función, dado el código de una asignatura, devuelve una tupla con dos elementos: el nombre de la asignatura y una lista de nombres de los estudiantes matriculados en ella. Si no hay ninguna asignatura con el código pasado, devuelve el propio código (al no tener un nombre) y una lista vacía.

```

def get_enrolled(data, subject_code):
    subjects, students = data
    subject_name = get_subject_name(subjects, subject_code)
    enrolled_list = []

    if subject_name != subject_code:
        enrolled_list = get_enrolled_list(students, subject_code)

    return subject_name, enrolled_list

def get_subject_name(subjects, subject_code):
    subject_name = subjects.get(subject_code, (subject_code,))[0]
    return subject_name

def get_enrolled_list(students, subject_code):
    names = []

    for dni in students:
        name, subjects = students[dni]

        if subject_code in subjects:
            names.append(name)

    return names

```

[ver ejecución >>>](#)

El problema se divide en dos subproblemas: encontrar el nombre de la asignatura en el primer diccionario y, si esto tiene éxito, encontrar los nombres de los estudiantes matriculados en ella, que están en el segundo diccionario.

Para resolver el primer problema, se tiene el código de la asignatura y se intenta obtener la información asociada a la misma mediante una consulta directa con la operación `.get()`. **Sería absurdo aplicar un esquema de búsqueda para iterar en el diccionario buscando el código.**

Se usa la operación `.get()` porque no se sabe si en el diccionario hay una asignatura con el código pasado como parámetro. Si se tuviera certeza de que existe, se podría cambiar la sentencia:

```
subject_name = subjects.get(subject_code, (subject_code,))[0]
```

Por:

```
subject_name = subjects[subject_code][0]
```

La operación `.get()` está configurada para que, en caso de no encontrar el código de la asignatura, devuelva una tupla con el código de asignatura, permitiendo devolverlo como si fuera el nombre sin necesidad de más preguntas.

Para resolver el segundo problema no se tiene ninguna clave. Los estudiantes matriculados en la asignatura serán aquellos que tengan en su lista de asignaturas matriculadas el código de la asignatura que se busca. En consecuencia, se recorre la lista de claves del diccionario de estudiantes aplicando un esquema acumulador selectivo para añadir al resultado los nombres de los estudiantes que cumplan la condición de tener en su lista de asignaturas matriculadas el código de la asignatura que se busca. Nótese que el operador *in* que se aplica a esta lista implica un esquema de búsqueda lineal, mientras que si, en vez de una lista, se utilizara un *set*, sería una operación de consulta directa.

Procesamiento de textos

Tema 7 Strings de caracteres

Operaciones de strings

Strings

Las *strings* representan secuencias contiguas de caracteres. Los valores literales de tipo *string* (*str*), se encierran entre comillas dobles o simples:

```
var1 = "hola"
var2 = 'hola'
str_var = "Hello world"
print(str_var)
Hello world
```

Se puede crear una string multilínea usando triples comillas (simples o dobles). Los caracteres de salto de línea serán parte de la string:

```
var3 = '''Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.'''
```

La longitud de una string (número de caracteres) se obtiene usando la función *len()*:

```
str_var = "Hello world"
print(len(str_var))
11
```

Secuencias de escape

Se pueden introducir caracteres especiales en las strings (tabuladores, saltos de línea, caracteres por código, ...) usando secuencias de escape, que son secuencias de caracteres que empiezan con el carácter `\` (llamado barra invertida o backslash).

Secuencia de escape	Ejemplo	Salida
<code>\'</code>	comilla simple <code>print('Hello \'world!\\')</code>	Hello 'world!'
<code>\"</code>	comilla doble <code>print("Hello \"world!\\")</code>	Hello "world!"
<code>\\</code>	backslash <code>print("Hello \\World!\\")</code>	Hello \world!\
<code>\a</code>	sonido de campana <code>print("Hello World!\a")</code>	Hello world!
<code>\b</code>	backspace, retroceso <code>print("Hello\b World")</code>	Hell world!
<code>\v</code>	vertical tab <code>print("Hello\v World!")</code>	Hello world!
<code>\xhh</code>	carácter por valor hexadecimal <code>print("\x40Hello World!")</code>	@Hello world!

Secuencia de escape	Ejemplo	Salida
<code>\n</code>	nueva línea <code>print("Hello\n World!")</code>	Hello world!
<code>\N{name}</code>	carácter Unicode por nombre <code>print("Hello\N{BABY ANGEL} World!")</code>	Hello 🧚 world!
<code>\ooo</code>	carácter por valor octal <code>print("\100Hello World!")</code>	@Hello world!
<code>\r</code>	retorno de carro (sobrescribe desde el principio la línea actual) <code>print("Hello\r World!")</code>	world!
<code>\uxxxx</code>	carácter Unicode por valor hexadecimal (16 bits) <code>print("Hello \u041b World!")</code>	Hello П world!
<code>\Uxxxxxxxx</code>	carácter Unicode por valor hexadecimal (32 bits) <code>print("Hello \U000001a9 World!")</code>	Hello Σ world!

Mayúsculas y minúsculas

Existen métodos para saber si las letras que contiene una string son mayúsculas o minúsculas y para hacer cambios de un caso al otro. El método *isupper()* devuelve *True* si las letras de una string son mayúsculas; el método *islower()* hace lo propio si son minúsculas.

```
test_string = "en un lugar de La Mancha de cuyo nombre"
a = test_string.isupper()      # False
```

El método *upper()* devuelve una nueva string, como la original, pero con sus letras transformadas a mayúsculas. Lo contrario hacen los métodos *lower()* y *casefold()* (ambos transforman a minúsculas).

```
b = test_string.upper()
# "EN UN LUGAR DE LA MANCHA DE CUYO NOMBRE"
```

El método *capitalize()* devuelve una nueva string, como la original, pero poniendo en mayúscula la letra inicial y en minúscula el resto.

```
c = test_string.capitalize()
# "En un lugar de la mancha de cuyo nombre"
```

El método *title()* devuelve una nueva string igual que la original, pero poniendo en mayúscula la letra inicial de cada palabra y en minúscula el resto. Existe un método llamado *istitle()* para verificar si una string sigue este formato.

```
d = test_string.title()
# "En Un Lugar De La Mancha De Cuyo Nombre"
```

El método `swapcase()` devuelve una nueva string invirtiendo mayúsculas y minúsculas.

```
f = d.swapcase()  
# "eN uN lUGAR dE lA mANCHA dE cUYO nOMBRE"
```

Categorías de caracteres

Existe una serie de métodos para conocer qué tipo de caracteres forman una string.

Método	Descripción
<code>isalnum()</code>	Devuelve <i>True</i> si todos los caracteres son alfanuméricos.
<code>isalpha()</code>	Devuelve <i>True</i> si todos los caracteres están en el alfabeto.
<code>isdecimal()</code>	Devuelve <i>True</i> si todos los caracteres son dígitos decimales.
<code>isdigit()</code>	Devuelve <i>True</i> si todos los caracteres son dígitos.
<code>isidentifier()</code>	Devuelve <i>True</i> si la string es un identificador. Una string se considera un identificador válido si solo contiene letras alfanuméricas (a-z, A-Z y 0-9), o guiones bajos (_). Un identificador válido no puede comenzar con un número o contener espacios.
<code>isnumeric()</code>	Devuelve <i>True</i> si todos los caracteres son numéricos.
<code>isprintable()</code>	Devuelve <i>True</i> si todos los caracteres son imprimibles.
<code>isspace()</code>	Devuelve <i>True</i> si todos los caracteres son separadores (espacios, tabuladores, saltos de línea, ...).

Acceso a los componentes de una string

Se puede acceder a un carácter concreto de una string escribiendo la string, o el nombre de la variable que la referencia, seguida del índice de la posición que ocupa el carácter deseado entre corchetes. El índice de la primera posición es 0.

```
str_var = "Hello world"  
print(str_var[0]) # Muestra (sin comillas) el primer carácter de  
str_var: 'H'  
print(str_var[4]) # Muestra (sin comillas) el quinto carácter de  
str_var: 'o'
```

También se pueden usar índices negativos para acceder desde el final de la string:

```
str_var = "Hello world"  
print(str_var[-1])  
print(str_var[-5])
```

```
d  
w
```

Se puede obtener una substring (subsecuencia) de una string usando el operador de segmento, o slice (`[:]`). El segmento deseado se indica mediante el índice del primer elemento y el índice de la posición siguiente al último elemento. Si se omite el segundo valor, se toma el segmento desde el primer índice hasta el final; si se omite el primero, toma desde el principio hasta la posición del índice anterior al indicado.

```
str_var = "Hello world"
print(str_var[2:5])
print(str_var[2:])
print(str_var[:5])
```

```
llo
llo world
Hello
```

Usando un paso de -1 se puede obtener la string invertida:

```
str_var = "Hello world"
print(str_var[::-1])
```

```
dlrow olleH
```

Concatenación de strings

El diccionario de la RAE define **concatenar** como "unir dos o más cosas". En el caso de las strings, concatenar dos strings consiste en formar una nueva string juntando, en orden, dos preexistentes.

En Python existen dos operadores de concatenación: el signo más (+) es el operador de concatenación de strings y el asterisco (*) es el operador de repetición (equivalente a realizar una concatenación repetidas veces).

```
str_var = "Hello world"
print('a' + 'a')           # Muestra "aa"
print(str_var + "TEST")    # Muestra el valor de str_var concatenado
                           # con "TEST": "Hello worldTEST"
print('ab' * 5)            # Muestra "ababababab"
print(str_var * 2)         # Muestra "Hello worldHello world"
```

Se podría decir que la repetición de strings es a la concatenación de strings lo que la multiplicación de números enteros es a la suma de números enteros. Nótese que la concatenación junta directamente las strings, sin añadir ningún carácter en medio.

Localización de una substring

El método *find()* permite localizar una substring dentro de una string:

```
s1 = "Esto es una string de prueba"
print(s1.find("una"))    # 8
```

El método *find()* devuelve el índice del primer carácter de la primera aparición de la substring buscada ("una" en el ejemplo) en la string sobre la que se aplica (la referenciada por *s1* en el ejemplo).

```
s1 = "Esto es una string, es una secuencia, pero no una lista"
p = s1.find("una")    # 8
```

En caso de que la substring a buscar no se encuentre en la string donde se busca, el método *find()* devuelve el valor -1.

También se puede usar el método *index()*, aplicable a cualquier tipo de secuencia (strings, listas, tuplas, ...).

```
s1 = "Esto es una string de prueba"
print(s1.index("una")) # 8
```

La diferencia es que, si la string buscada no se encuentra, el método *index()* produce un error. Una forma de evitarlo es usar el operador *in* para comprobar primero si la substring buscada está presente:

```
s1 = "Esto es una string de prueba"
if "una" in s1:
    print(s1.index("una"))
```

Si solo nos interesa saber si la substring buscada está presente en la string donde se busca, pero no se necesita conocer su índice, basta con el operador *in*:

```
print("una" in s1) # True
```

El método *find()* admite dos parámetros adicionales opcionales:

- *start*: índice de la posición donde comenzar la búsqueda (el valor por omisión es 0).
- *end*: índice de la posición donde dejar de buscar (el valor por omisión es la longitud de la string donde se busca).

Por ejemplo:

```
s1 = "Esto es una string de prueba"
print(s1.find("e ", 10, 24)) # 20
```

Otros métodos de localización

Los métodos *rfind()* y *rindex()* localizan la última ocurrencia de la substring buscada, a diferencia de sus contrarios, *find()* e *index()*, que localizan la primera aparición (la *r* indica que buscan desde la derecha).

Los métodos *startswith()* y *endswith()* devuelven *True* si la substring pasada como parámetro coincide con el comienzo o con el final, respectivamente, de la string donde se busca, y *False*, en caso de que no coincida.

El método *count()* devuelve el número de apariciones de la substring buscada en la string de búsqueda.

El método *partition()* devuelve una tupla con las tres partes en que se puede considerar dividida la string en que busca: la substring anterior, la primera ocurrencia de la substring encontrada y la substring posterior; si no se encuentra, las tres partes serán la propia substring buscada y dos strings vacías. El método *rpartition()* hace lo mismo desde el final, con la última ocurrencia de la substring buscada.

El método join()

El método *join()* devuelve una string concatenando las sucesivas strings contenidas en la secuencia pasada como parámetro, usando como separador la string con la que se llama al método.

Ejemplo:

```
words = ['sun', 'is', 'shining']
espacio = ' ' # un espacio
message = espacio.join(words)
```

```
print(message)
```

```
sun is shining
```

```
import random
numbers = [str(random.randint(1, 9)) for i in range(5)]
print(numbers) # p.e. ['9', '1', '9', '3', '2']
exercise = ' + '.join(numbers)
print(exercise, '= ') # "9 + 1 + 9 + 3 + 2 = " (sin comillas)
```

En los ejemplos anteriores, las secuencias pasadas al método *join()* son listas, pero pueden ser cualquier clase de secuencia, por ejemplo, otra string, en cuyo caso, el separador se intercalaría entre los caracteres de esta:

```
s1 = "ABC"
print("-".join(s1))
```

```
A-B-C
```

El método split()

El método *split()* crea una lista cuyos elementos son sucesivos trozos de la string sobre la que se aplica. Si se le llama sin parámetros, los elementos de la lista son los trozos que en la string original están delimitados por espaciadores (*whitespaces*), esto es, cualesquiera separadores estándar de texto, tales como espacios en blanco o/y tabuladores:

```
a = 'esto es un ejemplo'
b = a.split() # ['esto', 'es', 'un', 'ejemplo']
```

Se puede especificar un separador entre los trozos (se consideraría que existe un trozo string vacía entre dos separadores seguidos). En el siguiente ejemplo es la coma:

```
a = 'esto es, un ejemplo, de split, con parámetro'
b = a.split(',')
# ['esto es', ' un ejemplo', ' de split', ' con parámetro']
```

También se puede especificar, con un parámetro adicional, el número máximo de "cortes" a realizar. El siguiente ejemplo devuelve una lista con tres elementos, dado que se pasa un 2 como segundo parámetro.

```
a = 'esto es, un ejemplo, de split, con parámetros'
b = a.split(',', 2)
# ['esto es', ' un ejemplo', ' de split, con parámetros']
```

El método `replace()`

El método `replace()`, devuelve una string que es igual a la string sobre la que se aplica, cambiando las ocurrencias de una substring por otro valor. Suponiendo la siguiente asignación:

```
a = "Susanita tiene un ratón"
```

Si se supone ahora que el ratón de Susanita se escapó:

```
b = a.replace("tiene", "tenía") # b = "Susanita tenía un ratón"
```

El método `replace()` devuelve una copia de la string, reemplazando las ocurrencias de una substring por otra:

```
a = "Un globo, dos globos, tres globos"
b = a.replace("globo", "elefante")
# b = "un elefante, dos elefantes, tres elefantes"
```

Recorte y relleno: los métodos `strip()`, `lstrip()`, `rstrip()`, `ljust()`, `rjust()`, `center()` y `zfill()`

En ocasiones, una string contiene espacios al principio o al final que no resultan útiles y se desea eliminar. El método `strip()` elimina los espaciadores de los extremos de una *string*:

```
a = "    string con espacios    "
b = a.strip() # "string con espacios"
```

Los métodos `lstrip()` y `rstrip()` hacen lo mismo, pero solo en uno de los extremos (izquierdo - *left* o derecho - *right*):

```
left = a.lstrip() # "string con espacios    "
right = a.rstrip() # "    string con espacios"
```

También existen métodos para justificar una string en un tamaño determinado, ajustándola a la izquierda o a la derecha y rellenando el espacio sobrante con un carácter especificado:

```
a = "hola"
b = a.ljust(10, "_") # "hola_____"
c = a.rjust(10, "_") # "_____hola"
```

Combinando los métodos `ljust()` y `rjust()`, se puede lograr una justificación "centrada", aunque existe un método, `center()`, para hacer eso directamente:

```
mid = 5 + len(a) // 2
d = a.ljust(mid, "_").rjust(10, "_") # "____hola____"
d1 = a.center(10, "_") # "____hola____"
```

Por su parte, el método `zfill()` rellena una *string* añadiéndole ceros por la izquierda hasta alcanzar un tamaño determinado:

```
num = "12"
num1 = num.zfill(5) # "00012"
```

Obviamente, siempre es posible rellenar con cualquier carácter usando operaciones más básicas:

```
num = "12"
num1 = 'X' * (5 - len(num)) + num # "XXX12"
```

Formateo de strings

Texto con formato

Las dos capturas de pantalla que se muestran a continuación ilustran la diferencia entre un texto sin formato y uno formateado:

```
Aceite 9
Arroz 8
Azucar 10.5
Pollo 30.25
-----
Total 57.75
CAPTURA 1
```

```
Aceite 9.00
Arroz 8.00
Azucar 10.50
Pollo 30.25
-----
Total 57.75
CAPTURA 2
```

Ambas capturas muestran la misma información, pero la segunda presenta el texto formateado, alineando por un lado los productos y por el otro los precios, y unificando el número de dígitos decimales de estos últimos. **El texto con formato resulta más legible** y, en consecuencia, es más fácil interpretar la información que suministra.

Interpolación de strings: f-strings

Desde la versión 3.6 de Python, el método preferente para formatear texto son los llamados "literales de string formateados" o *f-strings*, que usan como técnica la interpolación de expresiones:

```
name = "John"
age = 24
print(f'{name} tiene {age} años') # Los espacios dentro de
                                # llaves no tienen efecto
```

[ver ejecución >>>](#)

El código previo muestra el mensaje:

```
John tiene 24 años
```

Al anteponer una 'f' (o 'F') a un literal de tipo string, se indica que se trata de una *f-string*. Una *f-string* es un literal string en el que se pueden intercalar expresiones de cualquier tipo, encerradas entre llaves, {}, que se evaluarán, se convertirán a string y se insertarán en la *f-string* en la posición correspondiente.

```
num1 = 12
num2 = 10
print(f"sumar {num1} y {num2} da {num1 + num2}")
ver ejecución >>>
```

```
sumar 12 y 10 da 22
```

Si se quieren incluir los caracteres de llaves en una *f-string*, es necesario doblarlos:

```
print(f"La palabra {{llaves}} se muestra entre llaves")
```

```
La palabra {llaves} se muestra entre llaves
```

Las *f-strings*, una vez evaluadas, dan como resultado una string normal, por lo que, por ejemplo, pueden ser asignadas a variables:

```
name = "John"
age = 24
message = f'{name} tiene {age} años'
print(message)
ver ejecución >>>
```

```
John tiene 24 años
```

Modificadores de formato

La captura de pantalla con el texto formateado que se muestra a continuación ha sido generada con el siguiente código, donde *prices* es un diccionario cuyas claves son nombres de productos, y los valores son sus correspondientes precios:

```
Aceite 9.00
Arroz 8.00
Azucar 10.50
Pollo 30.25
-----
Total 57.75
```

```
total = 0.0

for product, price in prices.items():
    print(f'{product:7}{price:5.2f}')
    total += price

print("-----")
print(f"Total {total:5.2f}")
ver ejecución >>>
```

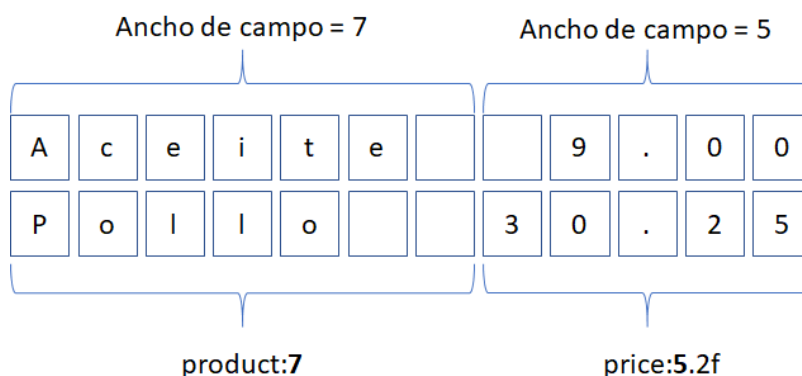
Como se puede observar, se usa una *f-string* en la que se interpolan las variables cuyos valores se quiere mostrar: `f'{product:7}{price:5.2f}'`. En lugar de incluir solo los nombres de las variables, se han añadido modificadores de formato, que son los textos que están entre el carácter dos puntos (':') y el carácter cierra llaves ('}'). Los modificadores de formato controlan cómo va a aparecer el resultado de la expresión que está a la izquierda de los dos puntos.

Modificadores de formato - ancho de campo

Los modificadores de formato controlan cómo va a aparecer el resultado de la expresión que está a la izquierda de los dos puntos.

```
f'{product:7}{price:5.2f}'
```

El primer número del modificador de formato indica el "ancho del campo" (número de caracteres) en el que se va a mostrar el valor.



Por omisión, el texto se alinea a la izquierda del campo especificado si es una string y a la derecha si es un número, pero puede especificarse qué tipo de justificación se quiere anteponiendo al ancho de campo los símbolos < para alinear a la izquierda, ^ para centrar y > para alinear a la derecha:

```
value = "align left"
print(f"[{value:<20}]")
```

```
[align left    ]
```

```
value = "align center"
print(f"[{value:^20}]")
```

```
[ align center ]
```

```
value = "align right"
print(f"[{value:>20}]")
```

```
[      align right]
```

Hay que tener en cuenta que el modificador de ancho de campo indica un mínimo, pero el campo se extenderá cuanto sea necesario para representar completamente el valor formateado. Si por el contrario fuera suficiente, por omisión el campo sobrante se rellena con espacios tal como se ha visto, si bien puede especificarse otro valor de relleno.

En el caso de que el valor a representar sea un número, un cero a la izquierda del ancho de campo indica que se rellene con ceros:

```
for num in range(5, 11):  
    print(f"{num:03}")
```

```
005  
006  
007  
008  
009  
010
```

En cualquier otro caso, el carácter de relleno tiene que ir seguido de un código de alineación (<^>) antes del ancho de campo:

```
for num in range(5, 11):  
    print(f"{num:*>3}") # > indica alineación derecha
```

```
**5  
**6  
**7  
**8  
**9  
*10
```

Modificadores de formato - Precisión

Un punto seguido de un número indica la precisión con la que se quiere formatear el valor. Véanse los siguientes ejemplos.

```
import math  
print(math.pi)      # 3.141592653589793  
print(1000000/3)     # 333333.3333333333  
print(0.0000555)     # 5.55e-05
```

Si no se especifica formato, al convertirlos a *string* (en los anteriores ejemplos, automáticamente, antes de ser visualizados con *print*) los valores *float* se representan íntegramente, reflejando la precisión con la que se almacenan internamente (obsérvese el número de cifras para π y para el resultado de $1000000/3$). La notación $5.55e-05$ expresa:

$$5.5 * 10^{-5}.$$

```
print(f"{math.pi:.2}") # 3.1  
print(f"{0.00555:.2}") # 0.0056  
print(f"{555.5:.2}")   # 5.6e+02
```

La precisión 2 aquí se refiere a los 2 primeros dígitos significativos (se redondea apropiadamente). Se incluye al menos un dígito a la derecha del punto. No aplicable a valores de tipo *int*.

```
print(f"{math.pi:.2f}") # 3.14  
print(f"{0.00555:.2f}") # 0.01  
print(f"{555:.2f}")     # 555.00
```

La *f* del final indica *fixed point*: la parte entera (incluso si es 0) se representará a la izquierda del punto, y a la derecha de este se incluirán exactamente (tras el apropiado redondeo) los 2 primeros dígitos fraccionarios (incluso los ceros).

```
print(f"{math.pi:5.2f}") # 3.14
print(f"{555:5.2f}") # 555.00
```

Lo mismo que el caso anterior, pero en un campo de ancho de al menos 5 posiciones (nótese que 555.00 necesita 6).

```
print(f"{str(math.pi):.2}") # 3.
```

El valor *float* del número π se ha convertido primero a un valor de tipo *string*, que es lo que se formatea, en este caso recortándola y quedando únicamente los 2 primeros caracteres de esta (nótese la diferencia con respecto al caso en que la misma especificación de formato se aplica sobre un valor de tipo numérico).

Modificadores de formato - Notación numérica

Al final del modificador de formato se puede añadir una letra para especificar cómo se desea interpretar el tipo de datos: o (octal), x (hexadecimal), f (punto flotante), e (notación científica), d (decimal). Véanse los siguientes ejemplos:

```
print(f"{12:5o}")
```

```
14
```

El valor entero 12 (por omisión, en la usual base diez) se imprime en octal (base ocho, que solo usa los dígitos del 0 al 7), en un campo de tamaño 5.

```
print(f"{12:5x}")
```

```
print(f"{12:5X}")
```

```
c
C
```

El valor entero se imprime en hexadecimal (base dieciséis, que usa los dígitos 0-9 y a-f). Las letras 'a', 'b', 'c', 'd', 'e' y 'f' de la representación hexadecimal se muestran en mayúscula si el modificador 'X' está en mayúscula.

```
print(f"{12:5.1f}")
```

```
12.0
```

Un valor, entero o real, se imprime como *float* (si el valor es entero, se añaden ceros como parte fraccionaria).

```
print(f"{-140:5.1e}")
```

```
print(f"{0.014:5.1E}")
```

```
-1.4e+02
1.4E-02
```

Un valor, entero o real, se imprime en notación científica (coma flotante) indicando el exponente de la potencia de diez: $-140 = -1.4 \times 10^2$, $0.014 = 1.4 \times 10^{-2}$. Obsérvese que se muestra "e" o "E" según el modificador.

```
print(f"{12:5d}")
```

```
12
```

Un valor entero se imprime en formato decimal (la usual base diez, que usa los dígitos del 0 a 9).

Carácter de signo

Es posible controlar la visualización del signo para números no negativos. Ejemplos:

```
print(f"{-12:5d}")
```

```
-12
```

Por omisión, el carácter para el signo (– o +) solo se muestra si el número es negativo.

```
print(f"{-12:+5d}")
```

```
print(f"{12:+5d}")
```

```
-12
```

```
+12
```

Si al principio de los modificadores de formato se añade el carácter '+' se mostrará, tanto el signo '-' como el '+', en función de si el valor numérico es negativo o positivo, respectivamente.

```
print(f"{-12:-2d}")
```

```
print(f"{12:-2d}")
```

```
print(f"{-12: 2d}")
```

```
print(f"{12: 2d}")
```

```
-12
```

```
12
```

```
-12
```

```
12
```

Añadir un carácter '-' no afecta al resultado (sólo se muestra el signo cuando el valor es negativo), pero si se añade un espacio, muestra este en lugar del signo cuando el valor no es negativo.

El método format()

Una alternativa a la interpolación de strings, propia de versiones anteriores de Python, es el método *format()*:

```
num1 = 12 / 7
```

```
print("Primero = {:.2f}, segundo = {:2X}".format(num1, 10))
```

```
Primero = 1.71, segundo = A
```

Como se ve en el ejemplo, es muy similar a la interpolación de strings, salvo que las expresiones cuyos valores se desean incluir en la string se pasan como parámetros del método *format()*, ocupando el lugar que tienen reservado en la string, en principio por orden de aparición.

Existe la posibilidad de dar un nombre a cada campo, lo que independiza su orden en la string del orden en que se pasan al método *format()*.

```
print("Primero = {num1:4.2f}, segundo = {num2:2X}".format(num1=num1, num2=10))
```

```
print("Segundo = {num2:2X}, primero = {num1:4.2f}".format(num1=num1, num2=10))
```


Tema 8 Expresiones regulares

Búsqueda de patrones

Cuando se usa un método como *find()* para buscar una substring en una string, se está realizando un tipo de búsqueda que se conoce como **búsqueda exacta**; se quiere saber si un texto contiene exactamente una secuencia concreta de caracteres conocida. Por ejemplo, si se tiene un documento, *text*, en forma de string, y se quiere saber si contiene una referencia a la fecha "22/12/1964", se escribiría algo como:

```
match_position = find(text, "22/12/1964")
```

La función devolvería la posición donde la secuencia "22/12/1964" aparece por primera vez en *text*, o -1 si no aparece. Pero ¿qué pasa si lo que se quiere es saber si aparece alguna fecha, o qué fechas aparecen, pero no se tiene la idea de cuáles pueden ser, solo del formato en que deben estar?

Si se supone que, como en el ejemplo, las fechas buscadas siguen un formato *dd/mm/aaaa* (día, mes y año separados por barras, con dos dígitos para el día, dos dígitos para el mes y cuatro dígitos para el año), este formato define un **patrón** para las fechas. Hay muchos tipos de información textual que siguen patrones: números de teléfono, DNIs, direcciones de correo electrónico, URLs, etc. Lo que se necesita es un lenguaje estandarizado para poder expresar esos patrones, junto con unos métodos que los interpreten para hacer búsquedas. Las **expresiones regulares** proporcionan un lenguaje estandarizado para expresar patrones de texto.

Un caso particular de la **búsqueda de patrones** es la **verificación de patrones**, que no busca saber si un texto contiene una string que responde a un patrón, sino si el texto completo responde a dicho patrón. Por ejemplo: ¿la dirección de correo introducida por un usuario en un formulario tiene un formato de email correcto?

Lenguaje de expresiones regulares

Una **expresión regular** es un patrón expresado como una secuencia de caracteres que sigue un lenguaje específico. El lenguaje estándar de expresiones regulares distingue entre caracteres corrientes, metacaracteres y secuencias de escape.

- Los **caracteres corrientes** se representan a sí mismos. Por ejemplo, la secuencia "hola" representa un patrón al que responde cualquier texto formado por una 'h' seguida de una 'o' seguida de una 'l', seguida de una 'a', todas en minúsculas.
- Los **metacaracteres** tienen un significado especial. Por ejemplo, los caracteres '(' y ')' no representan la presencia de paréntesis en el texto, sino que operan como agrupadores de elementos del patrón.
- Las **secuencias de escape** se forman anteponiendo una barra invertida (backslash - '\') a un carácter, lo que cambia el significado de este.
 - Si se antepone a un metacarácter, le hace perder su significado especial y pasa a representar al carácter corriente. Por ejemplo, si se quiere que en el texto aparezca un abre paréntesis, como el carácter '(' es un metacarácter de agrupamiento, se tendrá que usar la secuencia '\('.

- Si se antepone a determinados caracteres corrientes, les da un significado especial. Por ejemplo, el carácter 'd' se representa a sí mismo, pero la secuencia '\d' representa la aparición de un dígito.
- Muchos caracteres corrientes no adquieren un significado especial al anteponerles un *backslash* y siguen representándose a sí mismos; por ejemplo, 'a' es lo mismo que 'a'.

Secuencias de escape

Las **secuencias de escape** que anteponen el *backslash* a caracteres corrientes se usan principalmente para dos fines:

- Designar **clases de caracteres**, de forma que se pueda indicar que en una posición determinada del texto que se busca debe ir, no un carácter concreto, sino cualquier carácter de una clase predefinida.
- Indicar una **"frontera"**, como el punto de encuentro entre dos clases de caracteres o un extremo de la string en la que se busca el patrón.

Secuencias como clases de caracteres

Secuencia	Significado						
\d	Un dígito del 0 al 9						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>d\d\d\d</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	d\d\d\d	captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	d\d\d\d						
captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
\D	Cualquier carácter que NO sea un dígito						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>\D\D\D\D</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	\D\D\D\D	captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	\D\D\D\D						
captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
\w	Cualquier carácter alfanumérico (A-Z, a-z, 0-9 y _)						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>\w\w\w\w</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	\w\w\w\w	captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	\w\w\w\w						
captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
\W	Cualquier carácter que NO sea un alfanumérico						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>\W\W</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	\W\W	captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	\W\W						
captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
\s	Un <i>whitespace</i> (carácter espacio ' ', \n, \t o \r)						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>\s</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se \n llegó a la Luna\n</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	\s	captura	A las 2:56 (UTC) horas del 21-07-1969 se \n llegó a la Luna\n
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	\s						
captura	A las 2:56 (UTC) horas del 21-07-1969 se \n llegó a la Luna\n						
\S	Cualquier carácter que NO sea un <i>whitespace</i>						
	<table> <tr> <td>texto</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> <tr> <td>patrón</td><td>\S</td></tr> <tr> <td>captura</td><td>A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna</td></tr> </table>	texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna	patrón	\S	captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna
texto	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						
patrón	\S						
captura	A las 2:56 (UTC) horas del 21-07-1969 se llegó a la Luna						

Secuencias como fronteras

Secuencia	Significado
\b	Borde de palabra (principio o final de una secuencia alfanumérica)
texto	No es este mes el que testeo
patrón	es\b
captura	No es sino este mes el que testeo
texto	No es sino este mes el que testeo
patrón	\bes
captura	No es sino este mes el que testeo
texto	No es sino este mes el que testeo
patrón	\b
captura	N o es sino este mes el que testeo
\B	Posición que no es un borde de palabra (interior de secuencia alfanumérica)
texto	No es sino este mes el que testeo
patrón	es\B
captura	No es sino este mes el que testeo
\A	Principio absoluto de la string
texto	una vez, antes de la una llegó a la Luna
patrón	\Auna
captura	una vez, antes de la una llegó a la Luna
\Z	Final absoluto de la string
texto	una vez, antes de la una llegó a la Luna
patrón	una\Z
captura	una vez, antes de la una llegó a la Luna

Metacaracteres

Los **metacaracteres** se pueden agrupar en varias categorías: cuantificadores, anclas, definidores de conjuntos y rangos, agrupadores y otros.

Cuantificadores

Indican la repetición del elemento que les precede y cuantifican el número de veces que puede o debe repetirse.

MetaCar	Significado
*	Cero o más repeticiones del elemento anterior
texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
patrón	ho*la
captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
+	Una o más apariciones del elemento anterior
texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
patrón	ho+la
captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola

MetaCar	Significado	
?	Cero o una repetición del elemento anterior	
	texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
	patrón	ho?la
	captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
{n,m}	Entre n y m repeticiones del elemento anterior	
	texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
	patrón	ho{2,4}la
	captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
{n,}	n o más apariciones del elemento anterior	
	texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
	patrón	ho{3,}la
	captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
{3}	Exactamente n repeticiones del elemento anterior	
	texto	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola
	patrón	ho{3}la
	captura	hla**hola**hoola**hoola**hoola**hoola**hoola**hoola**hola

Los cuantificadores repetitivos, como '+', '*' y '?', son "avariciosos" (*greedy*), lo que significa que intentan abarcar el máximo número de repeticiones posible. Esto puede ser problemático en ciertas circunstancias.

En el siguiente ejemplo, se intenta obtener, separadas, todas las etiquetas de un documento html; una etiqueta del lenguaje html empieza con el carácter '<', seguido de cualquier número de caracteres y finaliza con el carácter '>'. Por ejemplo:

```
<html><head><title>Título</title></head><body>Cuerpo del documento</body></html>
```

Con la descripción dada de lo que es una etiqueta html, un patrón válido podría ser: `<.*>` (apertura, uno o más caracteres cualquiera, cierre).

Pero, aunque el patrón usado parece correcto (por ejemplo, '`<html>`' y '`<head>`' encajan aisladamente), el carácter avaricioso del cuantificador '*' hace que todo el texto (hasta el último '>') se considere como una única etiqueta html:

```
<html><head><title>Título</title></head><body>Cuerpo del documento</body></html>
```

Para evitarlo, en Python se puede incluir un signo de interrogación '?' detrás del cuantificador '*' para hacerlo "no avaricioso" (*non-greedy*), de modo que se extienda lo menos posible. Usar el patrón `<.*?>` daría el resultado deseado:

```
<html><head><title>Título</title></head><body>Cuerpo del documento</body></html>
```

Anclas o delimitadores

Las **anclas** son metacaracteres que indican que el texto buscado debe cumplir determinadas restricciones respecto a su posición en el texto de búsqueda.

MetaCar		Significado
^		Inicio de línea
	texto	tres tristes tigres tres comían trigo en un trigal a las tres
	patrón	^tres
	captura	tres tristes tigres tres comían trigo en un trigal a las tres
\$		Final de línea
	texto	tres tristes tigres tres comían trigo en un trigal a las tres
	patrón	tres\$
	captura	tres tristes tigres tres comían trigo en un trigal a las tres

Conjuntos y rangos de caracteres

Los **corchetes** se usan para indicar que en una posición del texto puede aparecer cualquier carácter de un conjunto o un rango especificado.

Expresión		Significado
[abc]		Cualquiera de los caracteres que están entre los corchetes
	texto	El niño y la niña jugaban con un balón
	patrón	niñ[oa]
	captura	El niño y la niña jugaban con un balón
[a-b]		Cualquier carácter del rango indicado entre corchetes
	texto	A las 2:56 (UTC) horas del 21 de julio de 1969 se llegó a la Luna
	patrón	[0-9]+
	captura	A las 2:56 (UTC) horas del 21 de julio de 1969 se llegó a la Luna
Se pueden combinar conjuntos y rangos		
	texto	El niño y la niña jugaBan con un balón
	patrón	[oaA-Z]
	captura	El niño y la niña jugaBan con un balón
Un carácter '^' al principio de los corchetes invierte su significado		
	texto	El niño y la niña jugaBan con un balón
	patrón	[^oaA-Z]
	captura	El niño y la niña jugaBan con un balón

Dentro de los corchetes, los caracteres especiales pierden su significado y son tratados como caracteres literales corrientes.

Otros metacaracteres

MetaCar	Significado
.	Cualquier carácter (excepto salto de línea)
	texto Caperucita tenía tres tíos por parte de madre
	patrón t..s
	captura Caperucita tenía tres tíos por parte de madre
	Selecciona entre dos alternativas
	texto El niño y la niña jugaban con un balón
	patrón niño niña
	patrón niñ(o a)
	captura El niño y la niña jugaban con un balón
\	Forma secuencias de escape para metacaracteres (p.e., '\.' para un punto literal)

Hay que señalar que el metacaracter '|' empieza a buscar las alternativas de izquierda a derecha y se detiene cuando encuentra la primera; por tanto, si una alternativa es prefijo de otra, se debe poner la más larga primero. Por ejemplo, en el texto "Ella no canta, antes cantaba", el patrón "canta|cantaba" encuentra dos apariciones de "canta":

"Ella no **canta**, antes **cantaba** "

Mientras que el patrón "cantaba|canta" encuentra una aparición de "canta" y otro de "cantaba":

"Ella no **canta**, antes **cantaba**"

En la string "23 56 12", El patrón [2-9]|[1-9]\d encuentra las ocurrencias: '2', '3', '5', '6', '12', mientras que el patrón [1-9]\d|[2-9] encuentra las ocurrencias '23', '56', '12', dado que las ocurrencias capturadas por [2-9] (un dígito distinto de '0' o '1') son prefijo de las capturadas por [1-9]\d (dos dígitos, el primero distinto de '0').

Grupos

Los **paréntesis** permiten agrupar partes de un patrón para capturarlas específicamente o para que se traten como una unidad. Nótese en los siguientes ejemplos la diferencia entre aplicar la selección entre dos alternativas limitando su alcance con paréntesis y no hacerlo.

texto	El niño y la niña jugaban con un balón
--------------	--

patrón	niño a
captura	El niño y la niña jugaban con un balón

patrón	niñ(o a)
captura	El niño y la niña jugaban con un balón

Los colores distintos de las letras 'o' y 'a' de "niño" y "niña" indican que, además de formar parte de la captura completa obtenida por el patrón, también se capturan como grupos específicos dentro de esta. Si no se desea que un grupo sea capturado, se debe incluir la secuencia "?:" al principio de los paréntesis.

patrón	niñ(?:o a)
captura	El niño y la niña jugaban con un balón

La captura de grupos específicos es útil porque se puede hacer referencia a ellos posteriormente:

```
fecha = "21-07-1969"
date = replace("(\\d\\d)-(\\d\\d)-(\\d{4})", fecha, "\\3-\\2-\\1")
print(date)
```

```
1969-07-21
```

Los grupos especificados por los paréntesis de patrón se numeran de izquierda a derecha, empezando en 1, y de fuera a adentro en caso de que haya paréntesis anidados. Estos números pueden usarse en secuencias de escape en un patrón para hacer referencia a los grupos capturados (*backreferences*). En el ejemplo previo, se usan *backreferences* en un patrón de sustitución referido al patrón de búsqueda aplicado en la función *replace()*. Lo que se hace es crear, en el patrón de búsqueda grupos para el día, el mes y el año, y luego, en el patrón de sustitución se usan *backreferences* a los grupos encontrados cambiándolos de orden.

En el siguiente ejemplo, se usan en el propio patrón de búsqueda para dar coherencia a las expresiones encontradas:

texto	21/07/1969 21-/07/1969 21/07-1969 21-07-1969
patrón	\\d\\d(- \\ \\)\\d\\d\\1\\d{4}
captura	21/07/1969 21-/07/1969 21/07-1969 21-07-1969

En este caso, se trata de encontrar fechas, considerando válidas solo aquellas que usan el mismo separador entre día y mes y entre mes y año. Para ello se captura el primer separador y se exige, con un *backreference*, que el segundo sea igual al que ya se ha encontrado.

Uso de expresiones regulares en Python

Preparación

Para usar expresiones regulares en Python, lo primero que se debe hacer es importar el módulo *re* (*regular expressions*):

```
import re
```

Los patrones se introducen como strings. El texto sobre el que se aplica el patrón también será una string:

```
pattern = "(\\d\\d):\\t\\w+\\b\\(\\1\\)"
```

Sin embargo, surge un problema: al crear la string que representa el patrón, Python procesa las secuencias de escape según sus propias reglas:

```
pattern = "(\\d\\d):\\t\\w+\\b\\(\\1\\)"
print(pattern)
```

```
(\\d\\d): \\w\\(\\)
```

Las secuencias '\d', '\w', '\(' y '\)' se han respetado porque no tienen un significado especial como secuencias de escape en las strings de Python. Sin embargo, la secuencia '\t' ha sido sustituida por un carácter de salto de tabulación (lo que puede ser correcto), pero '\b' ha sido sustituido por un carácter de retroceso (*backspace*), que no era lo que queríamos, y '\1' por el carácter de valor octal 1. Este no era el comportamiento deseado, ya que invalida el patrón de la expresión regular.

Para solucionarlo, Python ofrece la posibilidad de anteponer una 'r' a la string para marcarla como una "string sin procesar" (*raw string*), evitando que las secuencias de escape sean interpretadas al crearla:

```
pattern = r"(\d\d):\t\w+\b\(\1)"
print(pattern)
(\d\d):\t\w+\b\(\1)
```

De este modo, el patrón llega al módulo de expresiones regulares "tal cual" y será este módulo el que interprete qué secuencias corresponden a caracteres y cuáles a metacaracteres.

Podría haber un problema si se quiere incluir en el patrón una secuencia que tenga significado tanto para Python como para el módulo *re*, y se desea mantener la primera interpretación. En ese caso, existe la opción de no usar la 'r' y duplicar las barras invertidas de las secuencias ambiguas que deban ser interpretadas por el módulo *re*.

```
pattern = "(\d\d):\t\w+\\b\(\1\\)\b"
print(pattern)
pattern = "(\d\d):\t\w+\\b\(\1\\)\b"
```

En el ejemplo anterior se duplican las barras de la primera secuencia '\b' para interpretarla como frontera de palabra, y de la secuencia '\1' para interpretarlo como *backreference*, pero no la de la segunda '\b' porque se desea interpretar como *backspace*.

Operaciones

El módulo *re* de Python permite básicamente realizar tres operaciones con expresiones regulares: verificar una expresión, buscar un texto, o buscar y reemplazar un texto.

Verificar

Se entiende por verificar ver si un texto se ajusta a un patrón. El módulo *re* de Python ofrece dos funciones para hacer esto: *fullmatch()* y *match()*.

La función *match()* busca la ocurrencia de un patrón al principio de una string, mientras que *fullmatch()* verifica que el texto completo cumpla el patrón. Ambas devuelven un objeto *Match*, si hay coincidencia, o *None*, si no la hay:

```
import re

pattern = '(\d\d)(-|/)(\d\d)(\2)(\d{4})'
test_string = '30/06/1970 es una fecha muy popular'
result = re.match(pattern, test_string)

if result:
    print(result)
<_sre.SRE_Match object; span=(0, 10), match='30/06/1970'>
```

```

print(result.start())
0
print(result.end())
10
print(result.span())
(0, 10)
print(result.group(0))
30/06/1970
print(result.group(5))
1970

```

Como se puede observar, los objetos *Match* ofrecen diferentes métodos para extraer la información de la coincidencia:

- **start():** Índice del comienzo de la ocurrencia encontrada, que para el método *match()* es siempre 0.
- **end():** Índice del final de la ocurrencia, la cual va, como es usual, desde *start()* hasta *end()-1*, ambos inclusive.
- **span():** Una tupla con los valores de *start()* y *end()*.
- **group(n):** La substring que ha coincidido con el grupo cuyo número (n) se pasa por parámetro. Además, *group(0)* devuelve la ocurrencia completa.

Hay que tener cuidado con el uso del método *group()*. El grupo 0 siempre existe, pero dependiendo de la expresión, otros grupos podrían no existir (se retorna *None*), por lo que habría que preguntar si existen, antes de intentar acceder a ellos. En el siguiente ejemplo, los grupos 2 y 3 son excluyentes, de manera que, si en una ocurrencia aparece uno de ellos, no puede aparecer el otro. Por ejemplo, dado el siguiente patrón:

group(1)
 {
 group(3)
 {
 group(2)
 }
 }
 \d ((\d\d) | (\D\D)) _ \d

El grupo 2 está en la string "123_4", donde no está el grupo 3, y el grupo 3 está en la string "1AB_4", donde no está el grupo 2.

Por el contrario, varias substrings pueden encajar con un mismo grupo si este se encuentra dentro de una estructura repetitiva (p.e. con ***): *group()* devolverá solo la última de tales substrings.

Buscar

A la hora de buscar, se debe diferenciar si se busca solo la primera ocurrencia o todas las ocurrencias de un patrón en un texto.

La función *search()* devuelve un objeto *Match* para la primera ocurrencia del patrón en la string de búsqueda, independientemente de la posición donde se encuentre. Si no hay ninguna ocurrencia del patrón en la string de búsqueda, devuelve *None*.

```
import re
pattern = r'\d\d(-|/)\d\d\1\d{4}'
test_string = 'palabra1 30/06/1970 PALABRA2 12-12-2008 palabra3'
result = re.search(pattern, test_string)
print(result)
<_sre.SRE_Match object; span=(9, 19), match='30/06/1970'>
```

La función *finditer()* devuelve una secuencia iterable de objetos *Match* con todas las ocurrencias (no solapadas) del patrón en la string donde se busca:

```
import re

pattern = r'\d\d(-|/)\d\d\1\d{4}'
test_string = 'palabra 1 30/06/1970 PALABRA2 12-12-2008 palabra3'

for match in re.finditer(pattern, test_string):
    print(match)
    print(match.start())
    print(match.end())
    print(match.span())
    print(match.group(0))
```

En el ejemplo anterior se muestra:

```
<re.Match object; span=(10, 20), match='30/06/1970'>
10
20
(10, 20)
30/06/1970
<re.Match object; span=(30, 40), match='12-12-2008'>
30
40
(30, 40)
12-12-2008
```

Buscar y reemplazar

La función *sub()* sirve para sustituir las ocurrencias del patrón en la string de búsqueda por un nuevo valor:

```
import re

pattern = '(\d\d)-(\d\d)-(\d{4})'
test_string = 'palabra1 30-06-1970 PALABRA2 12-10-2008 palabra3'
new_string = re.sub(pattern, '<DATE>', test_string)
print(new_string)
```

El resultado del ejemplo anterior es:

```
palabra1 <DATE> PALABRA2 <DATE> palabra3
```

La string con el nuevo valor puede contener referencias a grupos de la ocurrencia que se va a sustituir:

```
import re

pattern = r'(\d\d)-(\d\d)-(\d{4})'
test_string = 'palabra1 30-06-1970 PALABRA2 12-10-2008 palabra3'
new_string = re.sub(pattern, r'\2/\1/\3', test_string)
print(new_string)
```

El resultado del ejemplo es:

```
palabra1 06/30/1970 PALABRA2 10/12/2008 palabra3
```

Con ello hemos conseguido intercambiar los grupos 1 y 2 de cada fecha, y sustituir los guiones por barras.

Existe una función parecida llamada *subn()* que, en lugar de devolver la string modificada, devuelve una tupla en la que el primer elemento es la string modificada y el segundo el número de sustituciones realizadas.

Anexo – Chuleta de expresiones regulares

Secuencias como clases de caracteres		Secuencias como fronteras	
\d	Un dígito del 0 al 9	\b	Borde de secuencia alfanumérica (principio o final)
\D	Cualquier carácter que NO sea un dígito	\B	Interior de secuencia alfanumérica (ni principio ni final)
\w	Cualquier carácter alfanumérico (A-Z, a-z, 0-9 y _)	\A	Principio absoluto de la string
\W	Cualquier carácter que no sea alfanumérico	\Z	Final absoluto de la string
\s	Un <i>whitespace</i> (carácter espacio ' ', \n, \t o \r).	Cuantificadores	
\S	Cualquier carácter que no sea un <i>whitespace</i>	*	Cero o más repeticiones del elemento anterior
Anclas o delimitadores		+	Una o más apariciones del elemento anterior
^	Inicio de línea	?	Cero o una repetición del elemento anterior
\$	Final de línea	{n,m}	Entre n y m repeticiones del elemento anterior
Conjuntos y rangos de caracteres		{n,}	n o más apariciones del elemento anterior
[abc]	Cualquier carácter de los que están entre los corchetes	{3}	Exactamente n repeticiones del elemento anterior
[a-b]	Cualquier carácter del rango indicado entre corchetes	Otros metacaracteres	
Se pueden combinar conjuntos y rangos		.	Cualquier carácter (excepto salto de línea)
Un carácter '^' al principio de los corchetes invierte su significado			Selecciona entre dos alternativas
		\	Forma secuencias de escape
Grupos			
Los paréntesis permiten agrupar partes de un patrón para capturarlas específicamente o aplicarlas como una unidad.			
Operaciones más usuales del módulo re			
• re.match(pattern, text) • re.fullmatch(pattern, text) • re.search(pattern, text) • re.finditer(pattern, text) • re.sub(searchpattern, replacepattern, text)			
Métodos de los Objetos Match			
.start() .end() .span() .group()			

Tema 9 Archivos de texto

Archivos de texto

Las aplicaciones del mundo real procesan (leen o/y producen) diversos datos que se guardan en archivos/ficheros externos. La información en los archivos se puede guardar en formato binario o en formato texto. Dado que la información en un ordenador siempre se representa en binario, el significado de la frase anterior es que en la codificación binaria de un fichero de texto lo que se representan son caracteres⁴, mientras que en otros tipos de ficheros pueden codificarse objetos de cualquier clase.

El contenido de un archivo de texto es una secuencia de caracteres, incluidos caracteres terminadores de línea como newline '\n'. Ejemplos de típicos de archivos con formato de texto incluyen aquellos con extensión .txt, .csv, .html ... ¡y, desde luego, .py!

Se puede también considerar un archivo de texto como una secuencia de líneas, siendo cada línea una secuencia de caracteres terminada con '\n'. Un archivo de texto puede contener líneas vacías, esto es, que solo tienen el carácter '\n'⁵.

Los archivos, sean del tipo que sean, son entidades externas a los programas. Son creados, modificados y borrados por programas, pero existen fuera de los mismos, almacenados en dispositivos físicos, como discos, que permiten su persistencia, y gestionados por el Sistema de Gestión de Archivos.

Un Sistema de Gestión de Archivos (SGA) o File Management System en inglés, es un conjunto de software y herramientas que permite la organización, almacenamiento, manipulación y recuperación de archivos en un sistema informático. El objetivo principal de un SGA es administrar de manera eficiente y organizada los datos almacenados en los dispositivos de almacenamiento, como discos duros, unidades de estado sólido, servidores o cualquier otro medio de almacenamiento. El SGA es una parte fundamental del sistema operativo (SO). Aunque el término "sistema de gestión de archivos" puede usarse en diferentes contextos (por ejemplo, en aplicaciones de almacenamiento en la nube), en el ámbito de los sistemas operativos, hace referencia a la funcionalidad que el SO proporciona para gestionar los archivos y directorios en el almacenamiento.

⁴ Todo archivo, del tipo que sea, es una secuencia de octetos (bloques de 8 bits), cuyas combinaciones de 0 y 1 codifican algún tipo de dato, de acuerdo con algún esquema preestablecido (que puede ser octeto a octeto o en grupos de octetos). Así pues, cuando se dice que un archivo de texto contiene una secuencia de caracteres, lo que se quiere decir es que cuando se construye el archivo, los octetos que lo componen se escriben utilizando alguna codificación [encoding] de caracteres definida en un estándar o norma. Naturalmente, al leerlos posteriormente, debieran ser interpretados de acuerdo con dicha codificación. Existen varios estándares de codificación de caracteres, no obstante, en las prácticas de este contexto se escribirá y leerá usando una misma codificación, por lo que esta cuestión no afectará a la ejecución.

⁵ Podría darse el caso (aunque no sea recomendable según el caso) de que la última línea no esté vacía y no termine en '\n', o incluso que el fichero esté vacío (contenga cero caracteres, ni siquiera un '\n'). Estrictamente, un fichero que contuviera un solo carácter que fuera un '\n' no estaría vacío, pues contendría una línea, eso sí, vacía.

Como los archivos son entes externos a los programas, para trabajar con ellos los programas deben seguir un protocolo de comunicación con el SGA que, en general, comprende tres etapas:

1. **Abrir el archivo** (se establece una conexión entre el programa y el archivo físico en el sistema de ficheros del ordenador).
2. **Transferir información**, es decir, leer de, o escribir en, el archivo (se envían datos desde el archivo al programa, o desde el programa al archivo).
3. **Cerrar el archivo** (se libera la conexión entre el programa y el archivo físico en el sistema de archivos; lo que pudiera requerir que previamente se completen operaciones de escritura aún pendientes).

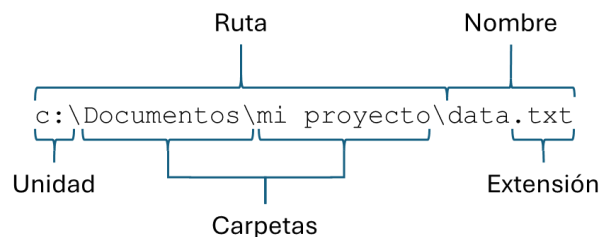
Apertura y cierre de un archivo

Para abrir un archivo de en Python llamamos a la función estándar *open()* pasándole el nombre del fichero que se desea abrir:

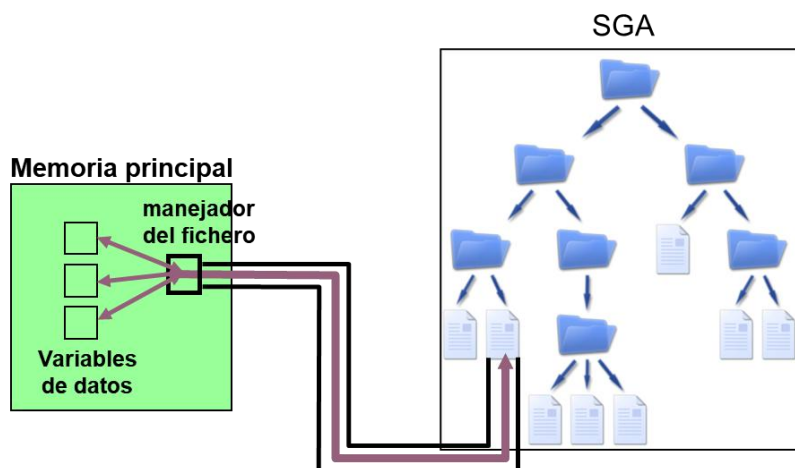
```
file = open('data.txt')
```

La figura muestra la estructura del nombre de un fichero en MS-Windows; es similar en otros sistemas. Si, como en el ejemplo, a la función *open()* solo se le

pasa el nombre "corto" del fichero, se supone que este se encuentra en la "carpeta por omisión", también conocida como "directorio actual de trabajo" asociada a la ejecución del programa. Se puede pasar la ruta completa o una ruta relativa a dicha carpeta por omisión.



La función *open()* crea un objeto manejador del fichero que representa el flujo de datos (flujo de entrada o flujo de salida) entre el programa y el fichero. Este objeto se asocia a una variable, *file* en el ejemplo, para usarlo en el resto del programa cada vez que se necesite interactuar con el fichero, para lo que ya no se necesitará su nombre.



Tal como se ha usado en el ejemplo, la función *open()* abre un fichero en modo lectura, para transferir información hacia el programa. La apertura en modo lectura requiere que el fichero exista. Si no hay en el disco un archivo con el nombre especificado, se genera un error (*FileNotFoundError*). Se puede especificar el modo de apertura añadiendo un segundo parámetro a la función *open()*, que, para el modo lectura tomaría el valor 'r'.

```
file = open('data.txt', 'r')
```

Los modos usuales de apertura de un archivo de texto son:

'r'	Abre un archivo en modo lectura . El archivo debe existir o se genera un error.
'w'	Abre un archivo en modo escritura . Si no existe se crea; si existe se sobrescribe
'a'	Abre un archivo para añadir contenido . Si no existe se crea; si existe, posiciona el curso de escritura al final, sin sobrescribir el contenido existente.

Existe un tercer argumento, opcional, que se puede añadir a la función `open()` para indicar la codificación a emplear en el archivo, por ejemplo, en caso de que no sea la estándar usada por omisión en el sistema:

```
f = open('data.txt', 'r', encoding = 'utf-8')
```

Cuando se termina de trabajar con un fichero, se debe cerrar aplicando el método `.close()` al manejador del fichero:

```
file.close()
```

Tratamiento de los datos de un archivo

Esquema estándar de tratamiento de un fichero de texto en modo lectura

El siguiente ejemplo muestra el esquema "estándar" de tratamiento (línea a línea) de un fichero de texto abierto en modo lectura. La llamada a `print()` se sustituiría con el tratamiento que se quisiera aplicar a cada línea.

```
def show_file(filename):
    file = open(filename, 'r')
    line = file.readline()

    while line != "":
        print(line, end="")
        line = file.readline()

    file.close()
```

1. Se abre el fichero (`file = open(filename, 'r')`)
2. Se intenta leer la primera línea (`line = file.readline()`). El método `.readline()` lee una línea cada vez que se llama, abarcando desde el final del texto ya leído hasta la siguiente marca de fin de línea (`'\n'`). Si ya se han leído todas las líneas del archivo, devuelve una string vacía. *Nótese que dentro del archivo no hay líneas que sean strings vacías, ya que una línea vacía incluye una marca de fin de línea.*
3. Se inicia un bucle (`while line != "":`) que concluirá cuando `.readline()` devuelva una string vacía, indicando que se ha acabado el archivo. En este bucle...
 - a. Se procesa la última línea leída (`print(line, end="")`)
 - b. Se intenta leer la siguiente línea (`line = file.readline()`)

En la llamada a `print()` se ha incluido el parámetro `end=""` porque la string `line` incluye la marca de fin de línea leída del archivo y se quiere evitar que `print()` añada una línea vacía. La siguiente tabla muestra como quedaría la salida con y sin dicho parámetro.

Contenido del archivo	<code>print(line, end="")</code>	<code>print(line)</code>
Twinkle, twinkle, little star, how I wonder what you are. Up above the world so high, like a diamond in the sky.	Twinkle, twinkle, little star, how I wonder what you are. Up above the world so high, like a diamond in the sky.	Twinkle, twinkle, little star, how I wonder what you are. Up above the world so high, like a diamond in the sky.

Si se quiere preparar la string para un tratamiento en el que no se desea que intervenga la marca de fin de línea, se podría usar el método `.rstrip()`:

```
def show_file(filename):
    file = open(filename, 'r')
    line = file.readline().rstrip()

    while line != "":
        print(line)
        line = file.readline().rstrip()

    file.close()
```

El método `.rstrip()` elimina las marcas de fin de línea y los espacios, tabuladores o equivalentes que pudiera haber al final de la string, respetando los que hubiese al principio. Si se quieren eliminar todos, al principio y al final, se usaría el método `.strip()`. Los métodos `.strip()`, `.rstrip()`, y su complementario `.lstrip()` pueden tener un parámetro opcional: una string indicando el repertorio concreto de caracteres que se quiere eliminar.

Al leer líneas de un archivo, puede ser interesante ver su contenido real (todos los caracteres, incluido el final de las líneas). La función `repr()` es útil para este propósito, pues muestra los valores en un formato que se podría usar para introducirlos como literales en el código Python:

```
def show_file(filename):
    file = open(filename, 'r')
    line = file.readline()

    while line != "":
        print(repr(line))
        line = file.readline()

    file.close()
```

```
Twinkle, twinkle, little star,\n
How I wonder what you are.\n
Up above the world so high,  \t\n
Like a diamond in the sky.\n
```

Observe que la tercera línea contiene más espacios y un tabulador antes del `\n` final, que con un simple `print()` no se podrían apreciar.

Tratamiento explícito como secuencia de líneas en Python

Nótese que este "esquema estándar de tratamiento de un fichero de texto en modo lectura" se corresponde exactamente con el "esquema de recorrido basado en una condición", tal como se

vio en el tema de Secuencias; un fichero de texto se puede interpretar como una secuencia de líneas (también como una secuencia de caracteres). Python ofrece la opción de considerar explícitamente el fichero como una secuencia de líneas y aplicar un "esquema general de recorrido":

```
def show_file(filename):
    file = open(filename, 'r')

    for line in file:
        line = line.rstrip()
        print(line)

    file.close()
```

Leer un fichero en una única string

A veces, normalmente con ficheros no muy grandes y con características específicas, se necesita leer el contenido de un archivo de entrada como una sola string:

```
file = open('poem.txt', 'r')
text = file.read()
print(len(text))    # total number of characters
print(text)         # contents of the file
file.close()
```

El método `.read()` puede tener como parámetro el número de caracteres a leer:

```
file = open('poem.txt', 'r')
print(file.read(1))    # first character
print(file.read(5))    # next 5 characters
print(file.read())     # the rest of the file's contents
file.close()
```

Esquema de escritura

El siguiente ejemplo muestra un esquema para volcar en un fichero información contenida en una secuencia de datos, por ejemplo, una lista:

```
def save_data(sequence, filename):
    file = open(filename, 'w')

    for item in sequence:
        print(item, file=file)

    file.close()
```

El fichero se abre en modo escritura, usando el parámetro `'w'`, y el bucle, siempre, recorre el origen de los datos, que en este caso es la secuencia, no el fichero, que es su destino. Para escribir en el archivo, usamos la función `print()`, especificando el parámetro `file`. Como alternativa, se puede usar el método `.write()`, como en el siguiente ejemplo.

```
def save_data(sequence, filename):
    file = open(filename, 'w')

    for item in sequence:
        file.write(item + '\n')

    file.close()
```

Nótese que se concatena al *item* una marca de fin de línea porque el método *.write()* no la añade automáticamente, a diferencia de la función *print()*, a la que habría que indicarle que no la añadiera si se quisiera conseguir el mismo efecto, que sería que todos los ítems se escribirían pegados en una única línea.

En todos estos ejemplos estamos suponiendo que *item* es de tipo *str*; si no lo fuera, habría que obtener su conversión a *str* antes de escribirlo: *str(item)*.

El siguiente ejemplo, aplica el mismo esquema para copiar un fichero, que se toma como secuencia de entrada:

<pre>def copy(in_name, out_name): origin = open(in_name, 'r') target = open(out_name, 'w') for item in origin: print(item, file=target) origin.close() target.close()</pre>	<pre>def copy(in_name, out_name): origin = open(in_name, 'r') save_data(origin, out_name) origin.close()</pre>
---	--

La versión de la derecha modulariza el problema, apoyándose en la función *save_data()*, en vez de manejar directamente los dos ficheros.

Al usar el modo de apertura *'w'* se está creando un archivo nuevo, eliminando la información que pudiese tener en caso de que ya existiese. Si lo se cambiara por *'a'*, se estaría abriendo el archivo en modo adición (*append*) por lo que, si el fichero ya existe, se respetará la información que tenga y la nueva se empezará a añadir al final; si el fichero no existe, se creará vacío.

Cláusula *with*

La cláusula *with* en Python es una forma de manejar recursos que necesitan ser "gestionados" de manera eficiente, como archivos, conexiones de red, bases de datos, entre otros. Es especialmente útil cuando se quiere asegurar que ciertos procesos se realicen antes y después de ejecutar un bloque de código, como **cerrar un archivo después de haberlo utilizado** o liberar recursos de una base de datos. La cláusula *with* asegura que se ejecuten métodos de entrada y salida de un contexto, incluso si ocurre un error en el bloque de código.

```
def save_data(sequence, filename):
    with open(filename, 'w') as file:
        for item in sequence:
            print(item, file=file)
```

Al usar la cláusula *with* no es necesario llamar explícitamente al método *.close()*. La cláusula *with* abre el archivo usando la función *open()* y lo cierra automáticamente al terminar su ámbito. La

asignación del objeto manejador del archivo devuelto por `open()` a la variable que lo va a referenciar (*file* en el ejemplo) se hace usando la palabra *as*, en vez del operador de asignación.

Se pueden abrir varios recursos anidando cláusulas `with` o en una única cláusula `with`, separándolos por comas:

```
def copy(in_name, out_name):
    with open(in_name, 'r') as origin,
        open(out_name, 'w') as target:
        for item in origin:
            print(item, file=target)
```

Anexo – resumen de las operaciones de manejo de ficheros de texto

Abrir	<code>file = open(filename, mode='r', encoding=default)</code> <code>with open(filename, mode='r', encoding=default) as file:</code> <i>los parámetros mode y encoding pueden omitirse</i>	
Modos de apertura (valores para mode)	'r'	Abre un archivo en modo lectura. El archivo debe existir o se genera un error.
	'w'	Abre un archivo en modo escritura. Si no existe se crea; si existe se sobrescribe.
	'a'	Abre un archivo para añadir contenido. Si no existe se crea, si existe posiciona el curso de escritura al final, sin sobrescribir el contenido existente.
Leer	<code>line = file.readline()</code> <code>text= file.read()</code>	
Escribir	<code>print(data, file=file)</code> <code>file.write(data)</code>	<i>atentos a los '\n' (print() los incluye, .write() no)</i>
Cerrar	<code>file.close()</code> <i>no es necesario si se abre con with</i>	

Anexo- el módulo os

El módulo *os* de Python proporciona una forma de interactuar con el sistema operativo, permitiendo realizar tareas como manipular archivos y directorios, obtener información sobre el entorno de ejecución, trabajar con rutas de archivos y más. Para usarlo, antes hay que importarlo (`import os`). Entre otras, ofrece las siguientes funciones:

os.getcwd()	Obtiene el directorio de trabajo actual.
<code>print(os.getcwd()) # Muestra la ruta del directorio actual.</code>	
os.chdir(path)	Cambia el directorio de trabajo actual a path.
<code>os.chdir('/nuevo/directorio')</code>	
os.mkdir(path)	Crea un nuevo directorio.
<code>os.mkdir('nuevo_directorio')</code>	
os.makedirs(path)	Crea un directorio, y si no existen los directorios intermedios, los crea.
<code>os.makedirs('directorio/subdirectorio')</code>	
os.rmdir(path)	Elimina un directorio vacío.
<code>os.rmdir('directorio_vacio')</code>	
os.remove(path)	Elimina un archivo
<code>os.remove('archivo.txt')</code>	
os.listdir(path)	Lista los archivos y directorios en un directorio específico.
<code>archivos = os.listdir('.')</code>	

<code>print(archivos)</code>	
<code>os.path.join(*paths)</code>	Une diferentes partes de una ruta de manera correcta, teniendo en cuenta el sistema operativo.
<code>ruta = os.path.join('directorio', 'archivo.txt')</code> <code>print(ruta) # Resultado: 'directorio\archivo.txt' (en Windows)</code>	
<code>os.path.exists(path)</code>	Verifica si una ruta existe.
<code>existe = os.path.exists('archivo.txt')</code> <code>print(existe) # True o False</code>	
<code>os.path.isdir(path)</code>	Verifica si una ruta es un directorio.
<code>es_directorio = os.path.isdir('directorio')</code> <code>print(es_directorio) # True o False</code>	

Glosario de términos de Fundamentos de Programación I

El glosario está organizado alfabéticamente y prioriza los términos que resultan necesarios para comprender los nueve temas de los apuntes. La columna Tema / páginas permite localizar cada concepto en la fuente. Tras el glosario general se incluye una tabla de consulta rápida con las operaciones de Python que se presentan en el documento; así se distingue entre los conceptos y los nombres de herramientas concretas. El glosario se ha generado automáticamente usando Manus AI.

Glosario alfabético

A

Término	Definición contextual	Tema / páginas
Algoritmo	Conjunto ordenado, finito y bien definido de instrucciones que permite resolver un problema. En programación, especifica qué datos se requieren y qué pasos producen un resultado.	2 / 26–28
Alias	Situación en la que dos o más variables referencian el mismo objeto en memoria. Si el objeto es mutable y se modifica por uno de esos nombres, el cambio se observa desde los demás.	4 / 98–100
Anotación de tipos	Información escrita junto a un parámetro, una variable o un valor de retorno para indicar el tipo que se espera. En Python informa y documenta, pero no impone ese tipo durante la ejecución.	2–3 / 56, 78–79
Archivo binario	Archivo cuyos octetos codifican datos que no se interpretan necesariamente como caracteres de texto, por ejemplo imágenes o estructuras de otro tipo.	9 / 164
Archivo de texto	Archivo cuyo contenido se interpreta como una secuencia de caracteres, habitualmente organizada en líneas terminadas con el carácter de nueva línea <code>\n</code> .	9 / 164–166
Asignación	Instrucción que evalúa la expresión situada a la derecha de <code>=</code> y asocia su resultado con la variable situada a la izquierda.	2 / 41–44
Asignación múltiple	Asignación que relaciona, en una sola instrucción, varios valores con varias variables mediante la correspondencia de sus posiciones.	2 / 44–45
Atributo	Componente direccionable de un registro que posee un significado propio; por ejemplo, el nombre o la fecha de nacimiento de una persona.	1 / 19–20

B

Término	Definición contextual	Tema / páginas
Backreference o referencia inversa	Referencia a un grupo capturado previamente por una expresión regular, como <code>\1</code> o <code>\2</code> . Permite reutilizar la parte de texto ya encontrada en el propio patrón o en una sustitución.	8 / 157–158
Base de datos	Conjunto estructurado de archivos o datos relacionados que se organizan para almacenarlos y procesarlos posteriormente.	1 / 18–21
BIOS / firmware	Software de bajo nivel almacenado de forma no volátil que inicializa y controla componentes básicos del equipo.	1 / 10–12
Bucle infinito	Iteración que no alcanza una condición de terminación y, por tanto, continúa indefinidamente hasta que se interrumpe desde fuera o mediante una instrucción específica.	3 / 68–69
Búsqueda dicotómica	Algoritmo de búsqueda sobre una lista ordenada que compara un elemento central y descarta, en cada paso, aproximadamente la mitad del espacio de búsqueda.	5 / 120–122
Búsqueda exacta	Búsqueda de una secuencia literal conocida dentro de una cadena, como la realizada con <code>find()</code> . Se diferencia de la búsqueda basada en patrones.	8 / 152
Búsqueda lineal	Recorrido secuencial que examina los elementos uno a uno hasta hallar el que cumple un criterio o hasta llegar al final de la secuencia.	4 / 91–94

C

Término	Definición contextual	Tema / páginas
Cardinalidad	Número de elementos de una colección o secuencia. En Python suele obtenerse con <code>len()</code> .	4 y 6 / 86, 128
Caso base	Caso de un problema recursivo que no admite más subdivisión y se resuelve directamente. Garantiza que las llamadas recursivas terminen.	5 / 115–116

Término	Definición contextual	Tema / páginas
Caso general	Regla de una solución recursiva que transforma el problema en una o varias versiones más pequeñas del mismo, convergentes hacia un caso base.	5 / 115–116
Clase de caracteres	Categoría de caracteres que puede usarse en una expresión regular, como <code>\d</code> para dígitos, <code>\w</code> para alfanuméricos o <code>\s</code> para separadores.	8 / 152–154
Clave	Valor inmutable que identifica y permite recuperar un dato dentro de un diccionario.	6 / 124–126
Clave primaria	Clave elegida entre las claves mínimas de una tabla o archivo de datos para identificar de manera única cada registro.	1 / 20–21
Código máquina	Conjunto de instrucciones binarias específicas que puede comprender y ejecutar directamente el procesador.	2 / 29–31
Codificación	Regla o estándar que determina cómo se representan los caracteres mediante octetos en un archivo de texto.	9 / 164
Compilador	Programa que traduce un programa completo escrito en un lenguaje de alto nivel a un ejecutable o a código máquina antes de su ejecución.	2 / 30–32
Comprensión de listas	Sintaxis abreviada para crear una lista a partir de una expresión, una cláusula <code>for</code> y, opcionalmente, una condición <code>if</code> .	4 / 101–102
Concatenación	Operación que une secuencias del mismo tipo, como cadenas, listas o tuplas. En Python se expresa habitualmente con <code>+</code> .	4 y 7 / 87, 140–141
Condición	Expresión cuyo resultado es <code>True</code> o <code>False</code> y que sirve para decidir qué instrucciones se ejecutan o si una iteración continúa.	3 / 64–69
Conjunto o <u>set</u>	Colección mutable, sin orden definido y sin elementos repetidos. Sus elementos deben ser valores inmutables.	6 / 127–130

Término	Definición contextual	Tema / páginas
Contenedor	Estructura de datos que almacena y organiza valores. Las listas, tuplas, cadenas, diccionarios y conjuntos son ejemplos de contenedores.	4 / 84–86
Contenedor asociativo	Estructura que permite acceder a sus elementos mediante una clave, no mediante una posición o índice. El diccionario es el ejemplo principal.	6 / 124
Copia profunda o <u>deep copy</u>	Copia que duplica un contenedor y todos los niveles de objetos que contiene, evitando que los objetos anidados permanezcan compartidos.	4 / 99–100
Copia superficial o <u>shallow copy</u>	Copia que duplica solamente el primer nivel de un contenedor; los objetos mutables anidados pueden continuar siendo compartidos.	4 / 99–100
CPU o microprocesador	Componente que interpreta y ejecuta instrucciones, realiza operaciones aritmético-lógicas y coordina la actividad básica del ordenador.	1 / 7–10

D

Término	Definición contextual	Tema / páginas
Dato	Representación simbólica de una característica, atributo o variable. Puede ser numérico, alfabético, algorítmico o espacial.	1 / 3–5
Depuración	Proceso de localizar, analizar y corregir errores de un programa mediante pruebas, ejecución controlada, inspección de valores y puntos de parada.	2 / 59–63
Diferencia de conjuntos	Operación cuyo resultado contiene los elementos del primer conjunto que no pertenecen al segundo. En Python usa <code>_</code> .	6 / 130–131
Diferencia simétrica	Operación que reúne los elementos que pertenecen a uno u otro conjunto, pero no a ambos. En Python usa <code>^</code> .	6 / 130–131
Diccionario o <u>dict</u>	Colección de parejas clave–valor. Sus claves son inmutables y permiten recuperar los valores asociados sin depender de una posición numérica.	6 / 124–127

Término	Definición contextual	Tema / páginas
Diseño descendente	Técnica de resolución que descompone un problema en subproblemas más pequeños, resuelve cada uno por separado y combina las soluciones obtenidas.	5 / 112
Docstring	Cadena de documentación colocada al inicio del cuerpo de una función, módulo o clase para explicar su finalidad y uso.	3 / 75–76
Dominio	Conjunto de valores válidos que puede tomar un atributo de una estructura de datos o una base de datos.	1 / 19–20

E

Término	Definición contextual	Tema / páginas
Eficiencia recursiva	Relación entre una solución recursiva y los recursos que consume, especialmente número de llamadas, memoria ocupada y tiempo de ejecución.	5 / 117–119
Empaquetado	Creación de una secuencia a partir de varios valores, por ejemplo, al agruparlos en una tupla.	4 / 88–89
Expresión	Combinación de valores, variables, operadores y llamadas que se evalúa para producir un resultado.	2 / 50–54
Expresión regular	Patrón de texto expresado mediante un lenguaje con caracteres literales, metacaracteres y secuencias de escape; sirve para buscar, verificar o sustituir texto.	8 / 152–163
Estructura de control	Construcción que determina el orden en que se ejecutan instrucciones, principalmente secuencias, selecciones e iteraciones.	3 / 64–71
Estructura multidimensional	Contenedor cuyos elementos pueden ser otros contenedores, formando varios niveles o dimensiones de datos.	4 / 106–109
Evaluación cortocircuito en	Comportamiento de <u>and</u> y <u>or</u> por el que dejan de evaluar operandos cuando el resultado lógico ya está determinado.	2 / 54–55

F

Término	Definición contextual	Tema / páginas
<u>f-string</u>	Literal de cadena precedido por <code>f</code> o <code>F</code> que permite intercalar expresiones entre llaves y formatear su resultado.	7 / 145–149
Función	Bloque de código reutilizable, definido con un nombre, parámetros y cuerpo, que se ejecuta cuando se invoca para realizar una tarea.	3 / 72–79
Función polimórfica	Función que puede aplicarse a valores de distintos tipos cuando las operaciones de su cuerpo resultan compatibles con dichos valores.	3 / 78
Formato o de modificador de formato	Especificación usada en una <u>f-string</u> o con <code>format()</code> para controlar ancho, alineación, relleno, precisión y representación de un valor.	7 / 146–150
<u>for</u>	Estructura de iteración que recorre los elementos de una secuencia o los valores de un rango.	3 / 69–71

G

Término	Definición contextual	Tema / páginas
Grupo de captura	Parte de una expresión regular encerrada entre paréntesis que se trata como una unidad y cuyo texto coincidente queda disponible para consultarlo o reutilizarlo.	8 / 157–158
Grupo no capturante	Agrupación de una expresión regular escrita como <code>(?:...)</code> ; organiza el patrón como una unidad, pero no guarda esa parte como grupo recuperable.	8 / 157

H

Término	Definición contextual	Tema / páginas
Hardware	Conjunto de componentes físicos de un sistema informático, incluidos dispositivos electrónicos, mecánicos y de almacenamiento.	1 / 5–7

I

Término	Definición contextual	Tema / páginas
Identificador	Nombre que se asigna a una variable, función u otro elemento del programa y que debe respetar las reglas sintácticas del lenguaje.	2 / 39–41
Índice	Posición de un elemento dentro de una secuencia. Python numera desde cero y también permite índices negativos desde el final.	4 / 86–88
Información	Datos procesados, organizados e interpretados de forma que transmiten un significado útil a quien los recibe.	1 / 3–5
Inmersión recursiva	Estrategia que generaliza un problema para formular una solución recursiva más eficaz; el problema original queda incluido como caso particular.	5 / 122–123
Inmutable	Propiedad de un objeto que impide alterar sus elementos después de crearlo. Las cadenas y las tuplas son inmutables.	4 / 96
Intérprete	Programa que traduce y ejecuta instrucciones de un lenguaje de alto nivel durante cada ejecución del programa, normalmente de manera progresiva.	2 / 30–32
Interpolación de cadenas	Inserción en un texto de los valores producidos por expresiones, como sucede dentro de las llaves de una <u>f-string</u> .	7 / 145–146
Iteración	Repetición controlada de un conjunto de instrucciones. Puede realizarse según una condición o recorriendo una secuencia.	3 / 64–71

L

Término	Definición contextual	Tema / páginas
Lista o <u>list</u>	Secuencia mutable y ordenada de elementos, creada normalmente con corchetes. Admite añadir, cambiar y eliminar elementos después de crearla.	4 / 84–86, 96–98

Término	Definición contextual	Tema / páginas
Lista encadenada	Estructura formada por nodos; cada nodo conserva un valor y una referencia al siguiente. Su acceso es secuencial, no directo por posición.	5 / 121–122
Literal	Valor escrito de forma explícita en el código fuente, como <u>42</u> , <u>3.14</u> , <u>True</u> o <u>"Hola"</u> .	2 / 41–42

M

Término	Definición contextual	Tema / páginas
Manejador de archivo	Objeto devuelto por <u>open()</u> que representa el flujo de datos entre un programa y un archivo, y ofrece operaciones de lectura, escritura y cierre.	9 / 165
Matriz	Estructura bidimensional compuesta normalmente por listas de listas, organizada en filas y columnas y accesible mediante dos índices, como <u>m[i][j]</u> .	4 / 106–109
Memoria RAM	Memoria de trabajo de acceso aleatorio que mantiene programas y datos mientras se ejecutan. Es volátil, por lo que pierde su contenido al apagarse el equipo.	1 / 10–12
Metacarácter	Carácter que tiene un significado especial dentro de una expresión regular, por ejemplo <u>*</u> , <u>+</u> , <u>?</u> , <u>()</u> , <u>[]</u> , <u>^</u> o <u>\$</u> .	8 / 152–157
Módulo	Archivo de Python, habitualmente con extensión <u>.py</u> , que agrupa funciones, variables u otros elementos reutilizables y proporciona un espacio de nombres.	3 / 80–82
Mutable	Propiedad de un objeto que permite modificar su contenido tras haberlo creado. Las listas, los diccionarios y los conjuntos son mutables.	4 y 6 / 96–100, 128

N

Término	Definición contextual	Tema / páginas
<u>None</u>	Valor especial de Python que representa la ausencia de un valor significativo. Una función sin <u>return</u> explícito devuelve <u>None</u> .	3 / 76–77

P

Término	Definición contextual	Tema / páginas
Paquete	Agrupación organizada de módulos de Python dentro de una carpeta, utilizada para estructurar programas de mayor tamaño.	3 / 82–83
Parámetro formal	Nombre declarado en la cabecera de una función para recibir un valor cuando esta se invoca.	3 / 73–75
Parámetro real o argumento	Valor o expresión que se proporciona en una llamada a función y que se asocia con un parámetro formal.	3 / 73–75
Patrón	Descripción de una forma textual que puede corresponder a diferentes textos concretos, como una fecha, un correo electrónico o un número de teléfono.	8 / 152
Periférico	Dispositivo conectado al ordenador que facilita entrada, salida, almacenamiento o comunicación de datos, por ejemplo, teclado, pantalla o impresora.	1 / 7–13
Precedencia	Regla que determina qué operadores se evalúan antes cuando aparecen varios dentro de una misma expresión.	2 / 52–54
Profundidad de recursión	Número de instancias de una función recursiva que permanecen activas desde la llamada inicial hasta que se alcanza un caso base.	5 / 117–118
Programa	Implementación de un algoritmo en un lenguaje de programación, destinada a ser ejecutada por un ordenador.	2 / 26–29
Programación estructurada	Enfoque que organiza los programas mediante estructuras de control y funciones para mejorar la claridad, la corrección y el mantenimiento.	3 / 64–83

R

Término	Definición contextual	Tema / páginas
<u>range()</u>	Función que genera una secuencia de enteros para controlar iteraciones, normalmente indicando inicio, fin exclusivo y, opcionalmente, paso.	3 / 69–71

Término	Definición contextual	Tema / páginas
Raw string o cadena sin procesar	Cadena con prefijo <code>r</code> que evita que Python interprete sus secuencias de escape antes de que las use una expresión regular.	8 / 158–159
Recursividad	Técnica de resolución en la que un problema se expresa mediante versiones más pequeñas de sí mismo y se resuelve mediante llamadas a la misma función.	5 / 112–116
Registro	Conjunto de atributos que describe una entidad o instancia concreta dentro de una estructura de datos, como una fila de una tabla.	1 / 19–20
Relación de recurrencia	Regla matemática o lógica que define la solución de un caso de un problema a partir de soluciones de casos más pequeños.	5 / 115–116
Repetición	Operación que reproduce una secuencia un número determinado de veces; en Python suele expresarse con <code>*</code> .	4 y 7 / 87, 140–141
Ruta absoluta	Dirección completa de un archivo o directorio, expresada desde la raíz del sistema de archivos.	9 / 165
Ruta relativa	Dirección de un archivo o directorio expresada respecto al directorio actual de trabajo.	9 / 165

S

Término	Definición contextual	Tema / páginas
Selección	Estructura de control que elige una alternativa de ejecución según el resultado de una condición, como <code>if</code> , <code>elif</code> y <code>else</code> .	3 / 64–67
Secuencia	Colección lineal cuyos elementos se disponen en un orden y pueden recorrerse. Las cadenas, listas y tuplas son secuencias en Python.	4 / 84–86
Secuencia de escape	Combinación iniciada por barra invertida que representa un carácter especial, como <code>\n</code> para nueva línea o <code>\t</code> para tabulación.	7 / 138–139

Término	Definición contextual	Tema / páginas
Sistema de Gestión de Archivos (SGA)	Parte del sistema operativo encargada de organizar, almacenar, manipular y recuperar archivos y directorios.	9 / 164–165
Sistema informático	Conjunto formado por ordenadores, software, datos, comunicaciones, personas y procedimientos que permite tratar información con un objetivo.	1 / 5–7
Slice o segmento	Subsecuencia obtenida mediante la notación <u>[inicio:fin:paso]</u> . En las secuencias inmutables y listas, el documento lo presenta como una nueva copia de la zona seleccionada.	4 / 87–88
Software	Conjunto de programas y datos que indican al hardware qué operaciones debe realizar.	1 / 5–7
Subcadena o substring	Secuencia de caracteres contenida dentro de otra cadena, que puede localizarse, extraerse o reemplazarse.	7 / 140–144
Subconjunto	Conjunto cuyos elementos pertenecen todos a otro conjunto. La inclusión puede ser estricta o admitir que ambos conjuntos sean iguales.	6 / 129–130

T

Término	Definición contextual	Tema / páginas
Tipo de dato	Clasificación de valores que determina su representación, las operaciones válidas y el modo en que se interpretan. Entre los básicos de Python figuran <u>int</u> , <u>float</u> , <u>bool</u> y <u>str</u> .	2 / 45–50
Tipado dinámico	Modelo de Python en el que una variable toma el tipo del valor que referencia y puede asociarse posteriormente a un valor de otro tipo.	2 / 47–50
Tupla o <u>tuple</u>	Secuencia ordenada e inmutable de elementos, creada habitualmente con paréntesis.	4 / 84–86, 96

U–V–Z

Término	Definición contextual	Tema / páginas
Unión de conjuntos	Operación que produce un conjunto con todos los elementos de los conjuntos de partida, sin repeticiones. En Python se expresa con `	`.
Valor	Dato concreto con el que trabaja un programa y al que una variable puede hacer referencia.	2 / 41–50
Variable	Identificador que referencia un valor almacenado o manejado durante la ejecución de un programa.	2 / 39–44
<u>while</u>	Estructura de iteración que repite un bloque de instrucciones mientras una condición se mantenga verdadera.	3 / 67–69
<u>zip()</u>	Función que permite recorrer dos o más secuencias en paralelo, produciendo tuplas con los elementos de una misma posición hasta la longitud de la secuencia más corta.	4 / 103–104
<u>zip_longest()</u>	Función de <u>itertools</u> que recorre varias secuencias hasta la longitud de la más larga y rellena los elementos ausentes con <u>fillvalue</u> o con <u>None</u> .	4 / 104–105

Términos complementarios de alta recurrencia

La tabla siguiente completa el glosario con conceptos que aparecen de forma reiterada en los apuntes y resultan especialmente útiles para interpretar ejemplos y ejercicios. Se agrupan aquí como referencia práctica adicional.

Término	Definición contextual	Tema / páginas
Acumulador	Variable que conserva un resultado parcial y lo actualiza al recorrer los elementos de una secuencia; por ejemplo, una suma o un producto.	4 / 90–94
Ámbito	Parte del programa en la que un identificador existe y puede utilizarse. Los parámetros y variables creados dentro de una función tienen, normalmente, ámbito local.	3 / 76–77

Término	Definición contextual	Tema / páginas
<u>break</u>	Instrucción que interrumpe inmediatamente el bucle más interno y transfiere la ejecución a la instrucción posterior al bucle.	3 / 70–71
<u>continue</u>	Instrucción que finaliza la iteración actual de un bucle y continúa con la siguiente, si la hubiera.	3 / 70–71
Chipset	Conjunto de circuitos de la placa base que gestiona y coordina la comunicación entre componentes del ordenador.	1 / 8–10
Código fuente	Texto de un programa escrito por una persona en un lenguaje de programación antes de su traducción o ejecución.	2 / 26–32
Consola terminal	o Interfaz textual desde la que pueden ejecutarse órdenes y programas, así como introducir y consultar datos.	1 / 22–24
Conversión de tipos	Creación de un valor de un tipo a partir de otro, mediante funciones como <u>int()</u> , <u>float()</u> , <u>complex()</u> o <u>str()</u> , cuando la conversión es válida.	2 / 46–50
Contador	Tipo de acumulador que aumenta o disminuye en una cantidad fija cada vez que se cumple una condición o se procesa un elemento objetivo.	4 / 90–94
Depurador	Herramienta que ayuda a ejecutar un programa paso a paso, observar variables y detenerse en puntos concretos para localizar errores.	2 / 59–63
Directorio actual de trabajo	Carpeta desde la que se considera que se ejecuta un programa y respecto de la cual se interpretan las rutas relativas.	9 / 165
Espacio nombres de	Contexto que asocia identificadores con objetos y evita confusiones entre nombres definidos en módulos, funciones u otros ámbitos.	3 / 80–82
Flujo de entrada / salida	Canal lógico por el que los datos se transfieren desde una fuente al programa o desde el programa a un destino, como un archivo.	9 / 165

Término	Definición contextual	Tema / páginas
HDD	Unidad de almacenamiento no volátil que guarda datos mediante tecnología magnética en discos.	1 / 10–12
IDE	Entorno integrado de desarrollo que reúne herramientas para escribir, ejecutar, depurar y organizar programas.	1 / 22–24
<u>if</u> / <u>elif</u> / <u>else</u>	Palabras clave que forman estructuras de selección: ejecutan un bloque si una condición es verdadera, consideran alternativas adicionales o proporcionan una alternativa final.	3 / 64–67
<u>input()</u>	Función que obtiene texto introducido por la persona usuaria a través de la entrada estándar.	2 / 34–36
Lenguaje de alto nivel	Lenguaje más cercano a la forma de pensar y expresar problemas de las personas que a las instrucciones específicas del procesador.	2 / 29–32
Lenguaje de bajo nivel	Lenguaje cercano a la arquitectura del ordenador, como el lenguaje ensamblador o el código máquina.	2 / 29–32
Lenguaje de programación	Lenguaje formal que proporciona reglas y construcciones para representar datos, procesos y algoritmos ejecutables.	2 / 26–32
Memoria volátil	Almacenamiento que conserva la información cuando el equipo deja de recibir energía, como un disco o una memoria de firmware.	1 / 10–12
Módulo <u>re</u>	Biblioteca estándar de Python que proporciona funciones y objetos para trabajar con expresiones regulares.	8 / 158–163
<u>main</u>	Nombre que adopta el módulo que inicia la ejecución de un programa Python; permite distinguirlo de un módulo que se importa.	3 / 81–82
Operador	Símbolo o palabra que indica una operación sobre uno o más operandos, como una suma, comparación o operación lógica.	2 / 50–55
Operando	Valor, variable o expresión sobre el que actúa un operador.	2 / 50–55

Término	Definición contextual	Tema / páginas
Punto de parada o breakpoint	Marca de depuración que detiene temporalmente la ejecución de un programa para examinar su estado.	2 / 59–63
<u>print()</u>	Función que muestra valores por la salida estándar; también puede escribirlos en un archivo mediante su parámetro <u>file</u> .	2 / 33–36; 9 / 168
<u>return</u>	Instrucción que finaliza una función y, opcionalmente, entrega un valor al punto desde el que fue llamada.	3 / 76–77
SGBD o DBMS	Sistema de gestión de bases de datos: software que media entre aplicaciones y datos almacenados, facilitando consulta, control e integridad.	1 / 20–21
Sistema operativo	Software básico que administra recursos del hardware y ofrece servicios para programas y personas usuarias, entre ellos la gestión de archivos.	1 / 12–16; 9 / 164–165
SSD	Unidad de almacenamiento no volátil basada en memoria de estado sólido, sin partes mecánicas móviles.	1 / 10–12
<u>str</u> o cadena de caracteres	Tipo de dato de Python que representa una secuencia inmutable de caracteres.	2 / 45–50; 7 / 138–151
<u>bool</u>	Tipo de dato lógico cuyos valores son <u>True</u> y <u>False</u> .	2 / 45–50
<u>complex</u>	Tipo de dato que representa números complejos en Python.	2 / 45–50
<u>float</u>	Tipo de dato que representa números reales en coma flotante.	2 / 45–50
<u>int</u>	Tipo de dato que representa números enteros.	2 / 45–50
Valor por omisión	Valor que se asigna a un parámetro formal cuando una llamada no proporciona un argumento para ese parámetro.	3 / 74–75
Variable local	Variable creada dentro de una función o correspondiente a un parámetro; solo existe durante la ejecución de esa función.	3 / 76–77

Término	Definición contextual	Tema / páginas
<u>with</u>	Construcción de Python para gestionar recursos, como archivos, asegurando la salida del contexto y el cierre automático del recurso al terminar el bloque.	9 / 169

Operaciones y herramientas de consulta rápida

La siguiente tabla reúne las operaciones más relevantes mencionadas en los apuntes. No sustituye las definiciones anteriores: sirve como recordatorio práctico para reconocer la terminología de código.

Área	Operación o elemento	Finalidad principal
Listas	<u>append(x)</u> , <u>insert(i, x)</u> , <u>extend(seq)</u>	Añaden, respectivamente, un elemento al final, un elemento en una posición y todos los elementos de otra secuencia.
Listas	<u>pop([i])</u> , <u>remove(x)</u> , <u>del</u>	Eliminan por posición y devuelven el elemento, eliminan la primera coincidencia de un valor, o borran un elemento o segmento.
Diccionarios	<u>d[k]</u> , <u>get(k, defecto)</u> , <u>pop(k)</u>	Recuperan un valor por clave, lo recuperan sin error con valor alternativo opcional, o eliminan un par devolviendo su valor.
Diccionarios	<u>keys()</u> , <u>values()</u> , <u>items()</u>	Proporcionan vistas iterables de las claves, los valores y las parejas clave-valor.
Conjuntos	<u>add(x)</u> , <u>update(iterable)</u> , <u>discard(x)</u>	Añaden un elemento, incorporan elementos de otra colección o eliminan un elemento sin error si no existe.
Cadenas	<u>find</u> , <u>index</u> , <u>rfind</u> , <u>rindex</u>	Localizan la primera o la última ocurrencia de una subcadena; <u>find</u> devuelve -1 y <u>index</u> genera un error si no hay coincidencia.
Cadenas	<u>join</u> , <u>split</u> , <u>replace</u>	Unen cadenas con un separador, dividen una cadena en partes o devuelven una cadena con sustituciones.

Área	Operación o elemento	Finalidad principal
Cadenas	<u>strip</u> , <u>lstrip</u> , <u>rstrip</u>	Eliminan caracteres en los extremos, solo a la izquierda o solo a la derecha.
Expresiones regulares	<u>re.match</u> , <u>re.fullmatch</u> , <u>re.search</u>	Comprueban coincidencia al comienzo, en todo el texto o en cualquier posición, respectivamente.
Expresiones regulares	<u>re.finditer</u> , <u>re.sub</u> , <u>re.subn</u>	Iteran las coincidencias no solapadas, sustituyen coincidencias o devuelven además el número de sustituciones.
Archivos	<u>open(nombre, modo, encoding=...)</u>	Abre un archivo y devuelve su manejador. Los modos usuales son ' <u>r</u> ', ' <u>w</u> ' y ' <u>a</u> '.
Archivos	<u>readline</u> , <u>read</u> , <u>write</u> , <u>close</u>	Leen una línea, leen contenido, escriben texto sin salto automático y cierran el manejador.
Archivos	<u>with open(...) as archivo</u>	Gestiona el recurso dentro de un contexto y asegura el cierre del archivo al salir del bloque, incluso si se produce un error.
Sistema operativo	<u>os.getcwd</u> , <u>os.chdir</u> , <u>os.path.join</u> , <u>os.path.exists</u>	Consultan o cambian el directorio de trabajo y permiten construir o comprobar rutas de manera portable.

Observación final

Las entradas de este glosario se han seleccionado por su valor conceptual y por su recurrencia en los ejercicios de programación. Para estudiar, conviene relacionar cada término con un ejemplo de código del tema correspondiente: comprender la definición y reconocer su uso son habilidades complementarias.